Guide Run — Mode d'emploi pratique des runs Zêta

Statut : modèle initial (v0.7), enrichi cas par cas (v0.16). Document de travail en français ; la version définitive sera produite en anglais à la fin de la série de tests.

Projet : Riemann_Lab · **Auteur :** hprzeta · **Sources :** `Guide_Scripts_Cluster_Zeta.md` (état du 17/10/2026) et captures réelles du 24/10/2026 sur PC1 (zeta-lab).

0. Comment lire ce guide

Chaque fait est étiqueté pour qu'on sache sur quoi il repose :

Étiquette	Sens
mesuré	visible sur une capture ou une sortie de terminal de ce guide
guide	affirmé par `Guide_Scripts_Cluster_Zeta.md`, non revérifié ici
déduit	raisonnement à partir de ce qu'on voit ; à confirmer par une commande indiquée
à tester	pas encore essayé : sera traité dans un cas de test
rapporté	tiré d'une session Claude Code (lecture des sessions et de `git log`), non revérifié ici
vérifié	relu dans le dépôt, par `git` ou par `sha256sum`, avec la preuve indiquée (ajouté en v0.16)

Principes de travail : une commande à la fois (captures lisibles), toujours dire sur quelle machine on la lance, ne rien lancer de lourd sans avoir lu la fiche du cas, et ne jamais faire `git add -A` sans `git status` (règle du guide des scripts).

1. Modèle de fiche de cas

Chaque cas de test suit la même fiche, pour que le guide final soit homogène :

```
### Fiche X.Y — Titre du cas
Objectif        : ce qu'on veut apprendre ou vérifier
Machine         : PC1 / PC2 / cluster
Prérequis       : état attendu avant de commencer
Commandes       : celles lancées, une par ligne, avec leur rôle
Capture         : fichiers .log / .tm / image
Résultat attendu: ce que le guide annonce
Résultat obtenu : ce qu'on voit réellement
Lecture détaillée: ligne par ligne, avec étiquette mesuré / déduit
Remarques et pièges : erreurs rencontrées, écarts avec le guide
Verdict         : OK / à refaire / écart, puis ce qui reste à faire
À retenir       : la bonne pratique issue du cas
```

2. Partie A — Mise en place

A.1 Variables et dossier de captures (PC1)

```
zeta                                  # alias : se place dans ~/projet_zeta et active le venv zeta_env
export CAP=~/zeta_captures
mkdir -p $CAP
```

Le dossier `~/zeta_captures` est volontairement **hors du dépôt git** : les captures contiennent des noms de machines et, selon les commandes, des adresses IP. Elles ne se commitent pas et ne se publient pas sans relecture.

A.2 Capturer le terminal avec `script`

```
script -q -f -T $CAP/CAS.tm $CAP/CAS.log      # début (CAS = 1_0, 1_2, etc.)
# ... commandes du cas ...
exit                                           # fin : ferme l'enregistrement
scriptreplay -t $CAP/CAS.tm -s $CAP/CAS.log    # rejouer à la vitesse réelle
col -b < $CAP/CAS.log > $CAP/CAS.txt           # version texte lisible
```

Ce que fait chaque option : `-q` supprime les messages de début et de fin, `-f` écrit au fil de l'eau (le log est lisible pendant l'enregistrement), `-T fichier` enregistre le minutage pour pouvoir rejouer.

Quatre points constatés aux cas 1.0 et 1.1 (**mesuré**, voir les fiches) :

- `script` ouvre un **nouveau shell** : le préfixe `(zeta_env)` disparaît du prompt, donc le venv n'est plus actif. Les alias et fonctions de `~/.bashrc` sont en revanche rechargés (`zeta-mon-on` a fonctionné) et la bannière de démarrage « Boot sur disque LOCAL Seagate » s'affiche de nouveau.
- Le nouveau shell démarre dans le dossier courant du lancement, ici `~/zeta_captures`, **pas** `~/projet_zeta`. Toute commande à chemin relatif (`scripts/...`) échoue : c'est l'erreur du cas 1.0. Première commande à taper dans une session capturée : `zeta`.
- Avec `-q` il n'y a aucun message à la fermeture : sans `exit`, l'enregistrement continue sans que rien ne l'indique.
- **Après `script`, attendre la bannière et le prompt avant de taper ou coller.** Des lignes collées trop tôt s'affichent (écho du terminal) mais ne sont **pas exécutées** par le nouveau shell : constaté au cas 1.1, où trois lignes collées d'un bloc n'ont rien lancé. Une commande à la fois, en vérifiant que le prompt est revenu.

Règle : **ne pas lancer d'interface plein écran** (`htop`, `watch`, `tmux`, `vim`) dans une session capturée. Elles réécrivent l'écran en permanence et gonflent le log de séquences illisibles. On les ouvre dans une autre fenêtre de terminal, comme pour `htop` au cas 1.0.

A.3 Rappel : codes de retour

Code	Sens	Exemple
0	succès	commande terminée normalement
126	fichier présent mais non exécutable	oubli de `chmod +x`
127	commande ou fichier introuvable	chemin relatif depuis le mauvais dossier
1 à 3 pour `zeta_turbo_status.sh`	1 INACTIF, 2 DERIVE, 3 INVALIDE ou ILLISIBLE	**guide** : le code 1 n'est pas une erreur

On l'affiche avec `echo "code=$?"` **juste après** la commande.

3. Partie B — Cas 1 : run sur PC1 seul

Fiche 1.0 — État des lieux avant un run

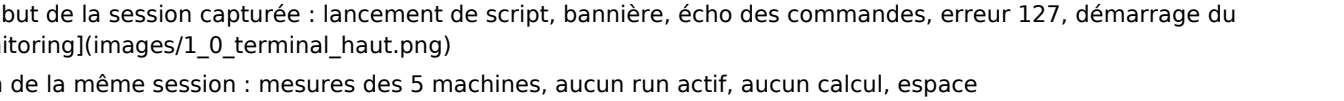
Objectif. Vérifier, avant tout run, que PC1 est prêt : turbo actif, monitoring (logger et watcher) actif, aucun calcul en cours, espace disque suffisant ; poser un repère « avant » pour lister ensuite les fichiers créés par le run.

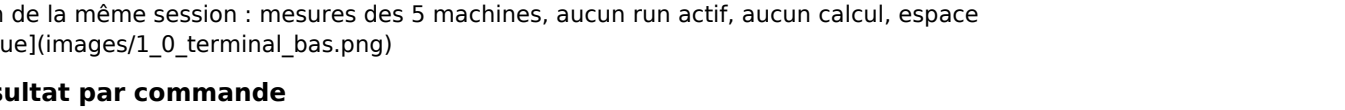
Machine. PC1 (terminal noir, `riemann@zeta-lab`), dans une session `script`.

Commandes lancées (collées d'un bloc, d'où l'écho groupé sur la capture) :

N°	Commande	Rôle
1	`scripts/zeta_turbo_status.sh; echo "code=$?"`	lit l'état réel du turbo (**guide** : dernière ligne `TURBO_STATUS=...`)
2	`zeta-mon-on`	arrête puis relance le logger (1 s, 5 machines) et le détecteur de runs, puis affiche le statut
3	`zeta-mon-status`	processus actifs, 5 dernières mesures, derniers runs, journal du watcher
4	`zeta-pid`	affiche le pid du run **distribué** (`/tmp/zeta_distribute.pid`) ou « Aucun run actif »
5	`pgrep -af compute_zeros || echo "aucun calcul en cours"`	cherche tout processus dont la ligne de commande contient `compute_zeros`
6	`df -h / /mnt/data | tail -2`	espace disque des deux systèmes de fichiers
7	`touch $CAP/avant.stamp`	crée un fichier repère daté ; plus tard `find -newer` listera ce qu'un run a écrit

Captures.





Résultat par commande

N°	Résultat obtenu	Statut
1	`bash: scripts/zeta_turbo_status.sh: Aucun fichier ou dossier de ce nom`, `code=127`	**ÉCHEC de procédure** (voir remarque 1), **repris en 1_0b** : `TURBO_STATUS=INACTIF`, code 1
2	deux jobs lancés, deux processus vivants (PID 10931 et 10932), statut affiché	OK
3	même contenu que le statut de `zeta-mon-on`, donc affiché deux fois	OK
4	`Aucun run actif`	OK, mais ne couvre que les runs distribués
5	`aucun calcul en cours`	OK
6	`/` 73 Go, 46 Go utilisés, 24 Go libres (66 %) ; `/mnt/data` 651 Go, 54 Go utilisés, 565 Go libres (9 %)	OK
7	aucune sortie, aucune erreur	OK (**déduit** : `$CAP` était bien défini, sinon `touch` aurait échoué)

Lecture détaillée de `zeta-mon-on`

Ce qu'on voit	Lecture
`[1] 10926` et `[2] 10927`	bash annonce deux jobs mis en arrière-plan (n° de job, PID du **lanceur**). **mesuré**
`[1]- Fini` et `[2]+ Fini`	les **lanceurs** ont terminé, ce qui est normal : `setsid` détache le programme et le relance sous un autre PID. Ce n'est pas un arrêt. La preuve est dans la section `processus` : 10931 et 10932 tournent. **mesuré**
`setsid nohup nice -n 19 "$ZETA_PY" ... > ".../logs/logger.log" 2>&1`	recette de lancement : `setsid` (survit à la fermeture du terminal), `nohup` (ignore la déconnexion), `nice -n 19` (priorité la plus basse : le monitoring s'efface devant le calcul), `$ZETA_PY` (python du venv, ce qui explique que le monitoring marche même sans `(zeta_env)`), sorties dans un fichier log. **mesuré**
`== processus ==` : 10931 `zeta_log_metrics.py`, 10932 `zeta_run_watch.py`	deux programmes vivants : le **logger** (mesure les 5 machines chaque seconde) et le **watcher** (détecte qu'un run démarre). Les noms sont coupés par l'affichage. **mesuré**
`== dernière mesure ==`	5 lignes, une par machine, détaillées plus bas. **mesuré**
`== dernier run ==` : `prod`, `test`, `historique`	**ce ne sont pas des runs** : ce sont les sous-dossiers de `monitoring/runs/`. Le libellé est trompeur (**guide** : `zeta-runs` liste les sous-dossiers depuis la v2). Pour voir les vrais runs : `ls -1t ~/projet_zeta/monitoring/runs/prod | head -3`.
`nohup: les entrées sont ignorées`	message **bénin** de nohup (l'entrée standard est ignorée), écrit dans `watch.log`. **mesuré**

Ce qu'on voit	Lecture
`watcher actif | motif = compute_zeros | racine = .../monitoring | run repris : None`	le watcher surveille les processus dont la ligne de commande contient `compute_zeros` ; `run repris : None` veut dire qu'il n'a aucun run en cours à reprendre (**déduit**). L'heure `21:17:46` est celle du lancement.
statut affiché deux fois	`zeta-mon-on` affiche déjà le statut à la fin, puis `zeta-mon-status` l'a répété. Pas une anomalie. **mesuré**

Lecture des mesures des 5 machines

Ligne type : `2026-10-24T21:17:47,PC1,zeta-lab,46.0,0.25,22.3,3534.0,3500.0,1`. L'en-tête du CSV n'est pas visible sur la capture, donc les colonnes sont **déduites** ; à confirmer par la commande de la remarque 6.

Colonnes déduites : horodatage, machine, hôte, température (°C), charge, RAM (%), RAM utilisée (Mo), fréquence (MHz), indicateur de réponse (1).

Machine	Température	Charge	RAM	Fréquence	Lecture
PC1 zeta-lab	46,0 °C	0,25	22,3 % (3 534 Mo)	3 500 MHz	repos. Les 3 500 MHz ne disent rien du turbo : le gouverneur est `powersave` (voir 1_0b). Colonne à ne pas utiliser pour juger l'état du turbo
PC2 zeta-calc-second	32,0 °C	0,24	16,6 % (1 295 Mo)	2 992 MHz	repos
PC3 zeta-backup	35,0 °C	0,0	15,5 % (898 Mo)	2 700 MHz	repos
PC4 zeta-secure	40,0 °C	0,06	pas de mesure	1 800 MHz	**guide** : pas de mesure de RAM sous OpenBSD, cohérent avec les champs vides
PC5 zeta-monitor	51,4 °C	0,0	17,2 % (270 Mo)	1 298 MHz	la plus chaude au repos : à surveiller, sans alarme

Seuils de température du **guide** (`zeta_temp_monitor.py`) : 75 °C avertissement, 90 °C critique. Les 5 machines sont loin en dessous. Les 5 lignes portent la même seconde (21:17:47) : le logger interroge bien les 5 machines à chaque tour. La valeur 3 534 Mo sur 15,5 Go est du même ordre que l'affichage de htop (3,19 G sur 15,5 G), les deux relevés n'ayant ni le même instant ni la même définition de « utilisé ».

Lecture de htop (PC1 au repos)

![htop sur PC1 au repos : 4 cœurs presque inactifs, monitoring à priorité basse, connexions SSH persistantes](images/1_0_htop_repos.png)

Zone	Sur la capture	Lecture
Barres CPU 0 à 3	7,1 % · 16,0 % · 5,8 % · 6,3 %	4 processeurs logiques (i7 2 cœurs, 4 threads). Barres bleues et rouges : **bleu = priorité basse** (notre monitoring à `nice 19`), **rouge = noyau**. Pas de vert : aucun calcul en priorité normale.
Tasks	155 tâches, 616 threads, 120 threads noyau, 1 en cours	« 1 running » est htop lui-même.
Load average	0,61 · 0,42 · 0,39	moyennes sur 1, 5 et 15 minutes du nombre de tâches prêtes à tourner. Avec 4 processeurs logiques, la saturation commence vers 4,0 : 0,61 correspond à environ 15 %, soit le repos.
Uptime	04:33:48	PC1 est démarré depuis 4 h 34. **Guide** : le monitoring ne survit pas à un redémarrage, d'où l'intérêt de `zeta-mon-on`.
Mem	3,19 G / 15,5 G	environ 21 % : marge confortable.
Swp	0 K / 16,0 G	swap **inutilisé** : bon signe. Un run qui fait monter cette valeur est une alerte.

Lignes de la liste des processus (vue en arbre, triée par PID) :

Ligne	Lecture
`gnome-text-editor ...claude-traitem...` (PID 5393 à 5397)	l'éditeur de texte ouvert sur un fichier du projet ; les lignes vertes sont ses **threads** (308 Mo résidents, 1,9 %). Sans rapport avec le run.
`chrome_crashpad_handler`	rapporteur de plantage de Chrome, inoffensif. Sa colonne VIRT affiche 52 Go mais RES 3,9 Mo : **VIRT est de la mémoire réservée, pas utilisée**, il faut lire RES.
10931 `python ...zeta_log_...` (+ 5 sous-lignes 28763 à 28772)	le **logger**. NI 19 / PRI 39, RES 16 Mo, 1,3 % de CPU. Les 5 sous-lignes de même taille sont des threads, **probablement un par machine** (déduit).
10932 `python ...zeta_run_...`	le **watcher**. NI 19, RES 14,7 Mo, 1,9 % : un peu plus que le logger parce qu'il scrute régulièrement la liste des processus (**déduit**).
4 lignes `ssh: /home/riemann/.ssh/zeta_cm_...` (10975, 10978, 10981, 10984)	connexions SSH **persistantes et partagées**, une par machine distante (PC2 à PC5, soit 4 : cohérent). Elles évitent de rouvrir une connexion à chaque seconde. Mécanisme « ControlMaster » **déduit** du nom `zeta_cm_` ; à vérifier dans `~/.ssh/config`.

Coût total du monitoring : environ 3 % d'**un** cœur et 31 Mo de mémoire résidente (1,3 + 1,9 % de CPU ; 16 144 + 14 764 Ko). Il est négligeable et tourne en priorité minimale.

À regarder dans htop pendant un run (attendu, à valider au cas 1.3)

#	À regarder	Attendu pendant un calcul	Signal d'alarme
1	barres CPU 0 à 3	toutes proches de 100 %, en **vert** (workers en priorité normale)	un seul cœur occupé : parallélisme perdu (régression déjà vue en v4)
2	Load average	proche ou au-dessus de 4 (nombre de processeurs logiques)	durablement très supérieur à 4 : surcharge
3	Mem et Swp	Mem stable, **Swp reste à 0 K**	Swp qui monte : risque d'OOM, calcul ralenti
4	processus `compute_zeros_v16.py`	plusieurs processus workers en tête de liste (tri CPU% : touche P ou F6)	aucun processus alors que le run est « lancé »

#	À regarder	Attendu pendant un calcul	Signal d'alarme
5	colonne S	R (en cours) ou S (en attente)	D prolongé (attente disque) ou Z (zombie)
6	TIME+ des workers	augmente pour tous ; un worker qui a fini devient inactif (le **guide** dit que le CSV est écrit à la fin de chaque worker)	un worker à 0 % tant que les autres calculent
7	processus du monitoring	restent à NI 19 et sous 3 % de CPU	le monitoring consomme beaucoup : il concurrence le calcul
8	onglet I/O	lectures et écritures disque par processus : utile pour la cartographie du cas 1.8	écriture continue énorme et inattendue

Raccourcis utiles : `F4` filtrer (taper `compute`), `F5` ou `t` arbre, `P` trier par CPU, `M` trier par mémoire, `H` masquer les threads, `Tab` changer d'onglet, `q` quitter. `F9` **envoie un signal au processus sélectionné : à éviter par réflexe.** Pour n'afficher que le calcul : `htop -p $(pgrep -d, -f compute_zeros)`.

Reprise 1_0b — lecture de l'état du turbo

Nouvelle session `script` (1_0b), ouverte dans `~/zeta_captures`, puis `zeta` pour se placer dans `~/projet_zeta` (le prompt retrouve `(zeta_env)`), puis `scripts/zeta_turbo_status.sh; echo "code=$?"`. Les deux lignes `exit` de la fin sont normales : la première est la commande tapée, la seconde est le message du shell qui se ferme.

![Sortie de zeta_turbo_status.sh sur PC1 : gouverneur powersave sur les 4 CPU, TURBO_STATUS=INACTIF, code 1](images/1_0b_turbo_status.png)

Ligne	Valeur	Lecture
cpu0 à cpu3	`gouverneur=powersave, epp=balance_performance`	les 4 processeurs logiques sont en mode économie d'énergie ; la préférence d'énergie (EPP) est « équilibré, penchant performance ». **mesuré**
Gouverneurs présents	`powersave · performance sur 0/4 CPU`	aucun CPU en mode « performance » : c'est le critère du turbo de session. **mesuré**
Boost matériel (`no_turbo`)	0	le boost **matériel** du processeur n'est pas désactivé : le processeur garde le droit de monter en fréquence de lui-même. Ce n'est pas la même chose que le mode « performance » imposé par le turbo de session. **mesuré**
`vm.swappiness`	10	le noyau évite de recourir au swap (valeur basse). **mesuré**
`power-profiles-daemon`	profil `balanced` (peut réécrire le gouverneur)	service de profils d'énergie du bureau. S'il réécrivait le gouverneur pendant un run, le statut passerait en DERIVE. **mesuré** pour le profil, **guide** pour le risque
État sauvegardé	aucun	`zeta_turbo_on.sh` n'a pas enregistré d'état initial depuis le démarrage : il n'a donc pas tourné. **mesuré**
`TURBO_STATUS` et code	`INACTIF`, code 1	**guide** : le code 1 veut dire INACTIF, ce n'est **pas** une erreur

Interprétation. Le turbo de session est **inactif**. C'est cohérent avec le **guide** : il l'active au démarrage d'une session Claude Code (hook SessionStart) et le restaure au « bye bye » ; ici l'état sauvegardé est vide, donc rien ne l'a activé depuis le démarrage (**déduit**). Conséquence : un `zeta_run.sh` lancé maintenant tournerait en `powersave`. L'effet attendu est faible : les benchmarks du 16/10/2026 cités par le **guide** donnent un écart d'environ 2 % à T=5000 et T=20000, inférieur à la dispersion, et le « +15 à 30 % » annoncé n'est pas démontré. En revanche, pour comparer deux runs, il faut **noter l'état du turbo dans chaque fiche**.

Correction d'une interprétation précédente : la fréquence de PC1 relevée par le logger (3 500 MHz au repos) avait été présentée comme un indice de turbo actif. C'est **faux** : le gouverneur est `powersave`. La colonne « fréquence » du CSV ne renseigne donc pas sur l'état du turbo (valeur probablement maximale ou théorique ; à confirmer avec l'en-tête du CSV).

Pour activer ou restaurer le turbo à la main (**guide**) : `zeta-turbo-on` puis `zeta-turbo-off` (alias avec `sudo`) ; `scripts/zeta_session_turbo.sh start`, `status` et `end` font la même chose à l'échelle d'une session.

Remarques et pièges

- **Erreur 127 sur** `zeta_turbo_status.sh`**.** La session `script` s'est ouverte dans `~/zeta_captures`, le chemin relatif `scripts/...` n'existe pas à cet endroit. Ce n'est ni une panne du turbo ni un défaut du script. Correction : taper `zeta` (ou `cd ~/projet_zeta`) d'abord, ou utiliser le chemin absolu. Vérifier `pwd` à chaque début de session capturée.
- **Le turbo a été relu en 1_0b : INACTIF.** L'indice tiré de la fréquence de PC1 (3 500 MHz au repos) était faux : cette colonne ne renseigne pas sur l'état du turbo. Seule la sortie de `zeta_turbo_status.sh` fait foi.
- **Collage d'un bloc.** Les 7 commandes collées d'un coup s'affichent d'abord en écho, puis les sorties arrivent à la suite : la capture est plus difficile à lire. Une commande à la fois, ou un `echo "=== 2 ==="` entre elles.
- **« Dernier run » ne montre pas de run** (voir plus haut) et `zeta-pid` **ne voit que les runs distribués** : pour un `zeta-run` simple, seul `pgrep -af compute_zeros` répond. C'est le contrôle fiable, quel que soit le type de run.
- `df -h / /mnt/data` **ne montre peut-être pas le disque qui contient** `~/projet_zeta` (par exemple si `/home` est une partition séparée). Version plus sûre : `df -h ~/projet_zeta /mnt/data`. **À vérifier.**
- **En-tête du CSV.** Pour confirmer les colonnes : `head -1 "$(ls -t ~/projet_zeta/monitoring/data/cluster_metrics_*.csv | head -1)"`.
- `zeta-mon-on` **redémarre le monitoring.** À lancer **avant** un run, pas pendant.
- **Confidentialité.** Aucune adresse IP n'est visible sur ces captures, seulement des noms de machines ; vérifier à chaque cas avant toute publication.

Verdict de la fiche 1.0

Contrôle	Résultat	Statut
Monitoring actif (logger + watcher)	2 processus vivants	OK
5 machines mesurées à la même seconde	oui	OK
Aucun calcul en cours	`pgrep` vide, `zeta-pid` vide	OK
Espace disque	24 Go libres sur `/`, 565 Go sur `/mnt/data`	OK
Turbo	lu en `1_0b` : `INACTIF`, code 1, gouverneur `powersave`	OK (état normal hors session, **déduit**) ; à noter dans chaque run
En-tête du CSV	non lu	à lire (remarque 6)

Reste à faire pour clore la fiche : lire l'en-tête du CSV (remarque 6). Le reste est acquis ; on peut passer au cas 1.1.

À retenir (bonne pratique n°1). Avant tout run, quatre contrôles lancés depuis `~/projet_zeta` : turbo, monitoring, aucun calcul (`pgrep`), disque. On pose aussi le repère `avant.stamp`. Aucun des quatre ne modifie le système, sauf `zeta-mon-on` qui relance le monitoring.

Fiche 1.1 — Tests de référence (3 fichiers de test sur 3 passés)

Objectif. Vérifier, par trois fichiers de tests, (1) la règle de consolidation V17, (2) le comportement de `compute_zeros_v17.py` en cas de panne, (3) le script turbo ; sans lancer de vrai run, et s'entraîner à lire ces sorties.

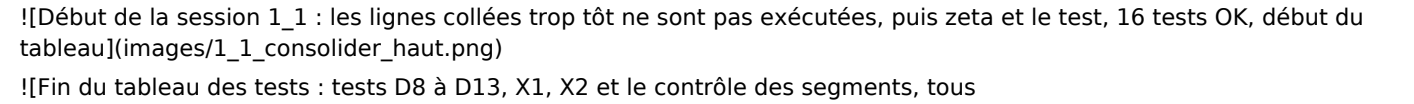
Machine. PC1, session `script` 1_1.

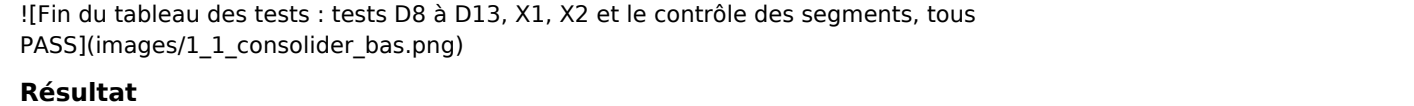
Commandes (une à la fois, dans la session capturée) :

```
zeta
zeta_env/bin/python3 -B -I tests_v17/test_consolider_v17.py      # fait : 16 tests OK
zeta_env/bin/python3 -B -I tests_v17/test_compute_zeros_v17.py   # fait : 9 tests OK
zeta_env/bin/python3 -B -I tests_turbo/test_zeta_turbo.py        # fait : 34 tests OK
```

Option	Rôle
`-B`	n'écrit pas de fichiers `.pyc` : le test ne laisse aucune trace dans le dépôt
`-I`	mode isolé : ignore les variables d'environnement `PYTHON*`, le dossier utilisateur de paquets et le dossier courant ; le résultat ne dépend pas de l'environnement (reproductibilité). **guide** : « toujours lancé depuis `~/projet_zeta` avec `-I` »

Captures.





Résultat

Élément	Valeur	Lecture
`Ran 16 tests in 0.010s` puis `OK`	16 tests, 0,010 s, aucun échec	sortie standard du module `unittest`. **mesuré**
Tableau des tests	16 lignes `[PASS]` : D01 à D13, X1, X2, `test_controle_segments`	tableau propre au projet : pour chaque test, objectif, entrée, attendu, observé. **mesuré**
Comparaison au guide	« 16/16 PASS le 15/10/2026 »	le résultat est **reproduit** neuf jours plus tard. **guide** + **mesuré**
Durée 0,010 s	quasi instantané	les entrées sont **synthétiques** : aucun calcul de zêta (**guide**).

Ce que ces tests vérifient

Le fichier teste `consolider_v17.py`, qui décide, pour des zéros candidats rapportés par plusieurs workers, lesquels sont le **même** zéro et lesquels sont **distincts**. **Principe de prudence** : on ne fusionne que sur une preuve forte, sinon on garde les deux lignes. Trois statuts possibles :

Statut	Sens
`DUPLICATE_CONFIRMED`	même zéro rapporté par deux workers adjacents dans la zone de chevauchement : une seule ligne conservée
`DISTINCT`	deux zéros différents, conservés tous les deux
`AMBIGUOUS`	indécidable : **les deux lignes sont conservées** (décision du 15/10/2026 d'après le **guide**)

Tests	Ce qu'ils vérifient (d'après le tableau)
D01, D10	doublon frontière confirmé seulement dans la zone [200 ; 200,5], bornes incluses ; hors zone : `AMBIGUOUS`
D02, D08	deux zéros très proches (7e-3) ou une chaîne espacée de 6e-3 restent `DISTINCT` ; l'ancienne méthode `dedupliquer(0,01)` n'en gardait que 2 sur 4
D03, D04, D13	précision insuffisante, valeurs presque identiques, provenance inconnue : `AMBIGUOUS`, rien n'est supprimé
D05, D06	des rayons d'incertitude qui se chevauchent n'impliquent pas l'identité ; règle R-RAYON
D07	le résultat ne dépend pas de l'ordre des lignes (20 permutations identiques)
D09	NaN et infinis : journalisés comme invalides, jamais fusionnés ni perdus
D11	relecture des mêmes checkpoints après une interruption : rien de perdu, résultat identique
D12, X2	traçabilité de chaque décision (fichier, ligne, worker, valeur, règle) et écriture atomique, sans écraser un dossier non vide
X1	un doublon confirmé pris dans une paire ambiguë garde son statut et est mis en quarantaine
`test_controle_segments`	contrôle complémentaire hors D1 à D13 : les segments reconstruits sont ordonnés

Remarques et limites

• **Ce que 16 PASS ne prouvent pas.** Ils valident la **règle de consolidation** sur des cas fabriqués. Ils ne prouvent ni que les zéros calculés sont corrects, ni qu'ils sont complets : cela relève de la validation Turing-Backlund.

• **D8 est instructif** : l'ancienne méthode peut supprimer de vrais zéros rapprochés. Le test ne dit rien sur l'origine des zéros manquants d'un run donné : ne pas relier les deux sans mesure.

• **Tableau tronqué.** Le tableau n'affiche que la première ligne du commentaire de chaque test, d'où des objectifs coupés en milieu de phrase (D4, D6, D10, `test_controle_segments`). Pour `test_controle_segments`, les champs entrée, attendu et observé sont vides : le test est compté dans « Ran 16 tests » mais son tableau est incomplet. Cosmétique.

• **Collage après** `script` : voir A.2, point 4. Le log 1_1 commence par ces deux lignes sans résultat ; la capture reste exploitable.

Fichier 2 — `test_compute_zeros_v17.py` : 9 tests OK en 3,3 s

Objectif. Vérifier que le calcul reste honnête quand l'environnement tombe en panne : rien n'est perdu ou écrasé en silence, et toute perte est signalée.

![Début de la sortie : GPU non supporté, mode CPU, segments des 8 workers, consolidation 138 → 138, avertissement fork](images/1_1_compute_debut.png)

![Fin de la sortie : disque plein simulé (188 avertissements), échec d'écriture finale signalé, 9 tests OK](images/1_1_compute_d2_fin.png)

Test	Panne simulée	Constat
D1a	3 consolidations successives dans le même dossier	138 bruts → 138 zéros, 0 doublon ; les 2e et 3e écrivent dans `consolidation_v17_2/` et `_3/` : l'existant est **préservé**
D1b, D1c	échec de `consolider` ; échec d'écriture de la consolidation	pas de repli silencieux vers l'ancienne méthode ; échec traité sans perte
D2a	disque plein (`Errno 28`)	188 avertissements `CHECKPOINT NON ÉCRIT`, **le calcul continue** : 202 zéros, `ckpt_échoués:188`
D2b	erreur d'E/S (`Errno 5`)	le « dernier partiel valide » (2 zéros) reste distinct de l'état courant
D2c	incohérence logique	elle n'est pas masquée par la gestion d'erreur
D2d	écriture finale impossible	`ÉCRITURE FINALE ÉCHOUÉE … résultat NON sauvegardé` : l'erreur est bruyante
D2e, D6	échec d'écriture atomique ; alignement de la provenance	OK

Remarques :

• **Les 188 avertissements sont simulés.** Dans un vrai run, ce message signifie disque réellement plein ou en erreur : le calcul continue mais la reprise après interruption n'est plus possible (à retenir pour le cas 1.10).

• **Avertissement GPU.** La GTX 960M (sm_50, capacité de calcul < 6.0) n'est pas supportée par CUDA 12.x : mode CPU, ainsi que « mode batch numpy » (annoncé ×7 à ×15 par le programme, **non mesuré ici**). Comportement prévu.

• `DeprecationWarning` **sur** `fork()`**.** Python 3.12 signale que forker un processus multi-thread peut provoquer des blocages. Bruit ici ; à garder en tête si un run se bloque sans raison apparente (**déduit**).

• **138 zéros sur [14 ; 300].** Cohérent avec l'estimation de Riemann–von Mangoldt N(300) ≈ 137,7 (ordre de grandeur, pas une preuve de complétude).

• **Workers 0 et 7.** Worker 7 : `arb_C:17 fallback:1` (un zéro par le chemin de repli). Worker 0 : 18 zéros mais `arb_C:4` ; je suppose un autre chemin à basse hauteur (**déduit**, à confirmer dans le code).

• **Zone de chevauchement.** Aucun doublon n'y apparaît : `DUPLICATE_CONFIRMED` n'est pas exercé ici, il l'est dans le fichier 1.

• **Aucune trace dans le projet** : les sorties vont dans `/tmp/tmp…`.

Fichier 3 — `test_zeta_turbo.py` : 34 tests OK en 4,65 s

Objectif. Vérifier que le script turbo (`zeta_session_turbo.sh`, `zeta-turbo-on/off`) active un mode performance **et restaure exactement l'état initial**, même en cas d'échec en cours de route.

![Les 34 tests du script turbo, tous ok](images/1_1_turbo_34.png)

Famille (nombre)	Ce qui est vérifié
`TestActivation` (4)	activer deux fois ne change rien ; plusieurs runs dans la même session ; `swappiness` déjà à 10 laissée telle quelle ; modifiée puis restaurée
`TestConcurrence` (1)	deux activations simultanées sans corruption de l'état
`TestDurcissement` (1)	un nom de service hostile écrit dans le fichier d'état n'est **jamais exécuté**
`TestEnveloppeSession` (7)	`start` puis end ; end sans état est idempotent ; restauration incomplète signalée ; sans privilèges, `start` ne bloque pas et ne prétend rien ; usage
`TestErreurs` (9)	pas de privilèges ; arguments inconnus ; échec à mi-parcours puis restauration ; état invalide ou manquant ; `--force-defaults` refusé si l'état est valide ; interface CPU absente ; variables de test ignorées en root
`TestRestauration` (5)	depuis un état initial actif, désactivé, mixte ; EPP restauré ; gouverneur initial autre que `powersave` restauré tel quel ; `off` sans rien d'actif idempotent
`TestServices` (3)	seuls les services arrêtés par le script sont redémarrés ; un service déjà inactif n'est pas démarré ; échec de redémarrage conserve l'état

Famille (nombre)	Ce qui est vérifié
TestStatut (4)	boost matériel non modifié mais signalé ; dérive puis réaffirmation sans toucher à l'état initial ; statuts (ACTIF / INACTIF / DÉRIVE / INVALIDE)

Remarques :

• **Aucun test n'a modifié le turbo réel.** Les tests s'exécutent en utilisateur normal ; l'un d'eux vérifie que des variables de test sont ignorées en root. Ils travaillent donc sur un système simulé (**déduit** du nom des tests, puis **confirmé** : `zeta_turbo_status.sh` relancé après les tests donne toujours `TURBO_STATUS=INACTIF`, code 1, gouverneur `powersave`, état sauvegardé « aucun »).

• **Ces tests ne prouvent pas un gain de performance.** Ils garantissent la **restauration** et la sûreté, pas les « +15–30 % » annoncés par le guide, non démontrés (benchmarks 16/10 : environ 2 %).

• **L'état réel du turbo** (INACTIF, `powersave`) est une mesure de la fiche 1.0b, pas un résultat de ces tests.

Verdict de la fiche 1.1

Fichier de test	Attendu (guide)	Obtenu	Statut
`tests_v17/test_consolider_v17.py`	16 PASS	16 PASS en 0,010 s	**OK**
`tests_v17/test_compute_zeros_v17.py`	9 OK	9 OK en 3,3 s	**OK**
`tests_turbo/test_zeta_turbo.py`	34 OK	34 OK en 4,65 s	**OK**

Au total, **59 tests, 0 échec**. Ces tests couvrent la consolidation, la tolérance aux pannes d'écriture et la restauration turbo ; ils ne couvrent pas la justesse des zéros ni leur complétude (Turing-Backlund), qui se vérifient sur un vrai run (cas 1.2 puis 2.5).

À retenir (bonne pratique n°2). Avant un run important ou après toute mise à jour du code, relancer les trois tests de référence avec `-B -I` depuis `~/projet_zeta` : environ 8 s au total, sans effet sur le système, et ils détectent une régression de la règle de consolidation, de la tolérance aux pannes ou de la restauration turbo.

Fiche 1.2 — Premier run T=5000 (PC1 seul)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Lancer un vrai run court, lire ce que le script affiche et mesurer la durée de référence.

Contrôles avant run (session `script` 1_2, bonne pratique n°1) : monitoring actif (`zeta-mon-status` : logger 10931 et watcher 10932 vivants, 5 machines mesurées) ; aucun calcul en cours (`pgrep -af 'compute_zeros_v'` : rien) ; disque (`df -h ~/projet_zeta /mnt/data` : le projet est sur `/home`, 25 Go libres, 86 % utilisé ; `/mnt/data` 565 Go libres) ; repère `touch $CAP/avant.stamp`.

Commande. `time scripts/zeta_run.sh 5000`



Ligne de la sortie	Lecture
`Log → …/logs/run_T5000_<date>.log`	journal du run, à suivre avec `tail -f`
`État initial enregistré : /run/zeta_turbo/state`	le script **sauvegarde** l'état du CPU avant de le modifier. **mesuré**
`Gouverneur performance : 4/4 CPU · swappiness 10 · boost matériel actif (no_turbo=0, non modifié)`	`zeta_run.sh` **met lui-même les 4 CPU en** `performance`. Le boost matériel était déjà autorisé : seul le gouverneur change. **mesuré**
`Lancement du calcul en arrière-plan (extinction/veille inhibées)`	le calcul est lancé sous `systemd-inhibit` : la machine ne peut ni se mettre en veille ni s'éteindre pendant le run. **mesuré** (voir htop, fiche 1.3)
`cups, ModemManager, NetworkManager-wait-online, snapd, fwupd, kerneloops redémarré`	six services sont arrêtés pendant le calcul pour libérer du CPU, puis relancés à la fin (**déduit** pour la raison). **mesuré** pour le redémarrage
`TURBO: SERVICES RESTAURÉS (turbo CPU maintenu)`	les services sont revenus, mais **le mode performance reste actif** : c'est à l'utilisateur de fermer la session turbo (`zeta-turbo-off`). **mesuré** (`zeta_turbo_status.sh` : ACTIF, code 0)

Durées : `real` 20,1 s, `user` 15,8 s, `sys` 0,5 s.

Erreur à ne pas répéter (mon erreur d'annonce). J'avais indiqué que ce run serait fait en turbo INACTIF, comme référence. C'était faux : le script l'active. **Le premier run du guide a donc été fait en mode** `performance`**.** Pour un point de référence en `powersave`, il faudrait désactiver le turbo et neutraliser cette étape du script.

Fiche 1.3 — Suivi en direct, trois fenêtres

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Voir un run pendant qu'il tourne : htop (processus), `zeta-proc` (échantillonnage écrit dans un fichier), terminal du run.

Fenêtres. A : session capturée avec le run. B : htop avec filtre. C : `zeta-proc`.

Réglage de htop. `F4`, taper `compute`, `Entrée` (le filtre reste actif), `F5` (arbre), `F6` puis `CPU%`. Avec le filtre `compute_zeros_v`, le watcher de monitoring n'apparaît pas.

Séries de mesures (turbo actif)

Run	real	user	user / real
T=5000	20,1 s	15,8 s	0,8

Run	real	user	user / real
T=20000	46,0 s	89,6 s	1,9
T=100000 (1er)	148,9 s	327,6 s	2,2
T=100000 (2e)	145,4 s	323,5 s	2,2

Reproductibilité : environ 2 % entre les deux runs T=100000. **Estimation corrigée deux fois** : mon extrapolation de la durée était fausse (120 s annoncées pour 149 s mesurées). Le temps croît environ en T^0,7 entre T=20000 et T=100000 (**heuristique**, peu de points).

Lecture de htop pendant un run

Zone	Valeur	Lecture
Barres CPU 0 à 3	99–100 %	les 4 cœurs logiques sont saturés. **mesuré**
Load average	3,8 puis 5,99, 8,58	monte au-dessus de 4 : plus de tâches prêtes que de cœurs. **mesuré**
Swp	0K	pas de swap. **mesuré**
Arbre	`systemd-inhibit` → `python …/monitoring` → `bash -c printf '100000\n0\n' | python …` → `python …/src/` → 8 workers	chaîne de lancement complète. Le PID affiché par `zeta_run.sh` est celui du `systemd-inhibit`. **mesuré**
8 workers en R	31 à 98 % de CPU chacun, 410 % au total	8 workers se partagent 4 cœurs logiques. **mesuré**
`PRI 20 / NI 0`	priorité normale	les workers ne cèdent la place à personne, contrairement au logger (`nice 19`)
VIRT ≈ 29,6 Go, RES ≈ 200 Mo	—	VIRT est de l'espace d'adressage réservé, pas de la mémoire utilisée
5 processus S à 0 %	copies du processus principal	rôle non identifié. **à vérifier**

Outil `zeta-proc` (échantillonneur de processus)

Pas d'équivalent dans les 74 documents du projet : htop ne peut pas écrire dans un fichier, et le `%CPU` de `ps` est une moyenne depuis le démarrage du processus, inutilisable pour voir une phase qui change. `scripts/zeta_proc_sample.py` lit `/proc` (lecture seule, priorité minimale, bibliothèque standard) et écrit dans `~/zeta_captures/` (hors dépôt git) un CSV et un résumé `_resume.md`. Alias `zeta-proc` ajouté à `~/.bashrc` (sauvegarde `~/.bashrc.avant_zeta-proc`). Usage : `zeta-proc --intervalle 2` dans une fenêtre à part, puis lancer le run.

Mesure du résumé (run du 25/10, T=100000)	Valeur
durée observée	142 s (60 échantillons)
CPU cumulé moyen	201 % (sur 400 % possibles)
pic de workers actifs	8
phase finale à 1 processus actif ou moins	**64 s (45 % de la durée)**
mémoire résidente maximale	2 062 Mo
load average maximal	9,79
swap maximal	0 Mo

Limite connue de l'outil. Le « CPU cumulé maximal » (511 %) est impossible (maximum théorique 400 %) : lectures retardées par la priorité minimale sur des cœurs saturés. La moyenne est fiable, pas le maximum. À corriger en v2 (horodatage pris à chaque lecture).

Chronologie d'un run T=100000

Étape	Intervalle (s)	Source
8 workers en parallèle	0 → 54–86 (fins échelonnées)	`zeta-proc`. **mesuré**
Processus 515765 (enfant du principal, même script)	86 → 114	`zeta-proc`. **mesuré**
Processus principal seul	114 → 136	`zeta-proc`. **mesuré**

Fin d'activité par worker : 53,8 · 57,2 · 65,1 · 67,6 · 69,7 · 74,1 · 76,1 · **86,1** s. Temps actif de 34 s (le plus rapide) à 62 s (le plus lent) : charge inégale entre workers. **mesuré**

Extrait du journal du run (`[PROFIL PHASES]`, temps cumulé) : `arb_C` 282 s (138 061 appels, 2,04 ms) · `detection` 217 s (8 appels) · `mpmath_fallback` 31 s · `turing` 18,65 s (1 appel) · `mpmath_petit_t` 9,88 s (1 appel). Résultat : **138 069 zéros,** `COMPLET — aucun zéro manquant dans [0, 99999.70]`, surplus de 4 zéros attribué au chevauchement. **mesuré** (journal).

Déduit, non vérifié : le processus 515765 exécute la validation Turing et `mpmath_petit_t` (28,5 s de journal pour 26 s d'activité mesurées) ; le processus principal fait ensuite la consolidation, l'écriture du CSV et le graphique. **Correction (fiche 1.5) :** le journal du run T=200000 montre que le deuxième groupe de processus est le **rescan** de segments en déficit (un processus par segment). Pour T=100000, le journal du run n'a pas été relu : l'attribution du processus 515765 au rescan reste à confirmer.

Pistes d'optimisation (hypothèses à vérifier dans le code)

- **Équilibrage des workers** : écart d'environ 32 s entre le premier et le dernier ; découper en plus de segments que de workers.

• **Validation Turing** : environ 28 s en un seul processus après les workers ; voir si elle peut démarrer pendant le calcul. (À reprendre après la fiche 1.5 : le deuxième groupe est le rescan, pas Turing.)

• **Finalisation** : environ 22 s séquentielles.

• Borne théorique si tout était parfaitement réparti : `user / 4` ≈ 81 s au lieu de 145 s (**heuristique**, pas une promesse).

Monitoring : pas de doublon, mais de faux runs

Les 4 runs du jour sont uniquement dans `monitoring/runs/prod/` : l'enveloppe intégrée à `zeta_run.sh` suffit, le watcher n'a pas créé de doublon (**mesuré** sur 4 runs). En revanche `monitoring/runs/auto/` contient 3 **faux runs** (`evince`, `eog`, `test_compute_zeros_v17`) : le watcher prend pour un run tout processus dont la ligne de commande contient `compute_zeros` (**déduit**). Voir écart E12.

À retenir (bonne pratique n°3). Un run de 20 s est trop court pour htop : pour observer un run, viser T=100000 (environ 2 min 30). Lancer `zeta-proc` avant le run pour ne rien perdre.

Fiche 1.4 — Graphique de suivi en direct (capture 1_4, T=100000)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Tester `zeta-proc-plot` (graphique rafraîchi toutes les 2 s à partir du CSV de `zeta-proc`) pendant un vrai run, et vérifier que seuls les processus du run sont comptés.

Machine. PC1. **Prérequis.** `zeta_proc_sample.py` et `zeta_plot_proc.py` dans `~/projet_zeta/scripts/`, alias `zeta-proc` et `zeta-proc-plot` dans `~/.bashrc`, session de capture ouverte, état du turbo noté (il restait actif depuis le run précédent).

Fenêtre	Rôle	Commande	Dans la capture ?
A	le run	`time scripts/zeta_run.sh 100000`	oui (`script … 1_4`)
B	htop filtré	`htop`, F4, `compute`, Entrée	non
C	échantillonneur	`zeta-proc --intervalle 2 --test 1_4`	non
D	graphique	`zeta-proc-plot`, environ 10 s après le début du run	non

Ordre. C d'abord (il attend l'apparition du run), puis A, puis D quand le CSV existe. La fenêtre D ne doit pas être lancée avant : sans CSV du test en cours, elle peut afficher un ancien fichier.

Nommage des fichiers (nouveau). L'option `--test 1_4` met le numéro du test dans le nom : `proc_1_4_AAAAMMJJ_HHMMSS.csv`, `proc_1_4_…_global.csv` et `proc_1_4_…_resume.md`. Sans l'option, le nom reste `proc_AAAAMMJJ_HHMMSS…`. Le graphique retrouve les deux sortes de noms.

Installer une nouvelle version de l'outil, avec contrôle (PC1, hors capture) :

```
F=$(ls -t ~/Téléchargements/zeta_proc_sample*.py | head -1); echo "fichier : $F"; sed -n 3p "$F" | grep -q "v3.2" && cp "$F" ~/pro
```

Le contrôle `sed -n 3p … | grep` refuse de copier un fichier qui n'est pas la bonne version. Il évite un piège rencontré pendant ce cas : un fichier téléchargé qui n'était pas la bonne version avait été copié sans contrôle (la ligne 3 affichait encore `v3`, sans l'option `--test`).

Mesure (run 1_4, T=100000)	Valeur
`time` : real / user / sys	2 min 25,7 s / 5 min 24,3 s / 0,9 s (user / real = 2,2)
durée observée par `zeta-proc`	143 s
CPU cumulé moyen / maximal	191 % / 382 %
pic de workers actifs	8
phase finale à ≤ 1 actif	64 s (45 %)
mémoire résidente maximale / swap	2 050 Mo / 0 Mo
load average maximal	9,25

Lecture du graphique (trois panneaux, de haut en bas) : CPU de chaque processus ; nombre de workers actifs (≥ 20 % de CPU) et cœurs occupés, avec la zone orange « phase finale » ; mémoire résidente et load. Le titre indique « en cours » puis « TERMINÉ », la durée, le CPU moyen et le swap maximal.

Combien de processus appartiennent au run ? (la question posée pendant le test)

L'impression d'avoir « plus de PID que de processus de calcul » était justifiée. Résultat de l'analyse du CSV du test 1_4 (11 PIDs, **mesuré**) :

PID	Parent	Vie (s)	Rôle
820374	820372	0 → 141	wrapper de monitoring `zeta_mon_run.py` : **pas du calcul**
820417	820415	0 → 135	processus principal `compute_zeros_v16.py`
820508 à 820515	820417	2 → 85	8 workers, nés et morts ensemble
821911	820417	89 → 115	processus fils créé après la mort des workers

Le wrapper avait été compté parce que sa ligne de commande contient, parmi ses arguments, le chemin de `compute_zeros_v16.py`. Le graphique ne l'affichait pas dans sa légende (filtre à 20 % de CPU), mais le CSV le contenait. **Écart E16** : corrigé en v3.2 (le script exécuté doit lui-même porter le motif ; test sur six lignes de commande simulées avant livraison).

Trois façons de compter, trois résultats différents

Source	Lignes	Pourquoi
htop filtré `compute`, vue en arbre (run 1_5, pool 1)	12	8 workers, le principal, `bash -c`, le wrapper et `systemd-inhibit` : la chaîne de lancement complète. **mesuré**
`zeta-proc` v3 (run 1_4)	11 PIDs au total	10 processus de calcul et le wrapper. **mesuré**
`zeta-proc` v3.2 (run 1_5)	11 PIDs au total	principal, 8 workers et 2 processus du rescan : tous des `compute_zeros_v16.py`. **mesuré**

La différence entre htop et `zeta-proc` est voulue : htop montre l'arbre de lancement, `zeta-proc` ne suit que le calcul. Dans htop, la touche `H` (majuscule) masque les threads, qui sinon ajoutent des lignes.

Verdict. OK pour le graphique (rafraîchissement régulier, aucune gêne visible sur le run). Écart E16 corrigé et revérifié à la fiche 1.5.

À retenir (bonne pratique n°4). Un PID qui existe n'est pas un processus qui calcule : compter les workers actifs (CPU ≥ 20 %), pas les PIDs. Nommer les captures avec le numéro du test (`--test`).

Fiche 1.5 — Run T=200000 : structure complète d'un run (capture 1_5)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

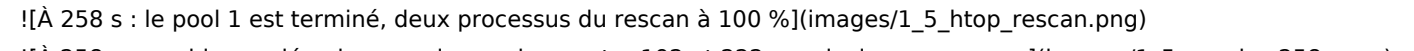
Objectif. Observer un run de plusieurs minutes avec htop (filtre `compute`) à côté du graphique, puis relier ce qu'on voit aux lignes du journal du run.

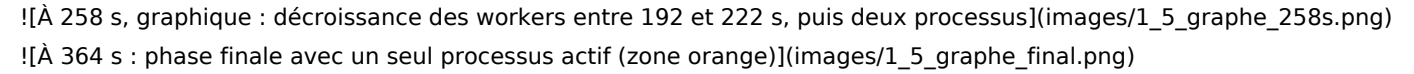
Machine. PC1, mêmes quatre fenêtres que la fiche 1.4, avec `zeta-proc --intervalle 2 --test 1_5`. Commande du run : `time scripts/zeta_run.sh 200000`. Après le run, dans la session capturée : `zeta-turbo-off`.

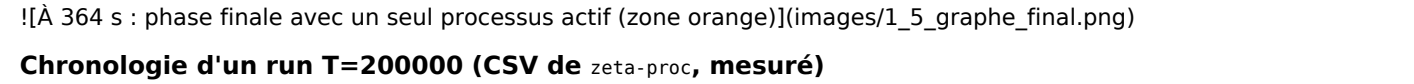
Annonce erronée à ne pas répéter. J'avais annoncé environ 4 minutes (extrapolation, marge ±1 min). Mesuré : **6 min 13,5 s**.

Mesure	T=100000 (1_4)	T=200000 (1_5)
`time` real	2 min 25,7 s	**6 min 13,5 s** (373 s)
`time` user	5 min 24,3 s	16 min 31,1 s (991 s)
`time` sys	0,9 s	2,2 s
durée observée par `zeta-proc`	143 s	364 s (143 échantillons)
CPU cumulé moyen / maximal	191 % / 382 %	248 % / 385 %
phase finale à ≤ 1 actif	64 s (45 %)	34 s (9 %)
mémoire résidente max / swap	2 050 Mo / 0	2 051 Mo / 0
load average maximal	9,25	14,26
mémoire disponible minimale	11 057 Mo	10 783 Mo

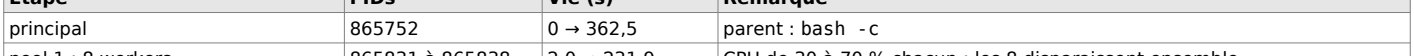
Rapport de durée : 373 / 146 ≈ 2,6 pour un T deux fois plus grand. Deux points seulement : **ordre de grandeur, pas une loi**. Cohérence de `time` : user / real = 991 / 373 ≈ 2,65 cœurs en moyenne, contre 2,48 pour `zeta-proc` ; l'écart (environ 90 s de CPU) n'est pas expliqué (processus non suivis, échantillonnage toutes les 2 s ?). Fin de session : `TURBO: RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial (cpu0=powersave), swappiness 10).`











Chronologie d'un run T=200000 (CSV de `zeta-proc`, mesuré)

Étape	PIDs	Vie (s)	Remarque
principal	865752	0 → 362,5	parent : `bash -c`
pool 1 : 8 workers	865831 à 865838	2,0 → 231,9	CPU de 30 à 70 % chacun ; les 8 disparaissent ensemble
transition	principal	232 → 236	pic de CPU du principal d'environ 60 % (lu sur le graphique)
pool 2 : rescan, 2 processus	868843, 868844	235,9 → 340,4	deux processus à 100 % : deux cœurs sur quatre
principal seul	865752	340,4 → 362,5	22 s

Un processus peut exister sans calculer : le graphique voit les workers décroître de 192 à 222 s (8 → 1 actif), alors que les 8 PIDs ne disparaissent qu'à 231,9 s. De même, l'un des deux processus du rescan cesse de calculer vers 330 s mais vit jusqu'à 340,4 s.

Ce que dit le journal du run (`logs/run_T200000_20261025_024911.log`, mesuré)

Pool 1 : temps de chaque worker.

Worker	Zéros	Temps (s)	Appels mpmath de secours
1	37 276	188,6	117
4	37 274	195,5	59
2	37 276	203,5	81
6	37 277	207,0	48
3	37 275	212,4	73

Worker	Zéros	Temps (s)	Appels mpmath de secours
5	37 276	216,0	52
7	37 276	218,2	34
0	37 276	**231,4**	**576**

Le partage est égal en nombre de zéros (≈ 37 276 chacun) mais pas en temps : le plus lent dépasse le plus rapide de 23 %. **Calculé :** le worker 0 a fait 576 des 1 040 appels de secours ; à 60,85 ms l'appel (contre 2,06 ms pour le calcul normal), cela fait environ 35 s, contre environ 4 s pour un worker moyen. Cela explique l'essentiel de son retard. Efficacité du pool 1 : 1 672 s de travail pour 8 × 231 s disponibles, soit 90 % (calculé).

Résultat du run principal. 298 200 zéros trouvés, 298 200 attendus (Weyl) ; durée 231,6 s ; 1 287,5 zéros/s. Cette vitesse ne couvre que le run principal : sur les 373 s totales, on tombe à environ 800 zéros/s (calculé). Répartition des méthodes d'affinage : `arb_C` 297 152 (99,6 %), `mpmath_fallback` 1 040, `mpmath_petit_t` 14. Premier zéro $t_1 = 14{,}134725\ldots$, dernier $t_n = 199\,999{,}7053\ldots$.

Comptage par segment et déclenchement du rescan.

Seg	Intervalle de t	trouvés / attendus	Verdict
0	14,0 → 31 185,7	37 275 / 37 273	OK
1	31 185,7 → 57 654,5	37 275 / 37 275	OK
2	57 654,5 → 82 794,2	37 275 / 37 276	⚠ **déficit +1**
3	82 794,2 → 107 138,2	37 275 / 37 275	OK
4	107 138,2 → 130 920,7	37 274 / 37 274	OK
5	130 920,7 → 154 274,0	37 274 / 37 275	⚠ **déficit +1**
6	154 274,0 → 177 281,1	37 276 / 37 274	OK
7	177 281,1 → 200 000,0	37 276 / 37 276	OK

- Le rescan se déclenche quand « trouvés < attendus » pour un segment ; un surplus ne le déclenche pas. **mesuré.**
- Calculé : la somme des « attendus » par segment vaut 298 198, celle des « trouvés » 298 200.
- Message du journal : « 2 segment(s) en déficit → rescan parallèle STEP=0,000681 » (le pas passe de 0,001362 à la moitié). **Un processus par segment en déficit** : à T=100000 on avait un seul processus, ici deux. Cela explique les deux processus de la chronologie. **mesuré / déduit.**
- **Résultat du rescan : « Après fusion rescan : 298200 zéros (+0 nets après déduplication) »** : 74 552 zéros bruts rescannés, tous dédupliqués, aucun zéro ajouté. Coût : environ 104 s, soit 28 % du temps réel, sur 2 cœurs. **mesuré.**
- Durée du rescan (calculé) : le worker 102 du rescan sort son 1000ᵉ zéro à 68,7 s, puis environ 0,7 s par millier ; soit environ 69 s + 26 s ≈ 95 s pour 37 276 zéros, proche des 104 s mesurés. **Déduit :** la majorité du temps du rescan est le balayage fin, avant le premier zéro.

Profil des phases (journal, cumul sur les 8 workers).

Phase	Cumul	Part du « mur × 8 »
`detection` (8 appels)	972,3 s	35,7 %
`arb_C` (298 192 appels, 2,06 ms)	614,3 s	22,6 %
`mpmath_fallback` (1 040 appels, 60,85 ms)	63,3 s	2,3 %
`turing` (1 appel)	16,8 s	0,6 %
`mpmath_petit_t` (1 appel)	16,2 s	0,6 %
Total	**1 682 s**	**61,8 %**

Calculé : 972,3 / 0,357 / 8 ≈ 340,5 s : le « mur » du journal correspond à la fin du rescan. Les 38 % manquants sont des cœurs inactifs ou du temps non compté. Où se placent `turing` et `mpmath_petit_t` dans la chronologie : **à établir** (horodatages du journal).

Interprétation du déficit (heuristique)

Le décompte exact des zéros d'ordonnée ≤ T vérifie la formule de Riemann–von Mangoldt $N(T) = \theta(T)/\pi + 1 + S(T)$, avec $S(T) = (1/\pi)\cdot\arg\zeta(\tfrac{1}{2}+iT)$. Le terme lisse $\theta(T)/\pi + 1$ donne ici 298 199,8 pour T=200000 (**calculé**), à comparer aux 298 200 trouvés. $S(T) = O(\log T)$ est un **théorème** ; qu'il reste typiquement petit (de l'ordre de ±1) est un fait **numérique connu**. Le nombre « attendu » par segment est une valeur lisse arrondie : l'écart sur un segment vaut $S(b) - S(a)$, et un écart de ±1 ou ±2 est donc **compatible avec la fluctuation naturelle**. Le « +0 » du rescan va dans ce sens. **C'est une hypothèse, pas une preuve** : un déficit de 1 pourrait aussi être une paire de zéros très proches manquée, et c'est précisément le cas que le rescan à pas fin veut couvrir. L'égalité des totaux ne certifie pas la complétude ; c'est le rôle de la validation de Turing, que le journal rapporte (voir fiche 1.3 : `COMPLET — aucun zéro manquant`). Sa ligne pour le run T=200000 n'a pas encore été lue.

Ce qui n'est pas expliqué

- Le **load average** a atteint 13,75 à 180 s puis 14,26, alors que 8 workers seulement sont prêts à tourner. Les autres tâches (graphique, htop, échantillonneur) contribuent peut-être. À mesurer avec `vmstat 2` (colonne `r`). **Hypothèse « plusieurs threads par worker » réfutée** à la fiche 1.5 bis (1 thread par worker, mesuré).
- Le **critère exact du déficit** et le **dimensionnement du pool 2** sont lus dans le journal, pas dans le code : à confirmer dans le code (voir Annexe D).

Verdict. OK. La structure d'un run est établie : pool 1 (8 workers, 62 % du temps), rescan (2 processus, 28 %), fin séquentielle (22 s). Écart E16 revérifié : plus de wrapper dans le CSV (v3.2). Nouveaux écarts E17 et E18.

À retenir (bonnes pratiques n°5 et n°6). Relire le journal du run avec `grep -nE "Worker|RESULTATS|RESCAN|déficit|PROFIL" logs/run_*.log`. Fermer la session de turbo dans la capture (`zeta-turbo-off`) et lire « RESTAURÉ ».

Fiche 1.5 bis — Exploiter le journal d'un run : 6 graphiques, fenêtre en direct, repères (capture 1_5)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Tirer du journal d'un run (`logs/run_T<N>_<date>.log`) des graphiques lisibles, à la fin du run et pendant le run, et superposer ses repères (fin de chaque worker, rescan) au graphique des processus de `zeta-proc-plot`.

Machine. PC1 (`riemann@zeta-lab`). Lecture seule : aucun de ces outils ne modifie le calcul ni le journal.

Les trois outils (installés et testés sur PC1 le 25/10/2026).

Alias	Script	Rôle	Sortie
`zeta-log-graph`	`zeta_log_graphes.py` (v1.0)	6 graphiques PNG du dernier journal, après le run	`monitoring/runs/prod/<run>/figures/g1_…g6_*.png`
`zeta-log-live`	`zeta_log_live.py` (v1.0)	fenêtre en direct (4 panneaux), à lancer **avant** le run (`--attendre`) ; écrit les 6 PNG une seule fois à la fin	idem, plus la fenêtre
`zeta-proc-plot`	`zeta_plot_proc.py` (v2)	graphique des processus + repères du journal (`--journal auto` par défaut)	fenêtre ou PNG (`--sortie`, ou `--sortie auto` dans `figures/`)

Installation faite (une commande à la fois) : contrôle de la version à la ligne 3 de chaque fichier, sauvegarde de l'ancien `zeta_plot_proc.py` en `~/projet_zeta/scripts/zeta_plot_proc_v1.bak`, sauvegarde de `~/.bashrc` en `~/.bashrc.avant_zeta-log`, ajout des deux alias. Les alias `zeta-log` et `zeta-logs` qui existaient déjà (autres usages) ne sont pas touchés. Test : `zeta-log-graph` a écrit les 6 PNG dans le dossier du run 20261025_024914_T200000. **mesuré**

Les six graphiques (journal du run T=200000). Chacun n'utilise que des lignes présentes dans le journal ; un bloc absent donne un graphique sauté.

#	Fichier	Ce que montre le graphique	Tiré de
1	`g1_progression.png`	zéros trouvés en fonction du temps pour chaque worker, et débit total du pool	lignes « [Worker k] zéro #N » (une toutes les 1000 zéros)
2	`g2_gantt.png`	chronologie : détection, affinage, attente de chaque worker, puis rescan ; appels mpmath de secours	lignes de fin de worker et « RESCAN CIBLÉ »
3	`g3_anneau_phases.png`	répartition du temps de calcul par phase	`[PROFIL PHASES]`
4	`g4_histogramme_blocs.png`	histogramme du temps par zéro (blocs de 1000), pool 1 contre rescan	lignes de progression
5	`g5_precision_lmfdb.png`	écart de chacun des 20 zéros à la référence LMFDB	tableau LMFDB
6	`g6_completude.png`	trouvés contre attendus par segment, points de contrôle de Turing	tableau « RESCAN CIBLÉ » et validation de Turing

Ce que les graphiques ajoutent à la lecture du journal (mesuré, T=200000).

- Temps par zéro (g4) : médiane 2,00 ms par zéro dans le pool 1 contre 0,60 ms dans le rescan, soit ×3,3. **Hypothèse** (non démontrée) : 8 workers se partagent 4 cœurs logiques (2 physiques), alors que 2 processus de rescan ont chacun un cœur.
- Gantt (g2) : la barre bleue (« départ + détection + 1er millier ») dure de 90 à 158 s selon le worker : c'est le temps écoulé avant la première ligne de progression, pas la détection seule (correction de la v0.8) ; le worker 0 finit à 231,4 s pendant que les autres attendent de 13 à 43 s.
- Précision (g5) : 20 zéros sur 20 sous 1e-9, le pire à 8,06e-10 (zéro n° 20) ; en dessous de t = 65, les écarts sont ≤ 1,6e-13.
- Complétude (g6) : points de Turing +1, +2, +2, 0, 0, verdict COMPLET.

Repères du journal sur le graphique des processus. `zeta-proc-plot` v2 lit le journal et trace des traits verticaux sur les trois panneaux : fin de chaque worker du pool 1 (gris pointillé, étiquettes W0 à W7), début et fin du rescan (orange).

Alignement des horloges. Le CSV compte depuis la détection du run ; le journal, depuis le lancement des workers. Les 8 PIDs du pool 1 disparaissent **tous ensemble** (mesuré au cas 1.5), on ne peut donc pas les aligner sur leur mort. L'outil prend la **dernière seconde où chaque worker a travaillé** (CPU ≥ 20 %), la trie, la compare aux fins du journal triées, et retient la médiane des écarts ; l'incertitude affichée est l'écart maximal à cette médiane.

Première mesure réelle (run 1_5), commande `zeta-proc-plot ~/zeta_captures/proc_1_5_20261025_024422_global.csv --journal ~/projet_zeta/logs/run_T200000_20261025_024911.log` *:*

Grandeur	Valeur	Statut
décalage entre les horloges	0,8 s	**mesuré**
incertitude de l'alignement	±1,3 s (8 workers appariés)	**mesuré** ; même ordre que le pas d'échantillonnage (2 s)
début du rescan, repère du journal	232,4 s	**mesuré**
naissance observée des processus du rescan (CSV)	235,9 s	**mesuré** ; écart de 3,5 s = transition du principal, cohérente avec les ~4 s du cas 1.5
fin du rescan, repère du journal	≈ 339 s	**mesuré** ; tombe sur l'arrêt du second processus du rescan (≈ 340 s)

Un essai préalable sur un CSV **synthétique** (décalage vrai de 2 s) a retrouvé 1,8 s : il valide le code, pas les données. Le premier essai avait donné −0,5 s parce que mon générateur n'imitait pas la convention de l'échantillonneur (le CPU d'un échantillon à *t* couvre *(t − 2 s, t]*) ; c'est le générateur qui était faux, pas l'outil.



Lecture du graphique du run 1_5 (étiquettes : mesuré, déduit, hypothèse).

Période	Ce qu'on voit	Statut
0 à 232 s, pool 1	8 workers actifs sur 4 cœurs logiques, chacun à 30–90 % de CPU (moyenne ≈ 47 %) ; 3,4 à 3,8 cœurs occupés	**mesuré**
id.	8 × 47 % ≈ 375 % : les workers sont limités par le CPU et se partagent les cœurs ; le goulot n'est ni le disque ni l'attente de données	**déduit**
≈ 190 à 232 s	les workers finissent un à un (W1 d'abord, W0 en dernier) ; chaque marche de « workers actifs » suit un trait pointillé à moins de deux pas de 2 s	**mesuré**
≈ 221 à 232 s	W0 tourne seul à 100 % (576 appels mpmath de secours) : c'est lui qui retarde la fin du pool	**mesuré**
≈ 90 s	creux isolé à 6 workers actifs pendant un seul pas ; tombe à la fin de la détection de W0 d'après le journal	**mesuré** ; cause : **hypothèse** (changement de phase), non démontrée
232 à 236 s	seul le principal travaille (pic ≈ 60 %), puis les deux processus du rescan naissent	**mesuré** ; rôle de ces 4 s : **déduit** (rassembler les résultats, choisir les segments), non lu dans le code
236 à 340 s, rescan	2 processus à 100 %, soit 2 cœurs sur 4 ; l'un s'arrête vers 330 s, l'autre vers 340 s	**mesuré**
id.	la moitié de la machine reste libre ~100 s : piste d'accélération si le rescan reste utile	**déduit**
330 à 364 s	« phase finale 34 s » du titre = ≈ 10 s de déséquilibre du rescan + 22 s du principal seul : deux causes sous un même seuil (≤ 1 actif)	**mesuré** ; voir E18
mémoire	≈ 1,9 Go dès le démarrage (8 workers), 2,05 Go au maximum, ≈ 850 Mo avec le rescan, ≈ 400 Mo avec le principal seul ; swap nul	**mesuré**
load average	monte à ≈ 14 vers 200 s puis redescend ; la montée lente est normale (moyenne sur 1 min) ; la valeur de 14 est trop haute pour 8 tâches	**mesuré** ; cause non expliquée (D6)

Test de l'hypothèse « plusieurs threads par worker ». Le CSV du run 1_5 contient le nombre de threads de chaque processus. Commande (PC1) : `awk -F, '$1>100 && $1<104 {print $2, $7}' ~/zeta_captures/proc_1_5_20261025_024422.csv | sort -u`. Résultat **mesuré** : le principal (865752) a 6 threads, chacun des 8 workers (865831 à 865838) en a **un seul**. **L'hypothèse est réfutée** : les workers ne peuvent produire que 8 tâches exécutables. Les ~14 du load average viennent d'ailleurs (autres processus du système, tâches en attente d'entrée-sortie ?) ; la mesure à faire est `vmstat 2` pendant un run (colonne `r` : tâches prêtes ; colonne `b` : tâches bloquées). Le principal a 6 threads alors qu'il est quasi inactif pendant le pool 1 : **hypothèse** (threads de gestion du pool de processus), non lue dans le code.

Limites connues.

- Le repère « début du rescan » est la fin du run principal du journal ; les processus du rescan naissent ~4 s plus tard (mesuré, voir plus haut).
- La légende du panneau du milieu masque une partie des marches 8 → 6 workers actifs : **écart E20**.
- `zeta-log-live` a été testé par rejeu, puis **validé sur deux vrais runs en direct** (fiche 1.6, T=20000 et T=200000). Le journal est écrit en temps réel (pas de tampon) : **mesuré**.
- Pendant la détection (de l'ordre de 90 à 160 s au début d'un run T=200000) le journal n'a aucune ligne de progression : la fenêtre affiche « En attente des premières lignes » ; ce n'est pas une panne.

Curiosités du journal repérées en l'exploitant (à confier à Claude Code, voir D10).

- « Surplus total : 5 » = 1 + 2 + 2, la somme des surplus aux points de contrôle de Turing ; l'écart final vaut 0.
- L'en-tête annonce « T=10 000 → ~4516 zéros » et « T=100 000 → ~49k » : valeurs qui semblent périmées (le nombre de zéros ordonnée ≤ 10 000 est ≈ 10 142 et ≤ 100 000 ≈ 138 070, **calculé** par la formule de Weyl).
- « Seuil petit-t : 65.0 » dans le journal contre `T_SEUIL_PETIT_T=20` dans la ligne de lancement : deux valeurs pour un même seuil, ou deux seuils distincts ?

Verdict. OK pour `zeta-log-graph` et `zeta-proc-plot` v2 (testés sur des données réelles). `zeta-log-live` : testé par rejeu, **validé sur un run réel à la fiche 1.6**.

À retenir (bonnes pratiques « Avant » n°14 et « Après » n°8). Lancer `zeta-log-graph` après chaque run. Lancer `zeta-log-live` avant le run (il attend le journal), mais pas sur un gros run de production tant que la règle « pas de fenêtres de graphiques » n'est pas relue (guide, Pendant un run, n°1).

Fiche 1.6 — Run « test » avec fenêtres de monitoring, puis validation de `zeta-log-live` sur un run long (captures 1_6 et 1_6b)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. (a) Voir ce que fait le mode `test` du lanceur (3 fenêtres de monitoring rangées) sur un run court ; (b) valider `zeta-log-live` en direct sur un run long (T=200000) ; (c) comparer deux runs identiques pour mesurer la reproductibilité ; (d) comprendre chaque phase d'un run.

Machine. PC1 (`riemann@zeta-lab`). **Commandes.**

Run	Commande	Fenêtres	Durée
1_6 (test)	`ZETA_MON_TYPE=test scripts/zeta_run.sh 20000`, puis `zeta-mon-fermer`	3 fenêtres de monitoring + `zeta-log-live` lancé avant	real 55,4 s ; user 100,05 s ; sys 0,64 s. **mesuré**
1_6b (prod)	`time scripts/zeta_run.sh 200000` avec `zeta-proc --test 1_6b`, `zeta-log-live` et `htop` ouverts avant	aucune fenêtre de monitoring	real 6 min 19,2 s (379,2 s) ; user 1001,7 s ; sys 2,43 s. **mesuré**

Dans les deux cas : `zeta-turbo-off` à la fin (le run laisse le mode performance actif, écart E13) et `zeta-mon-fermer` pour les fenêtres.

(a) Le mode test : trois fenêtres

`ZETA_MON_TYPE=test` ouvre trois fenêtres rangées : histogramme des espacements comparé à la loi GUE, machines du cluster (température, fréquence, load, RAM), zéros par tranche de t. Le mode `prod` (défaut) n'en ouvre aucune (règle « pas de fenêtres sur un gros run », risque de gel déjà vécu).

Observation à T=20000	Valeur	Statut
zéros trouvés	22 491 ; la fenêtre en direct indique 22 497 écrits (relevé sur le run du cas 1.6 bis, même T ; écart de 6 non expliqué)	**mesuré**
rescan	3 processus (segments 3, 5, 7) ; durées 8,4 / 5,9 / 6,2 s relevées sur le run du cas 1.6 bis (même T)	**mesuré**
durée totale	55,4 s contre 46,0 s à la fiche 1.3 (+20 %)	**mesuré** ; cause : **hypothèse** (fenêtres et rapport), non testée
histogramme des espacements	pic ≈ 1,05, la surmise de Wigner culmine à 0,94	**mesuré** ; voir D16 (**hypothèse** : effet de hauteur finie, ou normalisation)

(b) `zeta-log-live` en direct sur T=200000 : validé

Le journal est écrit **en temps réel** (une ligne dès qu'elle existe, pas de tampon) : **mesuré**, c'est la question restée ouverte à la fiche 1.5 bis. La fenêtre se met à jour progressivement, par pas de 1000 zéros par worker.

Instant (depuis le lancement)	Ce que montre la fenêtre	Statut
0 à ≈ 66 s	« En attente des premières lignes » : la détection n'écrit rien ; la première ligne (W0) arrive vers 66 s, les autres workers entre 84 et 178 s	**mesuré**
77 à 235 s	progression par worker, débit du pool, Gantt, histogramme du temps pour 1000 zéros, pourcentage de zéros écrits	**mesuré**
fin du run	« TERMINÉ », les 6 PNG sont écrits dans `monitoring/runs/prod/<run>/figures/`	**mesuré**

Un défaut a été trouvé et corrigé pendant cet essai : la fenêtre affichait **138 %** de zéros écrits, parce qu'elle additionnait aussi les zéros des processus du rescan (écart E22, corrigé en v1.1). Autres corrections (kit v1.3 / v1.2) : titre qui indique la phase (pool 1, rescan, validation, terminé), dernière fenêtre de débit normalisée par sa durée réelle, durée du rescan écrite dans la colonne de droite du Gantt.

(c) Reproductibilité : deux runs T=200000 identiques (1_5 et 1_6b)

Grandeur	Run 1_5	Run 1_6b	Écart
durée réelle (`time`)	373,5 s	379,2 s	+1,5 %
temps CPU utilisateur	991 s	1001,7 s	+1,1 %
CPU moyen (`zeta-proc`)	248 %	249 %	—
phase finale	34 s	34 s	0
appels mpmath de secours par worker (W0 à W7)	576, 117, 81, 73, 59, 52, 48, 34	identiques	0
appels mpmath au total / appels `arb_C`	1 040 / 298 192	1 040 / 298 192	0
fin de chaque worker	—	varie de ±22 s d'un run à l'autre	**mesuré**
load average maximal	14,26	14,83	—

Le calcul est **déterministe** (mêmes zéros, mêmes appels de secours), seule la durée fluctue d'environ 1 à 2 %. Le run 1_6b a eu lieu **sans** fenêtre `zeta-proc-plot` : l'hypothèse « ce graphique ralentit le run » est réfutée (+1,5 % de durée, pas de gain), et le load de 14,83 ne vient pas non plus de lui. La cause du load de ~14 pour 8 tâches reste non expliquée (D6) : mesure à faire, `vmstat 2` pendant un run (colonnes `r` et `b`).

Résumé de `zeta-proc` pour 1_6b (**mesuré**) : 370 s observées, CPU moyen 249 %, maximum 386 %, 8 processus actifs au plus, phase finale 34 s (9 %), mémoire maximale 2046 Mo, au moins 10 010 Mo disponibles, swap nul.

(d) Les phases d'un run, expliquées (sources : journal et documents du projet ; le code n'a pas été relu)

Phase	Ce qui se passe	Statut
petit t (t < 65)	14 zéros calculés avec mpmath, en un seul appel	**mesuré** (journal) ; durée variable : 16,2 s contre 7,1 s (D14)
détection	balayage de la droite critique par pas fixe (STEP ≈ 0,001362 à T=200000, soit environ 1/445 de l'espacement moyen 0,606) ; ≈ 1,468 × 10^8 évaluations de Z(t) par la formule de Riemann–Siegel avec corrections C0 + C1 (erreur en O(t^-3/2)), en C (`scan_arb.so`, double précision) ; un **changement de signe** de Z(t) = un zéro (théorème des valeurs intermédiaires)	**mesuré** (compteurs), méthode : documents du projet
raffinage	pour chaque changement de signe, méthode d'Illinois (variante de la fausse position) avec évaluations Arb en 64 bits fixes, tolérance 10^-12	documents du projet
secours mpmath	quand le raffinage Arb ne convient pas : 1 040 appels à T=200000 (55 % dans W0), identiques d'un run à l'autre ; **critère de déclenchement non lu** (D13)	**mesuré** (nombre), critère inconnu
rescan	un processus par segment dont le compte de zéros est en déficit : 1 à T=100000, 2 à T=200000 (segments 2 et 5), 3 à T=20000 (3, 5, 7) ; ≈ 104 s à T=200000 pour **+0 zéro net**, soit 28 % du temps du run (D1)	**mesuré**
validation Turing–Backlund	N(T) = θ(T)/π + 1 + S(T) (théorème), vérifié à 5 points de contrôle ; l'égalité avec le compte obtenu prouve que rien ne manque **si** S(T) est numériquement correct	théorème + **mesuré** (verdict COMPLET)

Structure dans le temps (T=200000, **mesuré***).* Pool 1 de 8 workers (≈ 232–236 s), transition d'environ 4 s (le principal travaille seul à ≈ 60 % de CPU), rescan (≈ 104 s), principal seul (≈ 22 s) : total 373 à 379 s. Les temps du `[PROFIL PHASES]` sont des temps d'horloge à l'intérieur des workers : 8 workers se partagent 4 cœurs logiques, chacun à ≈ 45 % de CPU, donc la durée d'horloge vaut environ le double du temps CPU.

Le camembert. Dans le camembert de phases de `zeta-proc`, « mur × 8 » est environ la durée du journal jusqu'à la fin du rescan. La part grise de 37,7 % correspond (**déduit**) aux attentes de fin de worker (≈ 178 s cumulées) plus la période du rescan où seuls 2 processus travaillent (8 × ≈ 105 s).

Lire `zeta-progress` pendant un run (écart E21)

Ce qu'il affiche	Ce que cela signifie	Statut
pourcentage plafonné à 99,3 % (71,1 % à T=20000)	seul le dernier pas de 1 000 zéros de chaque worker est compté : 8 × 37 000 / 298 200 à T=200000, 8 × 2 000 / 22 491 à T=20000	**mesuré**, corrigé et vérifié à la fiche 1.6 ter (correction de la v0.9 : la v0.8 disait « ne compte que le pool 1 »)
« ETA(√t) 0,0 min »	artefact du plafond : l'estimation tombe à zéro alors que le run dure encore ≈ 140 s	**mesuré**
« count @ t »	hauteur sur la ligne critique (valeur de t), pas des secondes ; correspond au tableau des segments	**mesuré**
rescan et fin du run	ignorés par `zeta-progress` (≈ 37 % du temps du run) ; utiliser `zeta-log-live`	**mesuré**

Questions ouvertes de la fiche

#	Question	Piste
1	+20 % de durée à T=20000 entre 1.3 (46,0 s) et 1.6 (55,4 s)	**résolu** aux fiches 1.6 ter et 1.7 bis : les fenêtres coûtent 3 à 4 s ; le reste vient des outils ouverts à côté (D18)
2	détection ×80 pour T×10 ; `arb_C` 4,42 ms contre 2,16 ms par appel	D12, E24
3	critère du secours mpmath et sa décroissance avec t	D13
4	298 192 appels `arb_C` pour 298 200 zéros	D14
5	load de ~14 pour 8 workers	D6, `vmstat 2`

Verdict. OK. Le mode test, `zeta-log-live` en direct et la reproductibilité du calcul sont vérifiés ; trois questions de mesure restent ouvertes (tableau ci-dessus).

À retenir (bonnes pratiques « Avant » 14, « Après » 10). Le journal est lisible en direct, mais sa première ligne n'arrive qu'après 66 s à T=200000 : une fenêtre vide pendant ce temps est normale.

Fiche 1.6 bis — Lancement automatique de `zeta-proc`, du graphique des processus et du journal en direct

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Demande de la fiche 1.3 : que les outils d'observation s'ouvrent seuls avec les fenêtres de monitoring. Il fallait lire `zeta_mon_run.py` (351 lignes) et `zeta_mon_common.py` (144 lignes), fournis dans le projet.

Ce que le code fait (lu, mesuré **sur le texte).** Le lanceur crée un dossier de run, puis un **superviseur** (priorité minimale) qui écrit `etat.json` toutes les 5 s et ouvre les fenêtres via `ouvrir_fenetres()` ; à la fin du run il génère `rapport.md` et `rapport.pdf`. Le type `test` ouvre les fenêtres, `prod` non. Les fenêtres sont des processus séparés ; leurs PIDs sont gardés

dans `runs/.fenetres.json`, que `zeta-mon-fermer` lit pour les arrêter.

Correctif v1 (`zeta_patch_mon_extra_v1.py`). Il crée une copie corrigée de `zeta_mon_run.py` (l'original reste intact tant qu'on ne le remplace pas) :

Élément	Comportement
activation	variable `ZETA_MON_EXTRA=oui` (désactivé par défaut)
échantillonneur	`zeta_proc_sample.py --intervalle 2 --test <nom du run> --attente-max 600`, sans fenêtre, hors de la liste des PIDs : il écrit son résumé puis s'arrête seul
fenêtres ajoutées	`zeta_plot_proc.py ~/zeta_captures --apres <début du run>` et `zeta_log_live.py --attendre`
indépendance	avec `ZETA_MON_EXTRA=oui` et un run `prod`, seuls les outils ajoutés s'ouvrent, pas les 3 fenêtres de base

Installation sur PC1 (faite le 25/10). Sauvegarde `zeta_mon_run.py.avant_extra.bak`, application sur `/tmp/zeta_mon_run.new.py` (compilation OK, 39 lignes de différence), remplacement, `--help` fonctionnel, 4 occurrences de `ZETA_MON_EXTRA`. Le kit v1.3 / v1.2 a été installé le même jour (sauvegardes `.v11.bak`).

Test réel. `ZETA_MON_TYPE=test ZETA_MON_EXTRA=oui scripts/zeta_run.sh 20000` :

Mesure	Valeur	Statut
fenêtres ouvertes	5 (3 de monitoring + graphique des processus + journal en direct)	**mesuré**
durée	real 56,2 s ; user 99,5 s ; sys 0,67 s	**mesuré**
surcoût des outils ajoutés	+0,8 s (+1,5 %) par rapport au run 1_6 à 55,4 s	**mesuré** (un seul essai)
journal suivi par `zeta-log-live`	`run_T20000_20261025_163029.log`, trouvé seul	**mesuré**
graphique des processus	fichier `proc_T20000_…163034_global.csv` suivi automatiquement ; CPU moyen 174 % ; phase finale 14 s ; alignement ±1,9 s (8 workers)	**mesuré**
PNG de fin de run	6 PNG écrits dans `monitoring/runs/test/20261025_163033_T20000/figures`	**mesuré**
fermeture	`zeta-mon-fermer` : « 5 fenêtre(s) fermée(s) », une seconde fois « 0 » ; `zeta-turbo-off` : `TURBO: RESTAURÉ`	**mesuré**

Limites. `zeta_plot_proc.py` et `zeta_log_live.py` n'ont pas d'option de position : les fenêtres se chevauchent, à ranger avec Super et les flèches (D15). Le correctif n'est pas encore intégré au dépôt git. Ne pas l'activer sur un gros run de production sans relire la règle « pas de fenêtres de graphiques » (risque de gel).

Piège rencontré. Les commandes `zeta-mon-*` ne figurent pas dans `~/.bashrc` : ce sont des **fonctions** définies dans `monitoring/zeta_monitoring_alias.sh`, chargées par la ligne `source "…/zeta_monitoring_alias.sh"` du `~/.bashrc` (écart E25). `type zeta-mon-fermer` affiche leur définition.

Verdict. OK pour la v1 du lancement automatique. Reste : positions des fenêtres (D15), intégration au dépôt, documentation dans le guide des scripts (D17).

Fiche 1.6 ter — Run T=20000 en production, sans aucune fenêtre (capture 1_6d), rapport de monitoring et `zeta-progress`

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. (a) Séparer le coût des fenêtres du coût de T (question 1 de la fiche 1.6) ; (b) lire le `rapport.md` généré automatiquement ; (c) observer `zeta-progress` à deux instants du run ; (d) tester sur des données la piste D16 (variance des espacements).

Machine. PC1. **Quatre terminaux** : A (session capturée `script -q -f -T $CAP/1_6d.tm $CAP/1_6d.log`, puis `zeta`, puis `time scripts/zeta_run.sh 20000`), B (`htop`, filtre `compute`), C (`zeta-proc --intervalle 2 --test 1_6d`, lancé avant), D (`zeta-progress`, lancé quelques secondes après le début du run). Pas de `ZETA_MON_TYPE=test` ni de `ZETA_MON_EXTRA` : aucune fenêtre ne s'ouvre. État initial : aucun calcul ni graphique, turbo INACTIF (powersave 4/4). Fin : `zeta-turbo-off` **dans la session capturée**, puis `exit`.

(a) Coût des fenêtres : trois runs T=20000 comparés

Run	Fenêtres	real	user	`zeta-proc` : durée / CPU moyen / phase finale	Statut
1_6d	aucune	**51,99 s**	94,5 s	44 s / 174 % / 14 s	**mesuré**
1_6	3 de monitoring	55,4 s	100,05 s	—	**mesuré**
1.6 bis	3 de monitoring + 2 outils (`ZETA_MON_EXTRA`)	56,2 s	99,5 s	44 s / 174 % / 14 s	**mesuré**
1.3 (référence)	—	46,0 s	89,6 s	—	**mesuré** (fiche 1.3)

Les fenêtres coûtent de 3 à 4 s de durée réelle et de 5 s de CPU (soit 6 à 8 %) ; le profil CPU du calcul est identique. Elles n'expliquent pas l'écart avec la fiche 1.3 : sans fenêtre, 52,0 s contre 46,0 s (+13 %). Chaque ligne n'a qu'un essai. **Correction de la v0.10 :** ce +13 % est **expliqué à la fiche 1.7 bis** : `htop`, `zeta-progress`, `zeta-proc` et `script`, ouverts à côté pendant 1_6d, coûtent environ 8 s ; l'hypothèse « température » est écartée.

(b) Le rapport de monitoring (`monitoring/runs/prod/20261025_170037_T20000/rapport.md`)

Le dossier contient `etat.csv`, `etat.json`, `figures/`, `meta.json`, `metrics.csv`, `rapport.md`, `rapport.pdf`, `session.md`, `superviseur.log`, `superviseur.pid`. Le rapport est généré à la fin du run, **même en production sans fenêtre** (**mesuré**).

Point du rapport	Valeur	Lecture
Turing–Backlund	COMPLET	22 491 zéros ; formule de Riemann–von Mangoldt : 22 491,0, écart −0,0. Le rapport le précise lui-même : vérification numérique, pas une certification
durée suivie	43 s (17:00:37 → 17:01:20), contre 52 s pour `time`	les ≈ 9 s d'écart tombent avant et après la fenêtre suivie : **déduit** des horodatages (journal 17:00:33, suivi 17:00:37, fin d'après `time` 17:01:25), probablement la préparation du turbo et la restauration des services
débit moyen	523 zéros/s	calculé sur les 43 s
PC1	température moyenne 71,6 °C, maximale 76,0 °C, **18 % du temps ≥ 75 °C** ; load max 3,84 ; RAM max 28,8 %	**mesuré** ; seuil d'avertissement atteint sur un run de 43 s
PC2 à PC5	inactives (température ≤ 52,5 °C, load ≤ 1,33)	**mesuré**
débit instantané	« non exploitable »	limite connue de la v16 : le CSV n'est écrit qu'en fin de worker

(c) `zeta-progress` : corrections apportées à la fiche 1.6

Deux captures : à ≈ 8 s (17:00:41) et à ≈ 37 s (17:01:10) après le lancement.

Ce qui est affiché	À 8 s	À 37 s	Lecture	Statut
barre du haut « PC1 100,0 % (N≈22491) — totalité de T_MAX »	100 %	100 %	constante : ce n'est **pas** une barre d'avancement ; sens probable : part de T_MAX confiée au PC1	**mesuré** (2 points) ; sens : **hypothèse** (code non lu)
barre du dessous « zéros 1000/22491 » puis « 16000/22491 »	4,4 %	71,1 %	16 000 = 8 workers × 2 000 : seul le dernier **pas de 1 000** de chaque worker est compté (≈ 2 811 zéros par worker)	**mesuré**, arithmétique vérifiée
plafond	—	71,1 % à T=20000	même mécanisme que le 99,3 % à T=200000 (8 × 37 000 / 298 200)	**mesuré**
« Détail workers (count @ t) »	—	W0 : #2000 @ t=2515 … W7 : #2000 @ t=19366	t est une **hauteur** sur la droite critique ; les segments sont consécutifs	**mesuré**
rescan R3, R5, R7 (id 103, 105, 107)	—	#2000 @ t = 10247, 14899, 19366	mêmes t que W3, W5, W7 : le rescan recalcule les mêmes segments	**mesuré**
« ETA(√t) 0,1 min »	« 1/8 workers démarrés » : ≥ 0,1 min	0,1 min	formule fondée sur √t ; non fiable en fin de run	**mesuré** (affichage)

(d) Variance des espacements normalisés selon la hauteur (piste D16)

Espacement normalisé $s = (\gamma_{n+1} - \gamma_n) \cdot \ln(\gamma_n / 2\pi) / 2\pi$; moyenne 1 par construction. Variance de s, calculée sur les CSV de zéros du projet (**mesuré**, un script de 15 lignes, zéros distants de moins de 10^{-6} fusionnés) :

Zéros	Nombre	Variance	Part s < 0,1
t ∈ [14 ; 20 000] (ce run, selon le rapport)	22 491	0,1569	0,062 %
t ∈ [14 ; 100 000]	138 069	0,1614	0,073 %
$t \in [4{,}9 \cdot 10^6 ; 5 \cdot 10^6]$	216 081	0,1693	0,095 %

Dans le fichier à 100 000, la variance par quart passe de 0,1581 à 0,1631 ; dans celui à $5 \cdot 10^6$ elle est constante à 0,1693 (4 quarts). **Mesuré** : la variance augmente avec la hauteur et reste sous la valeur GUE asymptotique (≈ 0,18, **théorie des matrices aléatoires**). **Hypothèse** (non démontrée ici) : effet de hauteur finie ; la littérature le décrit (Bogomolny, Bohigas, Leboeuf, Monastra, 2006, cité de mémoire, **à vérifier**), je n'ai pas vérifié que cet effet explique les valeurs exactes. Une variance plus petite que celle de la surmise de Wigner correspond à une distribution plus étroite, donc à un pic plus haut : cela s'accorde avec le pic de 1,05 observé à la fiche 1.6.

Verdict. OK. Le coût des fenêtres est mesuré (6 à 8 %), le rapport de production est lu, `zeta-progress` est corrigé, D16 est passé de l'hypothèse à une tendance mesurée. Le +13 % par rapport à la fiche 1.3 est expliqué à la fiche 1.7 bis (outils ouverts à côté).

À retenir (bonnes pratiques « Après » 11 et 12). Lire `rapport.md` ; ne pas comparer sa durée suivie (43 s) à `time` (52 s) sans savoir qu'elles ne couvrent pas la même période.

Fiche 1.7 — Rapports FR et EN, index des runs et interprétation (captures du 25/10, run `20261025_170037_T20000`)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Vérifier ce que font `zeta-report` (français, anglais), `zeta-index` et le fichier `interpretation.md`, et relever ce qui manque avant d'écrire des rapports définitifs en anglais.

Machine. PC1. **Dossier testé.** `~/projet_zeta/monitoring/runs/prod/20261025_170037_T20000/`.

(a) Générer la version anglaise

Mesure	Valeur	Statut
commande	`zeta-report "$PWD" --lang en`	—
durée	3,98 s (user 4,07 s)	**mesuré**
fichiers créés	`report_en.md` (4 449 octets), `report_en.pdf` (726 964 octets)	**mesuré**

Mesure	Valeur	Statut
rapports français	`rapport.md` (4 791 octets) et `rapport.pdf` (727 818 octets) : empreintes md5 **identiques avant et après** (`81edf6c4…`, `f3112a83…`)	**mesuré** : le français n'est pas touché
fichiers réécrits	`meta.json`, `metrics.csv`, `session.md` (même taille, heure modifiée)	**mesuré** ; contenu identique : non vérifié

Pourquoi ces réécritures. D'après le code de `zeta_run_report.py` (lu), `rapport()` relit les mesures des 5 machines pour la période du run (fichier journalier du logger) et réécrit `metrics.csv` et les figures à chaque appel. **Déduit** (non testé) : si le fichier journalier des mesures a été supprimé, un nouvel appel écraserait `metrics.csv` par des données vides ; relancer `zeta-report` seulement tant que ce fichier existe.

(b) Ce que contient un rapport (lu sur la version anglaise)

Six sections : 1 résumé (durée, code retour, turbo, zéros, débit moyen) ; 2 machines pendant le run (température, fréquence, charge, RAM) ; 3 zéros calculés (comptage comparé à la formule de Riemann–von Mangoldt, histogramme des espacements comparé à GUE et à Poisson, débit) ; 4 fiche de session ; 5 interprétation ; 6 limites (mesures environ toutes les 1 s par SSH, début et fin du run à quelques secondes près, sous-échantillonnage au-delà de 8 000 points par machine). Chaque affirmation porte une étiquette `[mesuré]`, `[heuristique]`, `[hypothèse]` ou `[conjecture]`. Le verdict Turing–Backlund est **lu dans le journal d'exécution** voisin du CSV (`execution_*.log`) ; le rapport précise que c'est une vérification numérique, pas une certification.

(c) Défauts relevés dans la version anglaise

#	Constat	Origine (code lu)	Verdict
1	`COMPLET` non traduit (2 lignes)	le statut est recopié tel quel du journal (« valeurs lues, jamais devinées »)	voulu ; à traduire seulement si l'on ajoute une table de correspondance
2	`actif (intel_pstate)` (3 lignes)	texte de `turbo_etat()` dans `zeta_mon_run.py`, stocké en français dans `meta.json`	oubli (E26)
3	en-tête `T moy °C`	texte fixe	oubli (E26)
4	`11.6 Go` et pied de page « MAJ ... lignes »	`ecrire_md()` de `zeta_mon_common.py`	oubli (E26)
5	« at moderate heights » (le français dit « sur de petites hauteurs »)	traduction non fidèle	à corriger (E26)
6	le rapport parle de corrections de « taille finie » pour la variance des espacements	formulation ; nos mesures (fiche 1.6 ter) montrent une dépendance en **hauteur** t	« hauteur finie » serait plus précis ; hypothèse de formulation

(d) Le fichier `interpretation.md`

Le rapport indique « *Not written yet. ... create* `interpretation.md` *in this folder then run* `zeta-report` ». Test : un fichier de 5 puces (757 octets, vérifié) a été créé dans le dossier du run, puis les deux rapports ont été régénérés. **Mesuré** : la section 5 des deux rapports reprend les puces.

Constat	Conséquence
le texte français apparaît **tel quel** dans le rapport anglais	pour un rapport définitif en anglais, il faudra une interprétation en anglais ; le code ne lit apparemment qu'un seul fichier (**hypothèse**, à confirmer) : E28
la section 5 affiche « *Rédigée par Claude Code à partir des chiffres ci-dessus — à valider.* » (« *Written by Claude Code... »* en anglais)	cette ligne est **fixe**, quel que soit l'auteur réel du fichier : l'attribution peut être fausse (E27, D19)

(e) L'index des runs

`zeta-index` régénère `monitoring/Monitoring-Runs.md` en 0,84 s : un tableau (date, run, état, durée, zéros, débit, température maximale de PC1, liens) trié du plus récent au plus ancien, puis le dernier run avec figures, la galerie de référence et la liste des commandes. Le fichier est **suivi par git** : après `zeta-index`, `git status` le montre modifié (normal).

(f) Incident de collage

Un bloc de commandes collé contenait deux lignes parasites (`</parameter>`, `</invoke>`) : bash a répondu « erreur de syntaxe près du symbole inattendu » et ignoré le bloc. Pourtant le fichier `interpretation.md` avait bien été créé (début du bloc exécuté). **Réflexe :** après une erreur de syntaxe sur un bloc collé, contrôler ce qui existe (`ls -l`) avant de relancer.

Verdict. OK : `zeta-report` en deux langues, `zeta-index` et `interpretation.md` fonctionnent. À corriger avant les rapports définitifs en anglais : E26 à E28, D19.

Fiche 1.7 bis — Les outils d'observation ralentissent le run (captures 1_6d, 1_6e et index)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Question. Le même run T=20000 dure 37 s ou 43 s de durée suivie selon les jours. Pourquoi ? Cela compte pour votre objectif : mesurer un gain d'optimisation de quelques pour cent suppose des chronométrages comparables.

Données de l'index (mesuré**) : T=20000, mode production sans fenêtre de monitoring.**

Run	Durée suivie	real (`time`)	user	Débit	T max PC1	Outils ouverts à côté
24/10 22:56 (fiche 1.3)	37 s	46,0 s	89,6 s	607,9 z/s	77 °C	—
25/10 17:00 (1_6d)	43 s	52,0 s	94,5 s	523,0 z/s	76 °C	`htop`, `zeta-progress`, `zeta-proc`, `script`
25/10 17:36 (1_6e)	**36 s**	**44,3 s**	**89,7 s**	624,8 z/s	77 °C	**aucun**
25/10 14:29	46 s	—	—	488,9 z/s	79 °C	3 fenêtres de monitoring

Run	Durée suivie	real (`time`)	user	Débit	T max PC1	Outils ouverts à côté
25/10 16:30	46 s	56,2 s	99,5 s	488,9 z/s	80 °C	3 fenêtres + 2 outils

Écart constant de ≈ 9 s entre `time` et la durée suivie du rapport, sur quatre couples (46,0 − 37 ; 52,0 − 43 ; 379,2 − 370 ; 373,5 − 365) : **déduit**, l'enveloppe de `zeta_run.sh` (préparation du turbo avant, restauration des services après). Il est donc inutile de chercher l'écart de 6 s dans l'enveloppe.

Hypothèses testées (état au 25/10).

Hypothèse	Test	Résultat
H0 : les fenêtres de monitoring seules	fiche 1.6 ter : 3 à 4 s	coûtent, mais n'expliquent pas tout
H1 : le code de calcul a changé	`git status` (seuls `Monitoring-Runs.md` et `zeta_mon_run.py` sont modifiés, tous deux sans rapport avec le calcul) ; dates des `.so` (`scan_arb.so` du 06/09, `illinois_arb.so` du 08/08, `illinois_mpfr.so` du 05/07)	**écartée** : même commit 72bb991, même gouverneur, mêmes bibliothèques
H1 bis : la température	T max de 77 °C pour le run le plus rapide, 76 °C pour un plus lent	**écartée**
H2 : les outils ouverts à côté	run 1_6e sans rien d'autre : 44,3 s contre 52,0 s	**appuyée** (un essai par cas) : +7,7 s réels, +4,8 s de CPU, soit 17 % ; +20 % de durée suivie (43 s contre 36 s)

Le CPU utilisateur sans outil (89,7 s) retombe sur celui de la fiche 1.3 (89,6 s) : la quantité de calcul est la même ; les outils prennent du CPU sur 4 cœurs logiques déjà saturés par 8 workers. On ne sait pas lequel des quatre pèse le plus : `zeta-progress` interroge PC2 par SSH toutes les 4 s (suspect, **non mesuré**).

Note sur la fiche de session. Le champ « N fichiers modifiés non commités » compte aussi les fichiers non suivis (dossiers de run, captures) : 21 le 24/10 et 35 le 25/10 ne signifient pas 14 fichiers de code modifiés en plus.

Règle à retenir (bonnes pratiques « Avant » 18 et « Pendant » 9). Pour **chronométrer** un run (comparer deux versions du code, valider une optimisation), ne rien ouvrir à côté et lire la durée dans le rapport ou l'index. Pour **comprendre** un run, ouvrir les outils et accepter qu'il soit plus lent. Vérifier avant de lancer : `pgrep -f 'htop|zeta_run_progress|zeta_proc_sample|zeta_log_live|zeta_plot_'` ne doit rien afficher.

Verdict. OK : le surcoût est expliqué, la mesure de référence de T=20000 est **36 s de durée suivie (44,3 s réels)**, reproductible à 3 % près (37 s et 36 s, un jour d'écart). Reste à mesurer outil par outil (D20).

Fiche 1.8 — Cartographie : quels fichiers un run écrit, dans quel ordre, et ce qu'il charge (captures du 25/10, runs T=20000)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Savoir où un run écrit ses résultats (pour savoir quoi sauvegarder), dans quel ordre, et quelles bibliothèques il charge. Complète l'annexe B.

Machine. PC1. **Outils.** `inotifywait` (installé pour l'occasion : `sudo apt install -y inotify-tools`, 54,5 ko, paquet `inotify-tools` 3.22.6), `find -newer`, `lsof`. `strace` est installé mais **non utilisé** (voir limites).

Outil	Question	Poids mesuré
`inotifywait -m -r` sur `logs/`, `calculs/`, `monitoring/runs/`, `/run/zeta_turbo/`	quels fichiers sont créés, fermés, déplacés, supprimés, et quand	run à 46,5 s réels (user 88,4 s) contre 44,3 s sans outil : ≈ +5 % (un essai)
`find ... -newer repère`	quels fichiers ont changé ailleurs, depuis un repère (`touch`) posé avant le run	aucun
`lsof -p <PID>` à 14 s	quels fichiers et bibliothèques les processus du calcul ont ouverts à cet instant	run à 45,1 s (user 90,7 s)

(a) Chronologie d'un run T=20000 (112 événements, run lancé à 17:47:01)

Heure	Événement	Lecture
17:47:01 → 05	`/run/zeta_turbo/state` écrit 8 fois (fichier temporaire `.state.<PID>` puis renommage)	préparation du turbo, ≈ 4 s ; écriture **atomique**
17:47:05	création de `logs/run_T20000.pid`, du journal `logs/run_T20000_<date>.log`, du dossier de suivi, de `monitoring/runs/.actif`, de `superviseur.pid`	début du suivi
17:47:06	`calculs/v16_T20000_<date>/` et `checkpoints/`	début du calcul
17:47:18 → 22	`worker_006`, `000`, `005`, `007`, `001`, `002`, `003`, `004` : un CREATE puis un CLOSE_WRITE chacun	chaque worker écrit son checkpoint **en une fois, à sa fin** (12 à 16 s après le début du calcul) ; aucun `.partial.csv` à T=20000
17:47:23	`zeros_v16_…_principal.csv`	résultat brut du principal
17:47:23 → 41	**18 s sans écriture** (38 % du run)	rescan, validation Turing–Backlund et consolidation : **déduit** (cohérent avec la phase finale de `zeta-proc`)
17:47:41 → 42	CSV final, `.png`, `execution_v16_…log`	résultats définitifs
17:47:43	`.fin`, puis `metrics.csv`	fin du run signalée au superviseur
17:47:43 → 46	3 figures, `session.md`, `rapport.md`, `rapport.pdf`, suppression de `.actif`	rapport : ≈ 3 s
17:47:47	journal du run fermé	dernière écriture du journal
17:47:48	`state` déplacé, `run_T20000.pid` supprimé	fin de `zeta_run.sh` : 47 s (46,5 s pour `time`)
17:48:35	`state` déplacé puis supprimé	l'appel de `zeta-turbo-off` (47 s plus tard)

Fréquences d'écriture du suivi (**mesuré**) : `etat.json` 9 fois (toutes les 5 s), `etat.csv` 6 fois (seulement quand le CSV du run est trouvé), `meta.json` 3 fois.

(b) Écritures hors des quatre dossiers surveillés

`find -newer` (repère posé avant le run, hors `zeta_env`, `.git`, `calculs`, `monitoring/runs`) ne trouve que trois fichiers : le journal du run, `monitoring/data/cluster_metrics_<date>.csv` (écrit en continu par le logger, indépendamment du run) et `monitoring/Monitoring-Runs.md`. **Rien dans** `/tmp`**.** Donc **chaque run modifie** `Monitoring-Runs.md`, fichier suivi par git : un `git status` après un run le montre toujours modifié (pas seulement après `zeta-index`).

(c) Carte des écritures d'un run (mesurée)

Où	Contenu	À sauvegarder ?
`calculs/v16_T<N>_<date>/`	8 checkpoints, CSV principal, **CSV final**, graphique, journal d'exécution	oui
`logs/`	journal du run, fichier `.pid`	oui (profil des phases, validation)
`monitoring/runs/<type>/<run>/`	suivi, figures, rapport	utile
`monitoring/Monitoring-Runs.md`	index des runs	régénérable
`monitoring/data/cluster_metrics_*.csv`	mesures des 5 machines	utile (nourrit les rapports)
`/run/zeta_turbo/`	état du turbo	non : en mémoire, perdu au redémarrage

(d) Ce que les processus du calcul ont ouvert (`lsof`, instantané à 14 s)

Observation	Lecture
`scan_arb.so` et `illinois_arb.so` (dans `src/calculs/optimisation/c_modules/`) mappés par **8 processus**	les 8 workers chargent les bibliothèques C du balayage et du raffinage Arb ; `illinois_mpfr.so` et `illinois_arb_lowprec.so` ne sont pas chargées à T=20000
12 processus écrivent sur le journal du run (`1w`, `2w`)	tous les processus du calcul, y compris l'enveloppe bash et `systemd-inhibit`, sortent sur le même journal
`systemd-inhibit` avec `/run/systemd/inhibit/140.ref`	confirme le verrou d'extinction et de veille
9 processus avec un descripteur `cuda-uvmfd`	contexte CUDA ouvert dans 9 processus ; VIRT ≈ 29,6 Go par worker dans `htop` (fiche 1.3) est la signature habituelle : **déduit**, non vérifié (D21)
aucune donnée CSV ouverte	à cet instant ; une lecture au démarrage puis refermée ne se voit pas avec `lsof`
10 processus `python3.12`	main et 8 workers font 9 : le dixième n'est pas identifié

Piège (erreur de filtre). `lsof -c python` prend **tous** les processus Python, dont le logger et le watcher du monitoring (lancés par `zeta-mon-on`, dossier courant `~/zeta_captures`) ; le watcher garde en mémoire `matplotlib`, `numpy` et `pandas` en permanence. Pour le calcul, sélectionner par la ligne de commande : `pgrep -f compute_zeros_v16`.

(e) Limites et suite

- `strace` (lectures au démarrage, appels système) est laissé de côté : `zeta_run.sh` appelle des scripts avec `sudo`, et un `sudo` lancé sous `strace` ne prend pas ses droits (**déduit** du fonctionnement de `strace` pour un utilisateur ordinaire, non testé). Piste : attacher `sudo strace -f -p <PID principal>` à un run lancé normalement (D22).
- Un seul run observé par outil, T=20000 : les écritures d'un run plus grand (checkpoints `.partial.csv`, rescan) ne sont pas cartographiées.
- Les outils d'observation sont légers mais ne sont pas gratuits : ne pas chronométrer ces runs (fiche 1.7 bis).

Verdict. OK. La carte des écritures est complète pour un run solo T=20000 ; la découverte la plus utile est le contexte CUDA dans 9 processus (D21).

À retenir (bonnes pratiques « Pendant » 10 et « Après » 14). Tout ce qu'un run produit est dans `calculs/v16_T<N>_<date>/`, `logs/` et `monitoring/runs/` : c'est ce qu'il faut sauvegarder.

Fiche 1.9 — Variantes de lancement : `zeta-mon-exec`, `zeta-mon-start` / `zeta-mon-stop`, détection par le watcher (captures du 25/10, PC1)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Savoir quelles façons de lancer un run existent, ce que chacune suit, et laquelle choisir. Les tests sont faits avec des commandes neutres (sans calcul), sauf le dernier (T=5000).

(a) Rappel : qui suit quoi

Façon de lancer	Qui ouvre le suivi	Code retour	Fenêtres	Rapport
`scripts/zeta_run.sh N`	`zeta_run.sh` lui-même, via `zeta_mon_run.py exec --nom "T<N>" --type ${ZETA_MON_TYPE:-prod} --csv calculs/ --` (ligne 59, **mesuré**)	oui	selon `ZETA_MON_TYPE`	oui
`zeta-mon-exec … -- commande`	`zeta_mon_run.py exec`	oui, celui de la commande	selon `--type`	oui
`zeta-mon-start` puis `zeta-mon-stop --code N`	l'utilisateur	celui qu'on donne à `stop`	selon `--type`	oui
compute_zeros lancé « à la main », sans lanceur	le **watcher** (filet de sécurité)	**inconnu**	non	oui, mais plus pauvre

Conséquence (déduite du code de `demarrer`, non testée). `zeta_run.sh` utilise déjà `exec` : envelopper `zeta_run.sh` dans un second `zeta-mon-exec` donnerait « Un run est déjà suivi » et s'arrêterait. Pour lancer un calcul, passer par `zeta_run.sh` seul.

(b) Test 1 — `zeta-mon-exec` avec un code retour non nul

Commande (PC1) : `zeta-mon-exec --nom T-exec-neutre --type prod -- bash -c 'sleep 15; exit 3'; echo "code shell = $?"`

Observation	Statut
suivi démarré, type prod, fenêtres non	mesuré
`code shell = 3` : le code de la commande ressort jusqu'au shell	mesuré
`rapport.md` généré avant le retour du prompt (`arreter(rc, attendre=True)`)	mesuré
aucun `erreur.txt` ; dossier : `etat.csv`, `figures`, `meta.json`, `metrics.csv`, `rapport.md`, `rapport.pdf`, `session.md`, `superviseur.log`, `superviseur.pid`	mesuré
rapport : `Code retour = 3` (ligne 15), `CSV des zéros : introuvable` (ligne 17), section 3 « Zéros calculés » non générée avec un message qui indique la marche à suivre (`zeta-run-hint NOM CSV` puis `zeta-report`)	mesuré
index : ligne « **échec (code 3)** », durée 0 min 16 s	mesuré

Lecture. `exec` suit **n'importe quelle commande** et dégrade proprement le rapport quand il n'y a pas de CSV de zéros : le code retour, la fiche de session et les mesures des machines restent. Un code non nul est classé « échec » dans l'index.

(c) Test 2 — `zeta-mon-start` puis `zeta-mon-stop`

Commande (PC1) : `zeta-mon-start --nom T-start-neutre --type prod; sleep 10; zeta-mon-stop --code 0 --attendre`

Observation	Statut
suivi ouvert, puis clos avec le code 0 ; `--attendre` bloque jusqu'à la fin du rapport	mesuré
index : « terminé », 0 min 10 s	mesuré
`runs/.actif` supprimé à la fin	mesuré

Différence avec `exec`. Avec `start`, le code retour est **celui qu'on donne** à `stop --code` : si on l'oublie, le suivi reste ouvert jusqu'à la clôture automatique du superviseur (12 h dans `superviseur()`, **lu dans le code**) ; `start --pid PID` clôt le suivi à la fin de ce processus. Un second `start` pendant un suivi actif est refusé.

(d) Test 3 — le watcher détecte un vrai run lancé sans lanceur (D5)

Calcul lancé à la main depuis la racine du dépôt, avec l'environnement de `zeta_run.sh` (`zeta_env`, `PYTHONPATH=…/src`), **sans** `zeta_mon_run.py`, T=5000 :

```
printf '5000\nO\n' | python src/calculs/optimisation/compute_zeros_v16.py
```

Observation	Statut
journal du watcher : `RUN DÉTECTÉ (sans lanceur)` à 18:09:28, le calcul démarre à 18:09:29	mesuré
`RUN TERMINÉ` écrit à 18:09:43, rapport à 18:09:45 (PDF oui) : la fin est confirmée après 3 sondages vides, soit **≈ 6 s de retard**	mesuré (journal) ; 3 × 2 s : déduit du code
`meta.json` : `t1` = 18:09:37, `duree_s` = 9,0, 4 520 zéros, **502 z/s**, CSV retrouvé tout seul	mesuré
`statut` = « terminé (**code inconnu**) »	mesuré
un seul PID (le processus principal) enregistré	mesuré ; cause (PID pris au moment de la détection, avant la création des workers) : déduit
Turing : COMPLET	mesuré
nom du run `compute_zeros_v16` : pas de T dans l'identifiant, type `auto`	mesuré

Lecture. Le watcher fonctionne, mais il fait un rapport **moins riche** : pas de code retour, pas de fiche de session complète, nom peu parlant. Un run de moins de 2 s pourrait être manqué (limite écrite dans l'en-tête du script), ce qui ne gêne pas un calcul réel. Le watcher reste en retrait quand `runs/.actif` existe : c'est pourquoi il n'a rien enregistré pour les runs du jour lancés par `zeta_run.sh`.

Exception à la bonne pratique « Pendant 3 ». Lancer `compute_zeros_v16` directement n'est acceptable que pour un test de très petit T (ici 5000, 9 s) : ce lancement n'arrête pas les services et ne garantit pas l'état du turbo.

(e) Écarts et limites

- Le motif `compute_zeros` reste trop large (E12) : le journal du watcher montre côte à côte le faux run `evince` du 24/10 et le vrai run du 25/10.
- L'en-tête du watcher annonce `--poll 5` ; le code a `2.0` par défaut (E31).
- Chaque test, même neutre, **ajoute une ligne à l'index** `Monitoring-Runs.md` et remplace le bloc « Dernier run avec historique machines » (E30, E32).
- Non testé : `start --pid`, `exec` avec `--fenetres oui`, `zeta-mon-exec` imbriqué dans `zeta_run.sh` (le refus attendu est déduit).

Verdict. OK. Trois variantes testées, comportements mesurés. Recommandation : calcul réel → `zeta_run.sh` ; commande ou suite de commandes quelconque suivie → `zeta-mon-exec` ; run lancé à côté (tmux, `setsid`) → `start` + `stop` ; le watcher est un filet, pas un mode de lancement.

À retenir (bonnes pratiques « Avant » 20 et « Après » 15). `zeta_run.sh` ouvre déjà son suivi : ne pas l'envelopper ; après tout test de monitoring, contrôler que `runs/.actif` a disparu.

Fiche 1.10 — Pannes volontaires : Ctrl+C, `pkill`, sans argument, argument invalide, `zeta-mon-off` (captures du 25/10, PC1)

> **Mesuré en V16** (production = V17 depuis le 03/11, commit `43134dd`) : les durées et comptages de cette fiche ne sont pas des mesures V17.

Objectif. Savoir ce qui reste sur la machine après un arrêt anormal, et comment réparer. Tous les essais sont faits avec T=20000 (ou sans calcul), sur PC1 seul.

(a) Ce que le script fait (lu, mesuré)

Élément de `zeta_run.sh`	**Conséquence**
ligne 11 : `set -e`, aucun `trap` (recherche faite avec `grep`)	toute commande qui échoue, ou tout signal reçu pendant `wait`, arrête le script **sans** passer par la restauration
lignes 13-17 : contrôle « argument présent » **avant** toute action	lancement sans argument : sans effet sur la machine
aucun contrôle du **contenu** de l'argument	abc est accepté
fin du script : `wait "$PID"` puis RET=$?, puis `off --services`, puis message □/□	avec `set -e`, un code non nul de `wait` quitte le script avant RET=$? : la restauration est sautée et le message □ `terminé avec erreur` **ne s'affiche jamais** (confirmé aux tests 2 à 4)

L'état initial, enregistré au début du run dans `/run/zeta_turbo/state`, contient : gouverneur d'origine (`powersave`, EPP `balance_performance`, 4 CPU), swappiness 10, et la liste des **6 services** coupés pendant un run : `cups`, `ModemManager`, `NetworkManager-wait-online`, `snapd`, `fwupd`, `kerneloops`. Il sert de point de retour.

(b) Tableau des pannes

Référence avant chaque essai : 38 services actifs (`systemctl list-units --type=service --state=running --no-legend | wc -l`), turbo `INACTIF`, pas de `runs/.actif`.

Panne	**Code de** `zeta_run.sh`	**Calcul**	**Suivi (monitoring)**	**Services**	**Turbo**	**Index**
Ctrl+C au bout de ~5 s	prompt rendu aussitôt, pas de ligne finale	arrêté (0 processus)	clos tout seul, rapport écrit, code 130	**34** (4 arrêtés)	**ACTIF**	« échec (code 130) », 0 min 05 s
`pkill -f compute_zeros_v16` à 8 s	143	arrêté	**superviseur orphelin** (1 processus), `.actif` **présent**, pas de rapport	**34**	**ACTIF**	aucune ligne tant que le suivi n'est pas clos
Sans argument	1 (« Usage : zeta_run.sh <T_MAX> »)	non lancé	aucun	38	INACTIF	rien
Argument abc	1, pas de □	échoue (3 s)	clos proprement, rapport écrit, code 1	**35** (3 arrêtés)	**ACTIF**	« échec (code 1) », 0 min 03 s
`zeta-mon-off`	—	—	logger et watcher arrêtés (0 processus)	38	INACTIF	—

Lecture. Les pannes qui touchent le calcul laissent la machine dégradée : services coupés et turbo actif. Le monitoring, lui, se clôt tout seul sauf si l'enveloppe `exec` est tuée (cas `pkill`). L'écart 34 / 35 services est **non expliqué** : hypothèse, un service qui se relance à la demande.

(c) Ctrl+C : détails

- Ctrl+C est envoyé à **tout le groupe de processus** du terminal, pas seulement au script : le calcul s'arrête aussi. Mon hypothèse initiale (« le calcul continue seul ») est **réfutée** (0 processus).
- Le journal du run se termine par `nohup failed with exit status 130` (130 = interruption).
- Restes : `logs/run_T20000.pid` (périmé, le run suivant le réécrit : **déduit**) et un dossier de calcul vide `calculs/v16_T20000_20261025_182337/` (avec `checkpoints/` vide).

(d) `pkill -f compute_zeros_v16` : détails

- `pkill -f` filtre sur la ligne de commande : celle de l'enveloppe `zeta_mon_run.py exec … bash -c "…compute_zeros_v16.py"` la contient (E16). L'enveloppe est donc tuée avant d'avoir prévenu le superviseur.
- En mode `exec`, le superviseur n'a pas de PID à surveiller : il continue jusqu'à `.fin` ou 12 h (lu dans `superviseur()`). `.actif` reste en place, ce qui bloquerait un nouveau suivi et le watcher.
- Après `zeta-mon-stop --code 143 --attendre`, le superviseur est clos et le rapport écrit, mais la durée de l'index (**1 min 11 s**) est celle du superviseur, pas du calcul tué à 8 s environ : **déduit**, donc non fiable dans ce cas.
- Pour arrêter un run proprement, préférer Ctrl+C dans le terminal du run à un `pkill` extérieur.

(e) `zeta-mon-off` et `zeta-mon-on`

- `zeta-mon-off` fait deux `pkill` (logger, watcher) et ne touche ni au calcul ni au superviseur.
- `zeta-mon-on` tue puis relance les deux et **écrase** `logger.log` et `watch.log` (> et non >>) : l'historique du watcher est perdu (E33). Copie de sauvegarde faite avant : `~/zeta_captures/1_10_watch_avant_off.log`.
- Run neutre de 20 s pendant l'arrêt : rapport généré sans erreur, section « Machines pendant le run » : « Aucune mesure machine sur cette période (logger arrêté ? `zeta-mon-on`) », pas de `metrics.csv`, `etat.csv` réduit à l'en-tête (**mesuré**).
- Non testé : les valeurs de température lues par le superviseur quand le logger est arrêté mais un CSV de zéros existe (D26).

(f) Procédure de réparation après une panne (testée)

Étape	Commande (PC1)	Résultat attendu
1	`pgrep -f '_superviseur'` et `ls monitoring/runs/.actif`	si un superviseur et `.actif` restent : `zeta-mon-stop --code <N> --attendre` (N = 143 pour un pkill)
2	`sudo scripts/zeta_turbo_off.sh --services`	6 services redémarrés ; « SERVICES RESTAURÉS (turbo CPU maintenu) »
3	`zeta-turbo-off`	« TURBO: RESTAURÉ (vérifié : 4 CPU à leur gouverneur initial, swappiness 10) »
4	contrôle : services = 38, `zeta_turbo_status.sh` : `TURBO_STATUS=INACTIF`	machine revenue à l'état de départ
5	facultatif : supprimer le dossier de calcul vide dans `calculs/` ; l'index garde la ligne « échec »	—

Ne pas lancer l'étape 2 si l'état initial a disparu (`/run/zeta_turbo/state` absent) : la restauration repose sur ce fichier.

Verdict. OK pour la réparation (testée à chaque fois, retour à 38 services et turbo INACTIF). Défaut à corriger dans `zeta_run.sh` : D27.

À retenir (bonnes pratiques « Pendant » 11 et 12, « Après » 16 et 17). Après toute interruption : contrôler le suivi, restaurer les services, puis le turbo, puis vérifier.

4. Partie C — Cas 2 : run distribué PC1 + PC2

Ce cas couvre le calcul réparti sur deux machines : PC1 (i7-7500U, 4 threads) calcule la partie basse de la plage, PC2 (Core 2 Duo E8400, 2 cœurs) la partie haute, le pivot étant choisi par `scripts/zeta_distribute.py`. **Les fiches 2.0 à 2.2 et 2.1 bis ont été établies le 25/10 et le 01/11/2026 avec Claude Code (lecture et mesures sur PC1 et PC2) ; je n'ai pas exécuté ces commandes moi-même : les résultats sont ceux que Claude Code a rapportés et que vous m'avez transmis.** Le rapport détaillé est sur PC1 : `~/zeta_inventaire/DIAGNOSTIC/20261025_PC2_v16_distributed_feasibility.md` (317 lignes au 01/11). Aucun run distribué réel n'a encore été lancé.

Fiche	Cas	Commande clé	Statut
2.0	Contrôle avant un run distribué : état de PC2, imports, cause de la panne du 15/10	`ssh -o BatchMode=yes zeta-calc-second '...'`	fait
2.1	Mesures PC1 contre PC2 en V16 (à T égal, plage complète)	`compute_zeros_v16.py --t-min A --t-max B`	fait (par Claude Code)
2.2	Le script de distribution : corrections, réglages, ce qui reste faux	`zeta_distribute.py --part-pc2`, `--dry-run`	fait (non exécuté en réel)
2.1 bis	Correction de la précision de la V16, script turbo corrigé, comparaison PC1/PC2 après recompilation	`illinois_arb.c` (second pas de Newton), `zeta_turbo_on.sh`	fait (par Claude Code, rapporté)
2.3	Run distribué réel et premier bilan	`zeta-distribute 20000` puis `100000`	**à faire**
2.4	Suivi distribué	`zeta-progress-tmux`, `zeta-log`, `zeta-pid`	à faire
2.5	Pannes (PC2, orchestrateur)	arrêt d'un calcul distant	à faire
2.6	Résultats et validation Turing-Backlund	`grep -i turing` sur le journal du run	à faire

Fiche 2.0 — Contrôle avant un run distribué : état de PC2, imports, cause de la panne du 15/10

Objectif. Savoir si PC2 est prêt avant de lancer un calcul, et pourquoi un run distribué avait échoué sans que personne ne le voie.

(a) État de PC2

Commande lue sur PC1 : `ssh -o BatchMode=yes -o ConnectTimeout=5 zeta-calc-second 'hostname; uptime; nproc; free -h | head -2'`

Observation (25/10 18:46, puis 01/11)	Statut
SSH par clé, sans mot de passe (`BatchMode`) : OK	mesuré
`zeta-calc-second`, 36 jours d'activité (42 le 01/11), charge 0,00 (0,24 le 01/11), **2** CPU, 7,6 Gio de RAM dont 6,3 disponibles	mesuré
aucun `compute_zeros` en cours sur PC2 le 01/11	mesuré
Core 2 Duo E8400, 2 cœurs / 2 threads, **sans AVX2, FMA ni BMI2** ; Debian 12, gcc 12.2 ; FLINT 2.9.0 et Arb 2.23 au niveau du système ; `python-flint` 0.8.0 (FLINT 3.3.1) dans le Python de l'utilisateur	mesuré (Claude Code)
`libflint` 3.3.1 de `python-flint` : **même empreinte SHA-256** (16 premiers caractères) sur PC1 et PC2 ; elle s'exécute sur le Core 2 (2000 appels à zêta sans erreur, aucun opcode illégal dans les journaux du noyau)	mesuré (Claude Code)

(b) La panne du 15/10 : cause et réparation

Constat	Statut
Le run distribué du 15/10 à 15:41 a **planté tout de suite sur PC2** (`ImportError: cannot import name gettz`) ; PC1 a continué seule environ **22 min** avant que l'erreur soit vue ; le dossier de résultats est vide	mesuré (journaux)
`pandas` et `matplotlib` ne s'importaient plus sur PC2 : les dossiers `dateutil` et `packaging` de `/usr/lib/python3/dist-packages` étaient **vides**, sans paquet dpkg	mesuré

Constat	Statut
Cause : le **19/09**, une série de `apt-get purge` et `autoremove --purge` (logiciels de loisirs, `vlc`, `packagekit`…) a supprimé `python3-packaging` (18:15), `python3-dateutil` (18:45) et `python3-pyparsing` ; pandas 3.0.3, `matplotlib` 3.11.0 et `numpy` avaient été installés par **pip** (`--break-system-packages`) et s'appuyaient sur ces paquets apt. `dpkg --audit` ne signale rien	mesuré (journaux apt)
Réparation faite par Claude Code, au niveau utilisateur, sans `sudo` : `pip install --user --no-deps --break-system-packages python-dateutil packaging pyparsing` (dans `~/.local` de PC2). `import compute_zeros_v15` réussit, `ARB_DISPONIBLE` vaut `True`. Retour en arrière : `pip uninstall --break-system-packages python-dateutil packaging pyparsing` (non exécuté)	mesuré

Leçon. Un paquet installé par pip peut dépendre d'un paquet apt ; un `autoremove` le casse sans bruit. Avant de purger sur une machine du cluster : simuler (`apt-get -s autoremove --purge`) et relire la liste. Après : relancer le test d'import.

(c) Le contrôle est maintenant intégré au script

`zeta_distribute.py` exécute désormais, **avant** tout calcul et avant le turbo, un test par SSH (20 s au plus) : `import compute_zeros_v15` et `ARB_DISPONIBLE`. Ce test, lancé en réel, a réussi (mesuré par Claude Code). S'il échoue, le script s'arrête sans rien lancer (fiche 2.2).

(d) Environnement de test sur PC2

Claude Code a créé `~/projet_zeta_v16_test` sur PC2 (12 Mo) : sources de la V16, headers `flint-headers-3.3.1` (24 fichiers), `.so` **compilés sur PC2** (jamais copiés), dépendances Python installées avec `pip --target`. Retour en arrière : `rm -rf ~/projet_zeta_v16_test` (non exécuté). Vérifications de la compilation (mesuré) :

Contrôle	Résultat
`scan_arb.so`	compilation sans avertissement
`illinois_arb.so`	avertissements `-Wunused-parameter` dans les headers `dlog.h` et `dirichlet.h` seulement
`-march=native` sur PC2	`core2` ; aucune instruction `ymm`, `vfmadd`, `mulx`, `adcx`, `adox` dans les deux `.so` (comptage par `objdump`)
`ldd illinois_arb.so`	`libflint`, `libmpfr` et `libgmp` du dossier `python_flint.libs` de PC2 ; ni `libflint-arb` ni `libflint.so.17`

Verdict. OK. La panne du 15/10 a une cause identifiée et réparée ; un test préalable évitera sa répétition.

Fiche 2.1 — Mesures PC1 contre PC2 en V16 : à T égal et sur la plage complète

Objectif. Savoir combien de fois PC1 est plus rapide que PC2, et si ce rapport dépend de la hauteur. Pour fixer le pivot de la distribution, ce rapport est la donnée centrale.

Conditions (mesuré / convenu). `compute_zeros_v16.py --t-min A --t-max B`, 8 workers (valeur écrite en dur), sorties hors du dépôt, un run à la fois. PC1 : turbo de session actif, services arrêtés comme avec `zeta_run.sh`. Critère de calme sur PC1 : **CPU occupé ≤ 10 % sur 3 s** avant chaque run (le *load average* de repos de PC1 vaut 2,2 à 2,6 à cause de deux tâches d'E/S bloquées, donc le critère « charge ≤ 1,0 » est inatteignable). PC2 : charge ≤ 1,0 et CPU ≤ 10 %.

(a) Premiers essais, T = 5000 et 20000 (25/10, PC1 sans turbo)

T	Machine	Zéros	Durée	Débit	Turing
5 000	PC2	4 520	4,9 s	924 z/s	complet
5 000	PC1	4 520	2,6 s	1 710 z/s	complet
20 000	PC2	22 491	37,9 s	593 z/s	complet
20 000	PC1	22 491	18,0 s	1 251 z/s	complet

Rapport PC1/PC2 : **1,85** (T = 5000), **2,1** (T = 20 000) (déduit). Le turbo n'explique pas l'écart (le guide des scripts donne environ 2 % de différence, mesure du 16/10).

(b) Tranches étroites de 5 000 de large, 2 répétitions (01/11)

Tranche	PC	Zéros	z/s (journal)	Remarque
A (95 000–100 000)	PC1	7 679	1 834 et 1 816	série préliminaire : 1 851 et 1 926
A	PC2	7 679	564 et 560	—
B (486 000–491 000)	PC1	8 961	654 et **437**	le second run est perturbé : charge de fond du poste ; série préliminaire : 714 et 684
B	PC2	8 961	171,0 et 171,2	—

Rapport PC1/PC2 : tranche A **3,25 ± 0,01** ; tranche B **2,55 à 3,83** selon la répétition (3,19 ± 0,90), jusqu'à environ 4,0 à 4,2 avec la série préliminaire (déduit : 684 ou 714 divisé par 171). Les deux machines ralentissent d'un facteur environ 3 entre $T \approx 10^5$ et $T \approx 5\times10^5$ (mesuré). La tranche B est **moins sûre** que A à cause du run perturbé.

(c) Plage complète [14, 100 000], 2 runs par machine (01/11)

Run	Machine	Zéros	Durée du journal	z/s	Durée totale	Condition avant le run
D1	PC1	138 069	87,3 s	1 582	188,1 s (anormale)	CPU occupé 4,7 %
D2	PC1	138 069	85,9 s	1 607	107,2 s	CPU occupé 4,2 %
E1	PC2	138 069	226,7 s	609	274,0 s	charge 0,00 ; CPU 1,7 %
E2	PC2	138 069	225,7 s	612	273,4 s	charge 0,98 ; CPU 1,2 %

- Rapport PC1/PC2 : **2,61** (2,598 et 2,628 ; écart-type 0,021). Turing **complet** pour les quatre runs ; 138 069 zéros contre 138 067 attendus par la formule de Weyl (mesuré).
- Durée totale de D1 : 80 s perdues hors des phases du journal, sans cause identifiée. **Hypothèse :** caches froids après le redémarrage de PC1 depuis le 25/10. Ces 80 s ne sont pas utilisées dans les débits.

• La durée totale dépasse la durée du journal de 17 à 43 s selon les runs : c'est la validation de Turing, que la V16 exécute **à chaque run, même sur une tranche** (mesuré / lu). Le verdict de Turing sur une tranche n'a pas de sens (« 130 389 zéros manquants » : il attend les zéros depuis t = 14) : ne pas l'utiliser comme critère.

(d) Lecture : un rapport qui dépend de la hauteur

Plage	Rapport PC1/PC2	Statut
T = 5 000	1,85	mesuré
T = 20 000	2,1	mesuré
[14, 100 000] complète	2,61	mesuré
tranche A, T ≈ 97 500	3,25	mesuré
tranche B, T ≈ 488 500	2,55 à 4,2 (bruyant)	mesuré

Tendance mesurée : le rapport augmente avec la hauteur. **Hypothèse** (non vérifiée) : sur PC2, le temps cumulé de la phase de détection (balayage C compilé pour `core2`) est 4,5 fois celui de PC1, alors que l'affinage `arb_C` n'est que 1,8 fois plus lent : le balayage serait le goulot (D32).

(e) Quel rapport utiliser pour planifier ? (correction de mes propos du 25/10)

J'avais écrit que le « 5,9 » (815 z/s contre 138 z/s en V15) était douteux, parce que les deux débits viennent de plages de T différentes. **C'était vrai pour un rapport « à T égal », mais ce n'est pas le bon critère pour fixer un pivot** : la distribution donne à PC1 la plage [14, pivot] et à PC2 la plage [pivot, T_MAX], donc il faut le rapport des vitesses **moyennes sur la plage propre à chaque machine**. Claude Code l'évalue à environ **5,4 à 6,2** en V16 pour T_MAX = 500 000 (v1 ≈ 1 000 z/s, v2 ≈ 177 z/s ; pivot entre 428 000 et 436 000 ; PC2 recevrait 14 à 16 % des zéros, 12 à 13 % avec la marge de 15 %). Les débits V15 par défaut (815 et 138 z/s) restent donc cohérents **pour un pivot proche de 349 000** ; avec les nouveaux pivots, ils sont périmés.

Gain attendu (déduit, à vérifier) : avec des vitesses constantes de 1 000 et 177 z/s, la durée vaut N/(v1 + v2), soit environ 0,85 fois la durée de PC1 seule : **gain ≈ 15 %**. Claude Code annonce 20 à 25 % avec un coût par zéro croissant, **mais son modèle de coût a échoué à la validation aux basses hauteurs** (27 000 z/s prédits contre 1 710 mesurés à T = 5000) : à prendre comme hypothèse. **Avec 50 % des zéros pour PC2**, la durée serait d'environ 0,5 × 1 000 / 177 ≈ **2,8 fois** celle de PC1 seule (déduit) : l'option soulage PC1 mais rallonge le run.

(f) Reproductibilité et précision (rapporté par Claude Code)

Constat	Statut
D'une répétition à l'autre, les fichiers de zéros sont **identiques bit à bit** sur chaque machine	mesuré
PC1 contre PC2, tranche A : écart maximal 5,8e-11 ; tranche B : 1,16e-10	mesuré
Ces deux écarts valent respectivement 4 et 2 unités du dernier chiffre d'un nombre à virgule flottante à ces hauteurs (2^{-36} ≈ 1,46e-11 autour de 97 500 ; 2^{-34} ≈ 5,8e-11 autour de 488 500)	déduit (calcul vérifié dans ce guide)
Plage complète : 134 284 zéros sur 138 069 sont strictement identiques ; 3 783 (2,7 %) diffèrent de plus de 1e-12 ; **maximum 1,019e-9** (zéro n° 106 074, t = 78 974,82)	mesuré
300 zéros tirés au hasard au-delà de T ≈ 17 000 : l'erreur de position (comparée à Arb 200 bits et à mpmath, qui s'accordent) est **identique sur les deux machines** : médiane 8e-12, 99 % sous 1,4e-9, maximum 3,1e-9	rapporté, non revérifié par moi
Paires de zéros proches : sur 4 zéros, les deux machines sont décalées pareillement de **5e-7 à 1,7e-6** par rapport au vrai zéro, très au-dessus de la tolérance de 1e-12 affichée dans les journaux	rapporté, non revérifié par moi

Lecture. La V16 sur PC2 n'est pas moins précise que sur PC1 : les écarts entre machines sont plus petits que l'erreur propre de l'algorithme. **Mais l'erreur propre dépasse la tolérance annoncée** sur certaines paires proches. **Hypothèse** (non cherchée) : le critère d'arrêt de l'affinage ne porte pas sur le résidu. À vérifier **avant** de présenter ces zéros comme précis à 1e-12 (D28). *Suite : cause trouvée et corrigée, voir la fiche 2.1 bis.*

(g) Nombre de workers sur PC2

N_WORKERS = 8 est écrit en dur dans v15, v16 et v17 (lu). Dans des copies du dossier de test (seule cette constante change), 2, 4 et 8 workers donnent le **même débit à moins de 0,5 %** (tranche A : 561 / 560 / 564 z/s ; tranche B : 172 / 171 / 171 z/s) : le CPU à 2 cœurs est saturé dès 2 processus (mesuré). Ce n'est donc pas un levier de performance ; il pourrait seulement réduire la charge et la mémoire (non mesurées).

Verdict. OK pour les mesures ; **à fiabiliser** par une mesure plus calme sur la tranche B. Décision prise à ce stade : ne pas appliquer de nouveaux débits par défaut tant que le pivot n'est pas pondéré par le coût.

Fiche 2.1 bis — Précision de la V16 : cause, correction, et ce que « 1e-12 » veut dire

Objectif. Comprendre pourquoi des zéros de paires proches étaient décalés de 5e-7 à 1,7e-6 (D28), ce qui a été corrigé le 01/11, et quel objectif de précision est réaliste. *Tout ce qui suit a été mesuré par Claude Code sur PC1 et PC2 ; je ne l'ai pas rejoué (rapporté).*

Réponse directe. Le raffinement de `illinois_arb.c` s'arrêtait trop tôt, puis ne faisait qu'**un** pas de Newton. Un **second** pas de Newton, déclenché quand la pente |Z′| est petite, ramène les erreurs des paires proches à moins d'un pas de nombre flottant. L'objectif « 1e-12 absolu » n'a de sens que jusqu'à t ≈ 16 384 : il faut une tolérance **relative**.

(a) Mécanisme (lu dans le code et mesuré)

• Illinois travaille sur Z_rs (Riemann–Siegel tronquée) et s'arrête quand l'intervalle est plus étroit que **1e-6**, valeur écrite en dur.

• Puis **un** pas de Newton avec Z_arb (au-dessus de t = 20 000 ; deux en dessous), la dérivée venant de Z_rs (donc biaisée).

• Le paramètre `tol` ne fait que sortir plus tôt ; il n'y a **aucun contrôle du résidu**.

• L'erreur finale est corrélée à 1/|Z′| (rang 0,84, mesuré) : une pente faible, donc une paire de zéros proches, amplifie l'erreur.

Correction de ma propre hypothèse (fiche 2.1 f) : je soupçonnais « le critère d'arrêt ne porte pas sur le résidu » ; c'est exact, mais la cause dominante est l'arrêt à 1e-6 suivi d'un seul pas de Newton à dérivée biaisée. L'hypothèse antérieure de Claude Code (« plancher dû à la dérivée de Z_rs ») a été contredite par ses mesures ultérieures.

(b) Correction appliquée

Élément	Avant	Après
Pas de Newton au-dessus de t = 20 000	1	**2 si |Z′| < 3** (constante `SEUIL_DZ_2NEWTON`)
Erreur sur les paires proches	5e-7 à 1,7e-6	réduite d'un facteur **≈ 6 500** (rapporté)
Écart PC1 / PC2 après recompilation sur chaque machine	non comparé	**≤ 1 pas flottant** (rapporté)
Surcoût de temps	—	+5,4 % (PC1), +2,8 % (PC2)
PC2 : même run deux fois	—	résultats identiques (déterministe)

Fichiers modifiés **non commités** (sauvegardes `.bak-20261101`) : `illinois_arb.c` et `illinois_arb.so` (suivis par git). Le `.so` doit être **compilé sur PC2** (`-march=native` donne du code `core2` : 0 instruction `vfmadd` sur PC2 contre 6 sur PC1).

(c) Pourquoi 1e-12 absolu est impossible au-delà de t ≈ 16 384

Un flottant 64 bits a un pas (ulp) de 2^(⌊log2 t⌋−52). Calculé :

t	pas flottant
8 192	1,82e-12
16 384	3,64e-12
70 000	1,46e-11
500 000	5,82e-11
5 000 000	9,31e-10

Dès que la moitié du pas dépasse 1e-12 (t > 16 384 environ), même le zéro exact arrondi ne peut pas être à 1e-12. Le CSV en flottant 64 bits est donc le plancher. **Objectif proposé (à valider par vous)** : erreur ≤ ~1 pas flottant, exprimée en **relatif**. À $5 \cdot 10^6$, cela vaut environ 1e-9 absolu.

(d) Ce qui reste ouvert

Point	Statut
Zéros de pente 3 à 4,4 entre t = $2 \cdot 10^4$ et $6{,}5 \cdot 10^4$: encore 50 à 90 pas d'écart, car le seuil |Z′| < 3 ne les couvre pas	Claude Code enquêtait ; **non conclu**
Précision au-dessus de $5 \cdot 10^6$ et sur les paires les plus serrées (< 0,03 ; la plus serrée à 10^5 : 0,0147)	non mesurée
Le jeu de $5 \cdot 10^6$ (V13 : 10 016 377 zéros, « 96 manquants » avec l'ancien `N_exact` ; **complété le 05/11 à 10 016 474 zéros** par 97 zéros, Turing COMPLET : voir fiche 2.2 bis, 1.3) a été calculé **avant** la correction	précision de position inconnue au-dessus de 10^5
Comptage 136 244 contre 136 479 sur une même plage	**non résolu** ; deux sessions parallèles avec le même label F1 se sont écrasées sur PC2 (E47)

Verdict. Défaut identifié et corrigé sur les cas mesurés ; la tolérance annoncée doit devenir relative (D35). Ne pas présenter les anciens résultats comme « précis à 1e-12 ».

Fiche 2.2 — Le script de distribution : corrections, réglages, et ce qui reste faux

Objectif. Savoir ce que fait `zeta_distribute.py` après les modifications du 25/10, comment imposer la part de PC2, et quelles limites subsistent.

Fichier : `scripts/zeta_distribute.py` (sauvegarde `scripts/zeta_distribute.py.bak-20261025`, ni `git add` ni commit ; `git status` le montre modifié).

(a) Modifications apportées par Claude Code (25/10)

Modification	Détail	Statut du test
`--part-pc2 FRACTION`	impose la part des zéros confiée à PC2 (strictement entre 0 et 1, par exemple 0.5) ; la valeur 1,2 est refusée ; un avertissement apparaît quand PC2 devient le facteur limitant	`--dry-run` : 3 essais conformes (mesuré)
Test préalable de PC2 (`verifier_pc2`)	par SSH, 20 s au plus, avant le turbo et avant tout calcul : `import compute_zeros_v15` et `ARB_DISPONIBLE`	réel : OK (mesuré)
Arrêt croisé (`attendre_deux_runs`)	sonde PC1 et PC2 toutes les 2 s ; dès qu'un des deux échoue, l'autre est arrêté (`os.killpg` sur le groupe de PC1, `pkill` ciblé sur PC2 par son horodatage)	faux processus : PC2 en échec arrête PC1 en 0,2 s ; PC1 en échec déclenche l'arrêt de PC2 (mesuré). **Non testé :** arrêt effectif d'un vrai `compute_zeros_v15.py` distant
Débits par défaut	822,48 / 512,19 z/s (T = 1000, non représentatifs) remplacés par **815,16 / 138,15 z/s** (V15, haut T)	voir fiche 2.1 (e) : périmés pour les pivots actuels

Modification	Détail	Statut du test
Commentaire de `calculer_pivot`	« −5 % » corrigé en « 15 % » (le facteur 0,85 donne −15 %)	lu

(b) Ce qui ne change pas ou reste à faire

• Le script appelle encore `compute_zeros_v15.py` : le run distribué n'utilise donc pas la V16 du Cas 1 (lu).

• **Brancher la V16** coûterait environ 9 endroits liés à « v15 » dans ce seul fichier ; une option `--version v15|v16` dériverait les noms ; les imports ne changent pas (proposition de Claude Code, non appliquée). Pour PC2, voie recommandée : exécuter la V16 dans `~/projet_zeta_v16_test`, où headers et `.so` sont prêts.

• `zeta_sync_pc2.sh` **ne convient pas à la V16** : il ne transfère pas les headers (la règle `--exclude="*"` saute les sous-dossiers) et lie `illinois_arb.so` contre la `libflint-arb` du système (Arb 2.23), mauvaise combinaison pour la V16 (lu).

• **Coût de la V16 par segment :** validation de Turing à chaque segment, soit 17 à 43 s sans option pour l'éviter (mesuré / lu) ; pas de rescan en mode tranche (lu).

• Le pivot reste calculé avec deux vitesses constantes ; Claude Code propose un pivot **pondéré par le coût croissant avec T** (D29).

• **Heuristique** : la fusion déduplique avec une tolérance de 0,01 autour du pivot. D'après la loi des petits espacements de GUE (conjecture de Montgomery), on attend quelques paires distinctes plus proches que 0,01 sur des millions de zéros : le risque par run est faible mais non nul ; à rendre paramétrable (D29).

(c) Ce qui est à faire avant le premier run réel

Étape	Statut
Décider du réglage de la part de PC2 (équilibre en temps ≈ 12 à 16 %, ou 50 % imposés)	décision à prendre
Test d'un vrai run distribué à petit T (20 000), comparé à PC1 seule	à faire (fiche 2.3)
Test d'un échec réel (tuer le calcul de PC2 en cours de run)	à faire (fiche 2.5)
Corriger le script turbo : il arrête `cups`, ce qui arrête `cups-browsed`, mais ne relance que `cups` (E39)	**fait** : corrigé et testé le 01/11 (fiche 2.1 bis)

Verdict. OK pour la lecture et les essais à blanc ; **le run distribué réel reste à faire**.

Fiche 2.2 bis — Mise à jour du 05/11 : ce qui a changé depuis le 01/11, et état des fiches 2.3 à 2.6

Étiquettes : **vérifié** = relu dans le dépôt, `git` ou par `sha256sum` le 05/11 ; **rapporté** = tiré des sessions Claude Code du 01/11 au 05/11, non revérifié.

Sessions non lues : 01/11 (1250, 1256, 1258), 1305 et 0747 lues par `grep`, `.log` de 1158 et 1920 non lus.

2.2 bis-1 Version du pipeline : toute la partie mesurée du guide est en V16

Élément	Guide v0.15	État au 05/11 (rapporté)	Conséquence pour le guide
`zeta_run.sh`	lance `compute_zeros_v16.py`	lance la **V17** (commit 43134dd, 03/11 ; `scripts/zeta_run.sh:56`, vérifié)	les fiches 1.2 à 1.10 ont été mesurées **avant** ce changement : ajouter la mention « mesuré en V16 » ; les durées de référence (T=20000 : 36 s suivies, T=200000 : 6 min 13 s) ne sont **pas** des durées V17
`zeta_distribute.py`	appelle la V15 (fiche 2.2, D31)	appelle la **V17** (`:216, :252, :284, :476`), basculé de V15 à V17 par `931a5bf` (03/11) ; il ne reste qu'un commentaire V15 (`:80`)	D31 : **fait**, mais vers la V17, pas la V16
Tests de référence	V17, alors que la production était V16 (E19, D4)	production = V17	E19 et D4 : **résolus** — **vérifié** (`scripts/zeta_run.sh:56` appelle `compute_zeros_v17.py`)
PC2	V16 dans `~/projet_zeta_v16_test` (fiche 2.0)	a reçu la V17 le 03/11 (18:01), puis `compute_zeros_v17.py` à jour le 05/11	SHA-256 identiques sur PC1 et PC2 pour `compute_zeros_v17.py`, `turing_validation.py`, `consolider_v17.py` (**vérifié** le 05/11, D45) ; mais `illinois_arb.c` diffère sur PC2 (voir 2.2 bis-2)

Remarque : `931a5bf` contient **les deux** (**vérifié** par `git show --stat 931a5bf` : 2 fichiers, 47 ajouts, 35 suppressions) : basculement de `zeta_distribute.py` de la V15 à la V17 (message : « non exécuté en réel »), et dans `compute_zeros_v17.py` `consolider(step=STEP)` et `_marge_bracket` (2 ulp).

2.2 bis-2 Ce qui était « non commité » le 01/11 est commité (D40 résolu)

Commit	Contenu (rapporté)
`180a63a`	second pas de Newton dans `illinois_arb`
`2f5b4be`	correctifs de `zeta_distribute`
`6263b1e`	turbo et `cups-browsed`, avec tests
`d750e3f`	correction de `N_exact` dans `turing_validation`
`1d5b8f1`, `a1708fc`	`CLAUDE.md` ; outils de monitoring
`d4a5bca` (03/11)	correctif de `zeta_distribute` (option `--yes`) ; cherry-picks `e9fe060`, `3c23ebd`, `419a5c8`

Vérifié le 05/11 (D45). PC1 : `illinois_arb.c` identique à HEAD (SHA `187054da…`, 01/11 17:39), `.so` du même instant. **PC2 :** `illinois_arb.c` **différent** (SHA `826af07f…`, 9 149 o contre 11 417 o, daté du 05/07) et `.so` du 02/08 : **PC2 n'a pas le**

second pas de Newton de `180a63a`. **Précision (vérifié le 05/11) :** PC2 a deux arborescences. `~/projet_zeta_v16_test` a un `illinois_arb.c` identique à PC1 (SHA `187054da…`) et un `.so` recompilé le 01/11 à 19:04 ; mais `zeta_distribute.py` utilise `PC2_PROJET=/home/hprzeta/projet_zeta` (`:75`), dont `compute_zeros_v17.py` charge `c_modules/illinois_arb.so` (`:148`, import réel : `ARB_DISPONIBLE=True`) : c'est l'ancien. Les `.so` de PC1 et PC2 ne se comparent pas (architectures différentes). `zeta_sync_pc2.sh` n'a **pas** été lancé : il est inadapté (E48 : pas d'en-têtes 3.3.1, lie contre Arb 2.23 système) et pourrait dégrader le `.so` de PC2. **Corrigé le 05/11 (mesuré) :** `illinois_arb.c` copié sur `/home/hprzeta/projet_zeta` (SHA `187054da…`, identique à PC1), `.so` recompilé sur PC2 avec les en-têtes 3.3.1 et la `libflint` de `python-flint` (`ldd` : `libflint`, `libmpfr`, `libgmp` du dossier `python_flint.libs`), sauvegardes `*.bak-20261105` sur PC2. Run distribué T=1000 (V17) : 649 zéros, Turing COMPLET, **0 écart** avec le run du 03/11 (ancien `.so`). **Limite :** à T=1000 le second pas de Newton ($|Z'| < 3$) n'est probablement pas sollicité : ce test ne prouve pas son effet ; il reste à tester à plus grand T. Les dates des commits du 01/11 sont toutes identiques (23:17:03, réécriture d'historique du 04/10) : elles ne servent pas à les ordonner.

2.2 bis-3 Le jeu de référence $5{\cdot}10^6$ (E50 en partie expliqué)

Question	Réponse (rapporté)	Statut
D'où vient « 97 » ?	dichotomie sur `N_exact` : déficit de 97 sur le CSV V13 (904 Mo), confirmé en 145 s ; script `localiser_manques.py` (7874e73) ; chaque zéro retrouvé vérifié par `mpmath.siegelz` (97 sur 97)	rapporté
Jeu obtenu	V13 + 97 zéros = **10 016 474 zéros**, Turing COMPLET, SHA b3a98a34…, gardé comme référence (décision du 05/11, 19:00)	rapporté
D'où vient « 96 » ?	le chiffre figure dans l'état du projet du 04/07 (« 10 016 377 zéros (96 manquants) ») ; **aucune trace** dans les sessions lues ; présent dans `SKILL.md:183`, `README.md:19`, wiki `analyse_problemes_v13_v15.md` §5	vérifié (dépôt)
Pourquoi 96 puis 97 ?	**déduit** : le « 96 » vient de 10 016 377 calculés sur 10 016 473 attendus (`SKILL.md:183`, README, `analyse_problemes_v13_v15.md`) ; `d750e3f` corrige `N_exact`, faux de −1 si $\mathrm{frac}(\theta/\pi) > 0{,}5$; l'attendu corrigé est 10 016 474 (= 10 016 377 + 97). `N_exact(5·10⁶)` **non recalculé**	déduit

Arithmétique : 10 016 377 + 97 = 10 016 474 (**calculé**) : le total est cohérent avec « +97 ».

2.2 bis-4 Rejeu des 13 isolés et décisions

Élément	Contenu (rapporté)
Rejeu	scan principal V17 13/13, rescan ancien 13/13, rescan décalé STEP/3 13/13 ; aucun gain mesurable ; clos (option 1, hprzeta, 05/11 19:12) ; `RESCAN_PHASE = 1/3` conservé (7e583d4)
Cause plausible	correctifs V17 absents de la V13 (931a5bf, marge de bracket de 2 ulp, `_elargir_bracket`) ; **non testée** (V13 contre V17 non comparés)
Zéros collés à la grille V13	les 13 isolés sont à $1{,}1{\cdot}10^{-7}$ – $5{,}7{\cdot}10^{-7}$ d'un point de grille (KS $p \approx 3{\cdot}10^{-41}$) ; les 84 paires sont uniformes ($p = 0{,}32$)
Non fait (décidé ou reporté)	run V17 complet [14 ; $1{,}77{\cdot}10^6$] (jugé non nécessaire) ; publication du jeu $5{\cdot}10^6$ (décision à part) ; comparaison V13 contre V17 ; documentation V17 dans le dépôt de code
Documentation faite	`Journal.md` et `Handoff.md` (c388cd7), `analyse_problemes_v16_v17.md` §6 (989fd38), `STACK.md` (a725226), wiki, poussés

Le rejeu **ne tranche pas** D1 (le rescan coûte 28 % du run T=200000 pour +0 zéro net) : D1 porte sur le critère de déclenchement, le rejeu sur le gain d'un décalage de phase.

2.2 bis-5 Résultats mesurés depuis le 01/11 à intégrer au guide (rapporté)

Sujet	Résultat	Session source
V16 contre V17, T=200 000	298 028 zéros identiques bit à bit (99,94 %), 172 différents, écart maximal $4{,}72{\cdot}10^{-8}$; les écarts viennent surtout du correctif `illinois_arb` du 01/11, pas de la V17	03/11 matin (`0950_jsonl`)
T=400 000 en V17	640 526 zéros, Turing COMPLET après ajout manuel de 3 zéros hors bracket ; l'élargissement du bracket (`d7fd9ac`) les retrouve ensuite, sans doublon	03/11
Panne de batterie	03/11 vers 09h40 : le run T=400 000 est mort ; rien de perdu côté git	03/11
Patch C `illinois_arb.c`	0 ulp d'écart sur 900 zéros ; **non appliqué**	03/11
Turbo dans `zeta_distribute`	`turbo_off` restaurait l'état initial et supprimait l'état sauvegardé (turbo de session coupé, `TURBO_STATUS=INACTIF`) ; corrigé par `d4a5bca` (`--services` : `zeta_distribute.py:642, 662, 666, 677`). **Risque restant, vérifié dans le code :** rien ne répare l'état initial effacé le 03/11 ; l'état enregistré à 18:07 est celui d'après l'effacement, donc le « bye bye » ne restaure pas forcément l'état d'avant la session (E51)	03/11 18:02

(e) État réel des fiches 2.3 à 2.6 (sessions du 01/11 au 05/11)

Fiche	Statut	Preuve	Certitude
2.3 run T=20000	**fait en V15, le 01/11**, hors sessions lues	dossier `calculs/v15_distrib…` (nom tronqué dans le rapport reçu) : PC1 19 721 zéros, PC2 2 770, fusion 22 491, 1,15 min, 325 z/s, Turing COMPLET, « Manquants : -1 »	run réel : sûr ; session qui l'a lancé : inconnue
2.3 run T=100000	aucune trace depuis le 01/11	seul un TODO apparaît dans `0747`	sûr pour « aucune trace »
2.3 run V17	**fait à T=1000 seulement**, le 03/11	`1751_jsonl` (18:02) : Turing COMPLET à 5 paliers, retour 0 sur PC1 et PC2, 0,14 min ; dossiers `calculs/v17_distribue_T1000_20261103_180221` et `…180745`	sûr
2.4 suivi	aucune trace	aucun des trois noms (`zeta-progress-tmux`, `zeta-log`, `zeta-pid`) dans les sessions du 01/11 au 05/11	sûr pour « aucune trace »

Fiche	Statut	Preuve	Certitude
2.5 pannes	aucune trace	pas de Ctrl+C ni d'arrêt de PC2 liés à `zeta_distribute` ; la coupure de batterie du 03/11 concerne PC1 et un autre run	sûr pour « aucune trace »
2.6 Turing	partiel	COMPLET à T=20000 (V15) et T=1000 (V17) ; pas à T=100000 ; pas de test de panne	sûr pour ce qui est fait

Conséquence. Le premier run distribué de référence du guide existe déjà (V15, T=20000). Il faut maintenant le refaire en **V17**, qui est la version de production, et ne plus planifier la V15.

(f) Décisions à prendre avant le prochain test

#	Décision	Éléments	Mon avis (à valider par vous)
1	Part de PC2	débits par défaut du script (815,16 et 138,15 z/s dans la V15 du 25/10) : PC2 ≈ 14,5 % (**calculé**) ; la V17 peut avoir d'autres défauts	lire la sortie de `--dry-run` : elle donne la valeur réelle ; **premier test avec le défaut**
2	Version testée	**V17** (rapporté), pas V15 ni V16	le dire dans chaque fiche
3	Fichiers non commités	seuls des `.bak-*`, `docs/archive/`, `resultats_v17/` (rapporté)	ne rien commiter pendant les tests ; **STOP** avant tout `git add` ; jamais `git add -A`
4	État turbo	risque E51	noter `zeta_turbo_status.sh` avant, juste après, et après `zeta-turbo-off`

(g) Contrôles avant chaque test (reprise des bonnes pratiques « Avant »)

Une commande à la fois, machine indiquée. Session `script` pour 2.3 et 2.5 ; pas pour 2.4 (`tmux` est plein écran, règle A.2).

N°	Machine	Commande	Ce qu'on vérifie	Pratique
1	PC1	`zeta`	dossier `~/projet_zeta`	Avant 1
2	PC1	`scripts/zeta_turbo_status.sh; echo "code=$?"`	état du turbo, à noter	Avant 2
3	PC1	`zeta-mon-on`	logger et watcher actifs	Avant 3
4	PC1	`pgrep -af 'compute_zeros_v'`	aucun calcul en cours	Avant 4, 9
5	PC1	`df -h ~/projet_zeta /mnt/data`	espace disque	Avant 5
6	PC1	`touch $CAP/avant.stamp`	repère pour `find -newer`	Avant 6
7	PC1	`systemctl list-units --type=service --state=running --no-legend | awk '{print $1}' | sort > $CAP/services_avant.txt`	noms des services avant	Avant 21
8	PC1	`ssh -o BatchMode=yes -o ConnectTimeout=5 zeta-calc-second 'hostname; uptime; nproc; free -h | head -2'`	PC2 joignable et calme	fiche 2.0
9	PC2	`pgrep -af compute_zeros || echo "aucun calcul"`	aucun calcul sur PC2	Avant 4
10	PC1	`git status --short | head`	aucun fichier de code modifié (seuls `.bak-*` et dossiers non suivis attendus)	Avant 11
11	PC1 puis PC2	`sha256sum <chemin>/compute_zeros_v17.py <chemin>/turing_validation.py <chemin>/consolider_v17.py`	mêmes empreintes sur les deux machines (D45) ; chemin sur PC2 : `/home/hprzeta/projet_zeta` (`zeta_distribute.py:75`) ; ajouter `sha256sum c_modules/illinois_arb.c` (différent le 05/11)	nouveau
12	PC1	`pgrep -f 'htop|zeta_run_progress|zeta_proc_sample|zeta_log_live|zeta_plot_'`	rien à côté **si on chronomètre**	Avant 18

Le préflight de PC2 (import et `ARB_DISPONIBLE`, 20 s au plus) est intégré à `zeta_distribute.py` (fiche 2.2, lignes ~622-640) ; ajouter à ces contrôles la comparaison de `illinois_arb.c` entre PC1 et PC2 (PC2 en retard au 05/11).

Fiche 2.3 — Run distribué réel : acquis V15 et V17, puis V17 à T=20000 et T=100000

Objectif. Passer du premier run distribué V15 (01/11) à un run de référence en V17, lire sa sortie, vérifier que PC1 et PC2 restent en bon état.

Acquis (rapporté, non revérifié).

Run	Version	Résultat
01/11, T=20000	V15	PC1 19 721 zéros (≈ 87,7 %), PC2 2 770 ; fusion 22 491 ; 1,15 min ; 325 z/s global ; Turing COMPLET ; « Manquants : -1 »
03/11, T=1000 (deux dossiers)	V17	Turing COMPLET à 5 paliers, retour 0 sur PC1 et PC2, 0,14 min

Part de PC2 dans le run du 01/11 : 2 770 / 22 491 ≈ 12,3 % (**calculé**), cohérente avec la fourchette de 12 à 16 % annoncée à la fiche 2.1 (e).

À faire. V17, T=20000, puis V17, T=100000. Même comptage attendu à T=20000 : 22 491 (**mesuré** en V16 et V15), à confirmer en V17.

Commandes (une à la fois ; **STOP** après la 2) :

N°	Machine	Commande	Rôle
1	PC1	`scripts/zeta_distribute.py --help`	syntaxe exacte (**déduit** : ne lance rien avec `argparse`)
2	PC1	`scripts/zeta_distribute.py <T_MAX> --dry-run`	pivot et parts sans calcul ; ordre des arguments à confirmer
—	—	**STOP** : vous validez pivot, parts et avertissements	—
3	PC1	`time zeta-distribute 20000`	run réel (le `--yes` ajouté par d4a5bca peut changer l'invite : lire `--help`)
4	PC1	`scripts/zeta_turbo_status.sh; echo "code=$?"`	état du turbo **avant** `zeta-turbo-off` (E51)
5	PC1	`zeta-turbo-off`	lire `TURBO: RESTAURÉ`

N°	Machine	Commande	Rôle
6	PC1	`scripts/zeta_turbo_status.sh; echo "code=$?", ls ~/projet_zeta/monitoring/runs/.actif, pgrep -af 'compute_zeros_v'`	état de PC1
7	PC1	`systemctl list-units --type=service --state=running --no-legend | awk '{print $1}' | sort > $CAP/services_apres.txt; diff $CAP/services_avant.txt $CAP/services_apres.txt`	services revenus ?
8	PC2	`pgrep -af compute_zeros || echo "aucun calcul"`	rien d'orphelin
9	PC1	`find ~/projet_zeta -newer $CAP/avant.stamp -type f -not -path '*/zeta_env/*' -not -path '*/.git/*' | head -50`	fichiers écrits par le run

À relever (par run) :

Grandeur	T=20000 (V17)	T=100000 (V17)
pivot ; zéros PC1 / PC2		
durée de chaque partie ; durée totale (`time`)		
zéros après fusion ; doublons retirés près du pivot		
verdict Turing-Backlund ; valeur de « Manquants »		
turbo avant / juste après le run / après `zeta-turbo-off`		
services avant / après (noms)		

Attendu (hypothèses). Comparaison à T=20000 avec le run V15 du 01/11 : même nombre de zéros, durée du même ordre ; l'écart de durée entre versions n'est pas interprétable seul (V15 contre V17). Pour T=100000, pas de prévision (bonne pratique « Pendant » 8). Le coût fixe (SSH, préflight jusqu'à 20 s, Turing) pèse davantage à petit T : à T=20000, **hypothèse** : le gain du partage est petit.

Verdict (grille). Comptage = 22 491 ou écart expliqué ; Turing COMPLET ; PC1 restauré (turbo, services, `.actif`) ; PC2 sans calcul orphelin. Chaque case à remplir après le test.

Résultats mesurés le 05/11/2026 (V17, PC2 avec `illinois_arb.so` à jour).

Grandeur	T=20000 (V17)	T=100000 (V17)
pivot ; zéros PC1 / PC2	17 826,52 ; PC1 [14 ; 17 826,5], PC2 reste (≈ 2 770 prévus)	88 889,19 ; PC1 121 061 prévus, PC2 17 008 mesurés
durée de chaque partie ; durée totale	PC2 0,17 min ; mur 0,53 min (55 s avec `time`)	PC2 0,60 min ; PC1 1,80 min ; mur 1,84 min (2 min 30 s avec `time`)
zéros après fusion	**22 491** (comme en V15 le 01/11)	**138 069**
verdict Turing-Backlund ; « Manquants »	COMPLET ; aucun manquant	COMPLET
turbo après le run	ACTIF (inchangé)	ACTIF (inchangé)
services	« SERVICES RESTAURÉS (turbo CPU maintenu) »	idem

Comparaison à T=20000 avec le run V15 du 01/11 : même nombre (22 491), écart max $3{,}64 \cdot 10^{-12}$ (2 pas flottants), 115 zéros > 10^{-12}. **PC1 seul à T=100000 :** 138 069 zéros en 2 min 40 s ; contre le distribué, 473 zéros diffèrent de plus de 2 pas (0,36 % en zone PC1, 0,23 % en zone PC2), maximum $3{,}35 \cdot 10^{-8}$: l'écart vient de la grille (dépend de `T_MAX`) et non de PC2. **V13 (jeu $5 \cdot 10^6$, $t \le 10^5$) contre V17 :** 138 069 contre 138 069, maximum $3{,}47 \cdot 10^{-8}$, 325 zéros > 10^{-9}, aucun > 10^{-7} (**mesuré**).

À retenir. À $T=10^5$ PC2 termine 3 fois plus tôt que PC1 (0,6 contre 1,8 min) : la part de PC2 (11 %) est trop faible, les vitesses par défaut (815 / 138 z/s) sont à recalibrer (D29, D32). Le gain mur mesuré est 2 min 40 s (PC1 seul) → 1 min 50 s (distribué), soit ≈ ×1,45.

Fiche 2.4 — Suivre un run distribué (V17, T=100000)

Objectif. Voir un run distribué en direct : avancement, charge de PC2, journal, PID.

Piège connu (E53). Le 01/11, `tmux new -s dist20000` a échoué avec « terminal does not support clear ». **Hypothèse** : commande lancée dans un shell sans terminal complet. Lancer `zeta-progress-tmux` dans une vraie fenêtre de terminal, pas dans le terminal intégré d'un éditeur ni dans une session `script`.

Fenêtre	Machine	Commande	Rôle
A	PC1	`zeta-distribute 100000`	le run
B	PC1	`zeta-progress-tmux`	avancement (plein écran)
C	PC1	`zeta-log`	journal
D	PC1	`zeta-pid`	PID (`/tmp/zeta_distribute.pid`) ; « Aucun run actif » ensuite
E (option)	PC1	`ssh -t zeta-calc-second htop`	charge de PC2 (2 CPU)

Ordre : contrôles du § 3.3, puis B et C, puis A, puis D. Ce run n'est **pas chronométrable** (outils ouverts à côté, « Avant » 18).

Observation	Attendu	Statut
PC2 en htop	2 CPU proches de 100 % (8 workers sur 2 cœurs, fiche 2.1 g)	**déduit**
pourcentage affiché	plafond semblable à E21 possible	**hypothèse**, à vérifier
journal	lignes de PC1 et PC2 entremêlées ou séparées ?	**à tester**
`zeta-pid`	un PID pendant le run	**guide**

Mesuré le 05/11 (partiel). `tmux 3.4` est installé. Les alias `zeta-pid`, `zeta-log` et `zeta-progress-tmux` (`~/.bashrc:150-152`) **ne suivent pas les runs actuels** : `zeta_distribute.py` n'écrit pas `/tmp/zeta_distribute.pid` (fichier absent pendant et après les runs), `zeta-log` lit `logs/distribute_T*_nohup_*.log` (le dernier date du 15/10, alors que les runs actuels écrivent `logs/distribute_pc1_*`, `distribute_pc2_*`, `distribute_pivot_*`), et `zeta-progress-tmux` suppose une session `tmux` nommée `zeta-progress` qui n'est pas créée par le script. Le test en vraie fenêtre de terminal (E53) **n'a pas pu être fait** depuis cette session sans terminal.

Verdict. Suivi en direct d'un run distribué : **non validé** ; en attendant, `tail -f logs/distribute_pc1_<horodatage>.log` et `…pc2…` (**mesuré** : les lignes `[Worker n] zéro #… à t=…` s'y écrivent). **À retenir.** Ne pas se fier à `zeta-pid` pour un run lancé directement ; correction à décider (écrire le PID et nommer les journaux comme attendu par les alias).

Fiche 2.5 — Pannes d'un run distribué (V17, T=20000)

Objectif. Tester l'arrêt croisé (faux processus seulement à la fiche 2.2) et l'état laissé par une panne réelle. **STOP** entre deux pannes : réparation et contrôle avant la suivante. Session `script` 2_5 sur PC1.

Panne	Comment	Question	À relever
P1 : PC2 tombe	sur PC2, ~10 s après le début : identifier le PID (`pgrep -af compute_zeros_v17`), puis `kill <PID>` (jamais `pkill -f` : E35)	PC1 s'arrête-t-il, en combien de temps ?	délai ; code retour ; services et turbo de PC1 ; `.actif` ; superviseur ; dossier de calcul vide
P2 : Ctrl+C sur PC1	une seule fois, dans le terminal du run	le calcul de **PC2** continue-t-il ?	`pgrep -af compute_zeros` sur PC2 après 10 s ; turbo et services de PC1 ; **hypothèse** : processus orphelin sur PC2
P3 : préflight en échec	**non recommandé** (casser l'import de PC2)	—	lire le code de `verifier_pc2`

Réparation après chaque panne (fiche 1.10 f, étendue à PC2) :

Étape	Machine	Commande
1	PC1	`pgrep -f '_superviseur'` et `ls monitoring/runs/.actif` ; si présents : `zeta-mon-stop --code <N> --attendre`
2	PC2	`pgrep -af compute_zeros` ; ne tuer que les orphelins identifiés, par PID
3	PC1	`sudo scripts/zeta_turbo_off.sh --services`, puis `zeta-turbo-off` (pas si `/run/zeta_turbo/state` a disparu)
4	PC1	contrôler les noms des services et `TURBO_STATUS=INACTIF`

Rappel E51 : après une panne, l'état turbo enregistré peut ne pas être celui d'avant la session : le contrôle de l'étape 4 est indispensable.

Résultats mesurés le 05/11/2026 (T=100000, V17).

Panne	Observé	Statut
P1 : `kill <PID>` du calcul de PC2 (5 s après le début)	`PC2 a échoué : arrêt de PC1` ; PC1 reçoit SIGTERM (code −15) et s'arrête en **27 s** ; `ret=255` côté PC2 ; aucun calcul orphelin sur PC1 ni PC2 ; turbo ACTIF ; services actifs	**mesuré**
P2 : Ctrl+C (SIGINT au groupe du script)	`KeyboardInterrupt` dans `attendre_deux_runs` (`zeta_distribute.py:542`) ; **PC2 : aucun orphelin** (le `ssh` meurt avec le groupe) ; **PC1 : le calcul continue seul** jusqu'à sa fin (≈ 2 min) ; **les services restent arrêtés** (cups inactif) ; turbo ACTIF	**mesuré** (2 essais ; le 1er envoi du signal a échoué faute de groupe de processus : essai non compté)
P3 : préflight en échec	non fait (déconseillé)	—

Réparation vérifiée : attendre la fin du calcul de PC1 (`pgrep -f 'compute_zeros_v17.py --t-min 14'`), puis `sudo scripts/zeta_turbo_off.sh --services` → « SERVICES RESTAURÉS (turbo CPU maintenu) » ; `TURBO_STATUS=ACTIF`.

Verdict. Panne de PC2 : **bien gérée**. Ctrl+C : **mal géré** (calcul de PC1 non arrêté, services non redémarrés) : cause lue dans le code, pas de `try/finally` autour de l'attente. **À retenir.** Après un Ctrl+C sur `zeta_distribute`, attendre ou arrêter le calcul de PC1 par PID, puis restaurer les services (E54, D48).

Fiche 2.6 — Validation Turing-Backlund d'un run distribué

État. Partiel : COMPLET à T=20000 (V15) et T=1000 (V17), pas à T=100000. Reste à faire : T=100000 en V17, et comprendre « Manquants : -1 » (D46).

N°	Machine	Commande	Rôle
1	PC1	`grep -inE "turing|complet|manquant" <journal du run>`	verdict (journal dans `logs/`)
2	PC1	`wc -l <CSV final>`	nombre de lignes, en-tête comprise (retrancher 1)
3	PC1	`zeta_env/bin/python3 -B -I <dossier>/comparer_zeros.py A.csv B.csv`	comparaison (§ 6 ; **non exécuté** ; dossier hors dépôt tant que le script n'est pas relu)

Rappels : le verdict de Turing n'a de sens que sur une plage qui commence à t = 14 ; l'égalité des totaux ne certifie pas la complétude (fiche 1.5) ; une comparaison entre deux versions mesure la différence entre deux algorithmes, pas une erreur (V16 contre V17, T=200000 : 172 zéros sur 298 200 diffèrent, maximum $4{,}72 \cdot 10^{-8}$, **rapporté**). Zone du pivot : la fusion déduplique à 0,01 près (**heuristique** : risque rare mais non nul, fiche 2.2).

Résultats mesurés le 05/11/2026. Turing-Backlund **COMPLET** sur les runs distribués V17 : T=1000 (649 zéros, 5 paliers), T=20000 (22 491 zéros), T=100000 (138 069 zéros). « Manquants : −1 » du run V15 du 01/11 : non reproduit en V17 ; cause **déduite** (run lancé avant la correction de `turing_validation.py` à 21:54, `turing_validation.py:297` : le −1 est un surplus). Comparaison V16/V13/V17 : voir fiche 2.3.

Verdict. Validation Turing d'un run distribué : **faite** à T=1000, 20000 et 100000 en V17. **À retenir.** Le verdict de Turing ne détecte pas deux erreurs qui se compensent ; la comparaison à une référence (V13, PC1 seul) reste nécessaire pour les positions.

5. Partie D — Bonnes pratiques (version provisoire)

Chaque ligne indique son origine ; elle sera confirmée ou corrigée par les cas de test.

Avant un run

#	Pratique	Origine
1	Se placer dans `~/projet_zeta` (alias `zeta`) avant toute commande à chemin relatif	observé au cas 1.0
2	Lire l'état du turbo (`scripts/zeta_turbo_status.sh`) et le **noter dans la fiche du run** ; `INACTIF` (code 1) hors session est normal	guide, observé au cas 1.0b
3	Lancer `zeta-mon-on` **avant** le run, et après chaque redémarrage de PC1	guide, confirmé au cas 1.0
4	Vérifier qu'aucun calcul ne tourne : `pgrep -af compute_zeros`	observé au cas 1.0
5	Vérifier l'espace disque du dossier de travail : `df -h ~/projet_zeta /mnt/data`	observé au cas 1.0
6	Poser un repère : `touch $CAP/avant.stamp`	cas 1.0
7	Pour un run distribué : préflight PC2 et fichier `~/.config/zeta/cluster_hosts.env`	guide
8	Après une modification du code V17 ou une mise à jour : relancer les tests de référence avec `-B -I` depuis `~/projet_zeta`	observé au cas 1.1
9	Après `zeta-mon-on`, chercher un calcul avec `pgrep -af 'compute_zeros_v'` (le watcher contient `compute_zeros` dans sa propre ligne)	observé au cas 1.2
10	Après un run, vérifier l'état turbo (`zeta_turbo_status.sh`) : `zeta_run.sh` laisse le mode performance actif	observé au cas 1.2
11	Ne jamais lancer `git add -A` : des `.bak*` non suivis traînent dans `scripts/`	observé au cas 1.3
12	Avant de copier un script téléchargé, contrôler sa version (`sed -n 3p fichier | grep "v3.2"`)	observé au cas 1.4
13	Lancer `zeta-proc --intervalle 2 --test <numéro>` **avant** le run ; `zeta-proc-plot` environ 10 s après le début du run	observé aux cas 1.4 et 1.5
14	Lancer `zeta-log-live` (qui attend le journal) avant le run si on veut la fenêtre du journal en direct	validé sur deux runs réels au cas 1.6
15	Pour tout avoir d'un coup (3 fenêtres de monitoring + `zeta-proc` + graphique des processus + journal en direct) : `ZETA_MON_TYPE=test ZETA_MON_EXTRA=oui scripts/zeta_run.sh N`. **Ne pas** lancer `zeta-proc` ni `zeta-log-live` à la main dans ce cas	validé au cas 1.6 bis
16	Les commandes `zeta-mon-*` sont des **fonctions** chargées par `zeta_monitoring_alias.sh` (ligne `source` du `~/.bashrc`), pas des alias : `alias | grep zeta-mon` ne les montre pas, `type zeta-mon-fermer` oui	observé au cas 1.6 bis
17	Contrôler la version de chaque script copié **séparément** : `sed -n 3p a.py b.py` ne montre que la ligne 3 du premier fichier (les deux sont lus comme un seul flux)	observé au cas 1.6 bis
18	Pour **chronométrer** un run (comparer deux versions, valider une optimisation) : aucun outil à côté (ni `htop`, `zeta-progress`, `zeta-proc`, `script`, ni fenêtre) ; vérifier avec `pgrep -f 'htop|zeta_run_progress|zeta_proc_sample|zeta_log_live|zeta_plot_'` ; lire la durée dans le rapport ou l'index. Référence T=20000 : 36 s suivies	mesuré au cas 1.7 bis
19	Après une erreur de syntaxe sur un bloc collé, contrôler ce qui existe (`ls -l`) avant de relancer ; un bloc collé d'un coup est refusé en entier	observé au cas 1.7
20	Pour tester le monitoring sans calcul, nommer le run `T-…-neutre` et savoir qu'il s'inscrira quand même dans `Monitoring-Runs.md`. Ne **pas** envelopper `zeta_run.sh` dans `zeta-mon-exec` : `zeta_run.sh` ouvre déjà son suivi	mesuré / déduit, cas 1.9
21	Comparer la **liste des noms** de services avant et après un run, pas seulement leur nombre : `systemctl list-units --type=service --state=running --no-legend | awk '{print $1}' | sort`. Le nombre varie (38, 39) à cause de services transitoires comme `systemd-timedated`	observé aux cas 1.10 et 2.1
22	Avant un `apt autoremove --purge` sur une machine du cluster : simuler (`apt-get -s autoremove --purge`), relire la liste, puis relancer le test d'import de `pandas` et `matplotlib` (paquets pip dépendant de paquets apt)	mesuré au cas 2.0
23	Sur PC1, le *load average* de repos vaut 2,2 à 2,6 (deux tâches d'E/S bloquées) : pour décider qu'un poste est calme, mesurer le **CPU occupé** (≤ 10 % sur quelques secondes), pas la charge	mesuré au cas 2.1
24	Une seule session Claude Code par chantier ; en lancer plusieurs sur les mêmes machines provoque des collisions de fichiers et de labels (E47)	mesuré au cas 2
25	Exprimer la précision en **pas flottants** (ulp) et en relatif, pas en absolu : 1e-12 est inatteignable au-delà de t ≈ 16 384 (fiche 2.1 bis)	calculé
26	Noter dans la fiche d'un test **la version réellement appelée** (V17 depuis le 03/11) : les mesures du guide v0.15 sont en V16	fiche 2.2 bis (proposition)
27	Comparer les empreintes SHA-256 des fichiers de calcul **et de** `illinois_arb.c` sur PC1 et PC2 avant un run distribué	D45 (proposition)

Pendant un run

#	Pratique	Origine
1	Pas de fenêtres de graphiques sur un gros run de production	guide (gel déjà vécu)
2	Ne pas se fier au débit instantané du monitoring v16 (CSV écrit en fin de worker)	guide
3	Ne jamais lancer `compute_zeros_v*` directement : passer par `zeta_run.sh` ou `zeta-distribute`	guide

#	Pratique	Origine
4	Ne pas lancer d'interface plein écran dans une session `script`	observé en A.2
5	Ne jamais copier un `.so` vers PC2 : recompiler avec `zeta_sync_pc2.sh`	guide
6	Dans htop, filtrer sur `compute` ; H (majuscule) masque les threads, K ceux du noyau ; htop montre aussi les parents du lancement, `zeta-proc` ne suit que le calcul	observé aux cas 1.4 et 1.5
7	Compter les workers **actifs** (CPU ≥ 20 %), pas les PIDs : un worker qui attend existe encore	observé au cas 1.5
8	Ne pas annoncer une durée par extrapolation : T=200000 a pris 6 min 13 s au lieu des 4 min annoncées	observé au cas 1.5
9	Les outils ouverts à côté ralentissent le run (T=20000, durée suivie : 36 s sans outil, 43 s avec `htop`, `zeta-progress`, `zeta-proc` et `script`, 46 s avec des fenêtres de graphiques) : ne pas chronométrer un run observé	mesuré au cas 1.7 bis
10	Pour savoir ce que le calcul ouvre : `lsof -p $(pgrep -f compute_zeros_v16)` ; **jamais** `lsof -c python` (il prend le logger et le watcher du monitoring)	mesuré au cas 1.8
11	Pour arrêter un run en cours : **Ctrl+C dans le terminal du run** (une seule fois). Ne pas utiliser `pkill -f compute_zeros…` : il tue aussi l'enveloppe et laisse un superviseur orphelin	mesuré au cas 1.10
12	Après tout arrêt anormal, ne pas relancer de run avant d'avoir restauré services et turbo (voir « Après » 16)	mesuré au cas 1.10
13	Donner un label unique à chaque run lancé sur PC2 (un label réutilisé écrase le précédent)	mesuré (E47)
14	Pour un test d'arrêt sur PC2 : identifier le PID, agir par PID, sur PC2 seulement	E35 (proposition)

Après un run

#	Pratique	Origine
1	Lire le code retour, puis lister les fichiers créés avec `find -newer`	à tester (cas 1.2)
2	Régénérer le rapport avec `zeta-report`, version `--lang en` pour la version définitive	guide
3	Valider la complétude (Turing-Backlund) : un rapport de monitoring ne la remplace pas	guide
4	`git status` avant tout `git add`	guide
5	Fermer la session de turbo dans la session capturée (`zeta-turbo-off`) et lire `TURBO: RESTAURÉ`	observé au cas 1.5
6	Relire le journal du run : `grep -nE "Worker|RESULTATS|RESCAN|déficit|PROFIL" logs/run_*.log`	observé au cas 1.5
7	Trier des nombres en notation scientifique avec `LC_ALL=C sort -g` (voir E15)	observé au cas 1.3
8	Lancer `zeta-log-graph` : les 6 PNG du journal vont dans `monitoring/runs/prod/<run>/figures/`	observé au cas 1.5 bis
9	Pour revoir un CSV de suivi avec les repères : `zeta-proc-plot <csv_global> --journal <journal>` ; avec `--sortie`, aucune fenêtre ne s'ouvre, seul le PNG est écrit	observé au cas 1.5 bis
10	Fin de session fenêtrée : `zeta-mon-fermer` (« 5 fenêtre(s) fermée(s) » avec `ZETA_MON_EXTRA`, 3 sans), puis `zeta-turbo-off`	observé au cas 1.6 bis
11	Lire `monitoring/runs/<type>/<run>/rapport.md` (généré aussi pour un run de production sans fenêtre) ; sa durée suivie est plus courte que `time` (43 s contre 52 s)	observé au cas 1.6 ter
12	Dans la session capturée : `zeta-turbo-off`, puis `exit`, puis fermer `htop` et `zeta-progress` avec `q`	observé au cas 1.6 ter
13	Régénérer un rapport (`zeta-report`) réécrit `metrics.csv` et les figures depuis le fichier journalier des mesures : ne pas le faire si ce fichier a été supprimé	déduit du code, cas 1.7
14	À sauvegarder après un run : `calculs/v16_T<N>_<date>/`, `logs/run_T<N>_<date>.log`, `monitoring/runs/<type>/<run>/` ; `/run/zeta_turbo/` est volatile ; `Monitoring-Runs.md` est modifié par chaque run (suivi par git)	mesuré au cas 1.8
15	Après un test de monitoring, vérifier que `runs/.actif` a disparu (`ls ~/projet_zeta/monitoring/runs/.actif` doit répondre « Aucun fichier ») : sinon le watcher reste en retrait	mesuré au cas 1.9
16	Réparation après panne : `sudo scripts/zeta_turbo_off.sh --services` puis `zeta-turbo-off` ; contrôler 38 services (`systemctl list-units --type=service --state=running --no-legend | wc -l`) et `TURBO_STATUS=INACTIF`	testé au cas 1.10
17	Si `runs/.actif` et un superviseur subsistent : `zeta-mon-stop --code <N> --attendre` ; la durée de l'index est alors celle du superviseur	mesuré au cas 1.10
18	Le verdict de Turing n'a de sens que sur une plage qui commence à t = 14 : sur une tranche, il annonce des « zéros manquants » sans signification	mesuré au cas 2.1
19	Pour planifier un partage PC1/PC2, utiliser les vitesses moyennes de chaque machine **sur sa propre plage** (rapport 5,4 à 6,2 pour T_MAX = 500 000), pas le rapport à T égal (1,85 à 3,3)	déduit au cas 2.1
20	Après toute modification du code C : recompiler **sur chaque machine** (PC1 et PC2), puis comparer les zéros (écart attendu ≤ 1 pas flottant)	rapporté, fiche 2.1 bis
21	Contrôler **les deux machines** après un test distribué (PC1 : turbo, services, `.actif` ; PC2 : orphelins)	fiche 2.5 (proposition)
22	Relever l'état du turbo avant **et** après `zeta-turbo-off` : l'état initial enregistré peut être faux (E51)	fiche 2.2 bis (proposition)

6. Annexe A — Lexique

Terme	Signification
`nice` / NI / PRI	priorité d'un processus : NI de -20 à 19 ; plus NI est élevé, plus le processus cède la place (19 = le plus effacé). PRI = priorité affichée par le noyau (39 pour NI 19).
`nohup`	empêche l'arrêt du programme à la fermeture du terminal (ignore le signal de déconnexion)
`setsid`	lance le programme dans une nouvelle session, détachée du terminal
load average	nombre moyen de tâches prêtes à tourner sur 1, 5 et 15 minutes

Terme	Signification
thread	fil d'exécution à l'intérieur d'un processus ; htop les affiche en vert dans la vue en arbre
VIRT / RES / SHR	mémoire virtuelle réservée / mémoire réellement résidente / part partagée ; **RES** est la valeur utile
état S	R en cours, S en attente, D attente disque, Z zombie
swap (Swp)	espace disque utilisé comme mémoire de secours ; il doit rester à 0 pendant un calcul
code retour	nombre rendu par une commande (voir A.3)
`pgrep -af`	liste les processus dont la ligne de commande contient le motif (`-f` ligne complète, `-a` l'affiche)
fichier `.stamp`	fichier vide servant de repère de date pour `find -newer`
ControlMaster	mécanisme SSH qui garde une connexion ouverte et la partage entre plusieurs commandes
gouverneur CPU	politique de fréquence du processeur : `powersave` (économie, fréquence variable) ou `performance` (fréquence haute maintenue) ; le turbo de session impose `performance`
EPP	préférence d'énergie transmise au processeur (ici `balance_performance`)
`no_turbo`	interrupteur matériel : 0 = le boost est autorisé, 1 = désactivé
`vm.swappiness`	tendance du noyau à utiliser le swap : valeur basse = l'évite (ici 10)
`power-profiles-daemon`	service du bureau qui gère les profils d'énergie ; il peut réécrire le gouverneur
`python -B`	n'écrit pas de fichiers `.pyc` (aucune trace dans le dépôt)
`python -I`	mode isolé : ignore l'environnement, le dossier courant et les paquets utilisateur ; résultat reproductible
test unitaire, données synthétiques	test sur des entrées fabriquées à la main ; rapide, valide une règle, ne valide pas un calcul réel
worker	processus qui calcule une tranche de t ; plusieurs workers tournent en parallèle pendant un run
STEP	pas du balayage de t pour détecter les changements de signe ; sert d'échelle pour juger si deux candidats sont proches
zone de chevauchement	intervalle de t couvert par deux workers voisins ; un même zéro peut y être rapporté deux fois
checkpoint	fichier intermédiaire écrit par un worker, relu après une interruption
`DUPLICATE_CONFIRMED`, `DISTINCT`, `AMBIGUOUS`	statuts de la consolidation V17 (voir fiche 1.1)
`systemd-inhibit`	commande qui empêche la veille ou l'extinction pendant qu'un programme tourne ; première ligne de l'arbre d'un run
`user` / `real` / `sys`	sortie de `time` : temps de calcul cumulé de tous les processus / durée réelle / temps noyau ; `user / real` donne le nombre moyen de cœurs occupés
phase finale mono-processus	fin de run où un seul processus travaille alors que les autres cœurs sont libres (45 % du run T=100000)
`zeta-proc`	alias de `scripts/zeta_proc_sample.py`, échantillonneur de processus d'un run
`TIME+` (htop)	temps de CPU consommé par le processus depuis sa création
`zeta-proc-plot`	alias de `scripts/zeta_plot_proc.py` : graphique rafraîchi toutes les 2 s à partir du CSV de `zeta-proc`
`--test 1_4`	option de `zeta-proc` : met le numéro du test dans le nom des fichiers produits
wrapper de monitoring	`zeta_mon_run.py` : enveloppe qui lance le calcul et alimente le suivi ; son processus contient le chemin du calcul dans ses arguments mais ne calcule pas
pool	groupe de processus créés ensemble par le processus principal : pool 1 = les 8 workers, pool 2 = les processus du rescan
rescan	nouveau balayage, à pas deux fois plus fin, des segments dont le nombre de zéros trouvés est inférieur à l'attendu
déficit de segment	nombre attendu − nombre trouvé > 0 pour un segment de t
formule de Weyl / Riemann–von Mangoldt	N(T) = θ(T)/π + 1 + S(T) : nombre de zéros d'ordonnée entre 0 et T ; le terme lisse θ(T)/π + 1 sert de nombre « attendu »
S(T)	(1/π)·arg ζ(½+iT) : terme de fluctuation du décompte ; O(log T) (théorème), typiquement de l'ordre de ±1 (constat numérique)
processus actif	processus dont le CPU instantané dépasse 20 % (seuil de `zeta-proc`) ; différent d'un processus qui existe

7. Annexe B — Carte des fichiers et dossiers d'un run

Table à compléter au cas 1.8 (colonne « Vérifié » vide tant que le cas n'est pas fait). Les chemins viennent du **guide** et des captures.

Chemin	Rôle	Écrit par	Vérifié
`~/projet_zeta/monitoring/logs/logger.log`	sortie du logger (ouvert en écriture par le processus du logger)	logger (via `zeta-mon-on`)	oui (cas 1.8)
`~/projet_zeta/monitoring/logs/watch.log`	journal du watcher (contient le message nohup)	watcher	oui (cas 1.0)
`~/projet_zeta/monitoring/data/cluster_metrics_AAAAMMJJ.csv`	une ligne par machine et par seconde	logger	oui (cas 1.0)
`~/projet_zeta/monitoring/runs/prod`, `test`, `historique`	un dossier par run	lanceur de monitoring	oui (cas 1.0)

Chemin	Rôle	Écrit par	Vérifié
monitoring/runs/<type>/<id>/	meta.json, session.md, etat.*, metrics.csv, figures/, rapport.md/.pdf, superviseur.log/.pid, .fin ; monitoring/runs/.actif pendant le run	lanceur et superviseur	oui (cas 1.8)
~/projet_zeta/logs/	run_T<N>_<date>.log, run_T<N>.pid (supprimé en fin de run) ; logs de distribution	zeta_run.sh, zeta_distribute.py	oui pour un run solo (cas 1.8)
~/projet_zeta/calculs/	v16_T<N>_<date>/ : checkpoints/worker_NNN.csv, zeros_v16_*_principal.csv, zeros_v16_*.csv, .png, execution_v16_*.log (4 Go au total selon le guide)	compute_zeros_v16.py	oui pour T=20000 (cas 1.8)
/tmp/zeta_distribute.pid	pid du run distribué	zeta-distribute	non
~/.config/zeta/cluster_hosts.env	adresses du cluster (droits 600, hors git)	à la main	non
~/.ssh/zeta_cm_*	sockets SSH persistants	SSH	oui (cas 1.0, via htop)
~/zeta_captures/	captures script de ce guide	script	oui (cas 1.0)
~/zeta_captures/proc_[<test>_]AAAAMMJJ_HHMMSS.csv, _global.csv et _resume.md	échantillonnage des processus (un fichier par processus et par échantillon, un par échantillon global, un résumé) ; <test> vient de --test	zeta-proc	oui (cas 1.4 et 1.5)
~/projet_zeta/scripts/zeta_proc_sample.py, zeta_plot_proc.py	échantillonneur et graphique de suivi (alias zeta-proc, zeta-proc-plot)	à la main	oui (cas 1.4)
~/projet_zeta/scripts/zeta_log_graphes.py (v1.3), zeta_log_live.py (v1.2)	graphiques du journal et fenêtre en direct (alias zeta-log-graph, zeta-log-live)	à la main	oui (cas 1.5 bis)
~/projet_zeta/scripts/zeta_plot_proc_v1.bak, ~/.bashrc.avant_zeta-log	sauvegardes avant l'installation de la v2 et des alias	à la main	oui (cas 1.5 bis)
monitoring/runs/prod/<run>/figures/g1_…g6_*.png	les 6 graphiques du journal	zeta-log-graph, zeta-log-live	oui (cas 1.5 bis)
~/zeta_captures/outils/	copies de travail (verifier_diff_csv.py, ancienne version de l'échantillonneur)	à la main	oui (cas 1.3)
~/projet_zeta/logs/run_T<N>_<date>.log	journal d'un run (profil des phases, validation Turing)	zeta_run.sh	oui (cas 1.3)
~/projet_zeta/calculs/v16_T<N>_<date>/	zeros_v16_*.csv, graphique .png, execution_v16_*.log	compute_zeros_v16.py	oui (nom lu dans le journal, contenu à explorer au cas 1.8)
monitoring/runs/prod/<date>_T<N>/	suivi et rapport.md d'un run	enveloppe de monitoring de zeta_run.sh	oui (cas 1.3)
monitoring/runs/auto/	runs détectés par le watcher sans lanceur ; contient de faux runs	watcher	oui (cas 1.3)
monitoring/runs/test/<date>_T<N>/	idem prod, pour un run lancé avec ZETA_MON_TYPE=test ; contient aussi figures/ (g1 à g6, debit, gue, metrics, zeros_tranches)	enveloppe de monitoring, zeta-log-live	oui (cas 1.6)
monitoring/logs/fenetre_<nom>.log, extra_proc_sample.log	sortie des fenêtres (live, gue, metrics, proc, loglive) et de l'échantillonneur lancé par ZETA_MON_EXTRA	superviseur	oui (cas 1.6 bis)
monitoring/scripts/zeta_mon_run.py.avant_extra.bak	copie avant le correctif ZETA_MON_EXTRA	à la main	oui (cas 1.6 bis)
~/projet_zeta/scripts/zeta_log_graphes.py.v11.bak, zeta_log_live.py.v11.bak	copies avant la v1.3 / v1.2	à la main	oui (cas 1.6 bis)
~/zeta_captures/1_6d.log, 1_6d.tm, proc_1_6d_20261025_165816*	capture du run 1_6d : terminal, minutage, échantillonnage et résumé de zeta-proc	script, zeta-proc	oui (cas 1.6 ter)
monitoring/runs/<type>/<run>/report_en.md, report_en.pdf, interpretation.md	rapport anglais ; interprétation écrite à la main (incluse dans les deux rapports à la génération)	zeta-report --lang en, à la main	oui (cas 1.7)
monitoring/Monitoring-Runs.md	index de tous les runs ; **suivi par git**, régénéré par zeta-index	zeta-index	oui (cas 1.7)
~/zeta_captures/1_8_*	1_8_inotify.log (112 événements), 1_8_avant.stamp, 1_8_run.out	à la main	oui (cas 1.8)
/run/zeta_turbo/state	état initial du CPU sauvegardé avant le mode performance	zeta_session_turbo.sh	oui (cas 1.2)

8. Annexe C — Écarts constatés entre le guide des scripts et la réalité

#	Cas	Écart	Suite
E1	1.0	procédure : chemin relatif lancé depuis ~/zeta_captures	corrigée (remarque 1, A.2)
E2	1.0	== dernier run == affiche des dossiers, pas des runs	connu du guide ; utiliser ls -1t sur runs/prod
E3	1.0	zeta-pid ne couvre que les runs distribués	utiliser pgrep -af compute_zeros

#	Cas	Écart	Suite
E4	1.0	`zeta-mon-on` affiche déjà le statut : doublon avec `zeta-mon-status`	cosmétique
E5	1.0	`df -h / /mnt/data` ne montre peut-être pas le disque de `~/projet_zeta`	à vérifier avec `df -h ~/projet_zeta`
E6	1.0	en-tête du CSV de mesures non documenté	à lire (remarque 6)
E7	1.0b	`zeta_turbo_status.sh` rend le code 1 quand le turbo est INACTIF	normal d'après le guide : pas une erreur
E8	1.0	la colonne « fréquence » du CSV (3 500 MHz) ne reflète pas l'état du turbo	ne pas l'utiliser ; seule la sortie de `zeta_turbo_status.sh` fait foi
E9	1.1	lignes collées juste après `script` : affichées mais non exécutées	procédure : attendre le prompt (A.2, point 4)
E10	1.1	le tableau des tests coupe les objectifs en milieu de phrase et `test_controle_segments` a des champs vides	cosmétique ; le test est bien compté et PASS
E11	1.1	l'avertissement `fork()` de Python 3.12 et les 188 avertissements « disque plein simulé » rendent la sortie de `test_compute_zeros_v17.py` très longue	normal ; lire la fin (`Ran 9 tests … OK`) ; filtrer avec `col -b < log | grep -E 'Ran|OK|FAIL|ERROR'`
E12	1.3	le watcher crée de faux runs dans `monitoring/runs/auto/` (`evince`, `eog`, `test_compute_zeros_v17`) : motif `compute_zeros` trop large	à confier à Claude Code : motif strict (`compute_zeros_v1[0-9]*\.py` lancé par `python`) ; ne pas supprimer sans décision
E13	1.2	`zeta_run.sh` active seul le mode performance et le laisse actif à la fin (« turbo CPU maintenu ») ; le premier run n'est pas une référence `powersave`	vérifier `zeta_turbo_status.sh` après chaque run ; fermer avec `zeta-turbo-off`
E14	1.3	`zeta-proc` : « CPU cumulé maximal » parfois supérieur à 400 %	artefact de mesure ; corrigé en v2 (horodatage par lecture) ; au run 1_5 le maximum est 385 %
E15	1.3	`sort -g` en locale française classe mal `9.3e-10` (le point décimal n'est pas reconnu) : classement des pires erreurs faux	utiliser `LC_ALL=C sort -g`
E16	1.4	`zeta-proc` v3 comptait le wrapper `zeta_mon_run.py` (ses arguments contiennent le chemin du calcul) : 11 PIDs dans le CSV du test 1_4 dont 1 hors calcul	corrigé en v3.2 (le script exécuté doit lui-même porter le motif) ; revérifié au cas 1.5
E17	1.5	graphique : 10 couleurs pour 11 PIDs, le dernier réutilise la couleur du principal	à corriger (palette de 20 couleurs) ; en attendant, lire le CSV
E18	1.5	le résumé « phase finale à ≤ 1 actif » donne 9 % au run T=200000, alors qu'environ 140 s (38 %, lu sur le graphique) se passent avec 1 à 2 processus actifs sur 4 cœurs	ajouter un seuil réglable (`--seuil-actifs`) ; ne pas conclure sur ce seul chiffre
E19	1.1 / 1.3	`zeta_run.sh` lance `compute_zeros_v16.py` ; les 59 tests de référence portent sur le code V17	les tests n'exercent pas le code de production ; à confirmer avec Claude Code (la V17 est-elle utilisée en production ?)
E20	1.5 bis	`zeta-proc-plot` v2 : la légende du panneau du milieu masque une partie des marches 8 → 6 workers actifs	à corriger (marge haute plus grande)
E21	1.6 / 1.6 ter	`zeta-progress` : le pourcentage ne compte que le dernier pas de 1 000 zéros de chaque worker, d'où un plafond à 99,3 % (T=200000) ou 71,1 % (T=20000) ; « ETA(√t) 0,0 min » est un artefact ; la barre du haut (« 100 % — totalité de T_MAX ») est constante ; « count @ t » est une hauteur sur la droite critique, pas des secondes ; la fin du run (≈ 37 % du temps à T=200000) n'est pas suivie	lire le journal ou `zeta-log-live` pour la fin du run ; à corriger dans `zeta_run_progress.py` (D18 bis)
E22	1.6	`zeta-log-live` v1.0 affichait 138 % de zéros écrits (les zéros des processus du rescan étaient additionnés)	corrigé en v1.1 (filtre `w < 100`) ; vérifié au cas 1.6
E23	1.6	légende du Gantt « détection (balayage) » : la barre bleue est « départ + détection + 1er millier »	légende corrigée en v1.2 ; phrase de la fiche 1.5 bis corrigée
E24	1.6	temps par appel `arb_C` : 4,42 ms à T=20000 contre 2,16 ms à T=200000 (plus cher à petit t)	question (D12)
E25	1.6 bis	`zeta-mon-*` n'apparaissent pas dans `alias` : ce sont des fonctions	documenté (bonne pratique « Avant » 16) ; à signaler dans le guide des scripts
E26	1.7	rapport anglais : restes de français (`actif`, `T moy`, `Go`, « MAJ … lignes », « moderate heights » pour « petites hauteurs ») ; `COMPLET` recopié du journal (voulu)	D19
E27	1.7	la ligne « Rédigée par Claude Code… » de la section 5 est fixe, même quand l'interprétation n'a pas été écrite par Claude Code	D19
E28	1.7	`interpretation.md` (français) est repris tel quel dans le rapport anglais	D19 (`interpretation_en.md`, hypothèse à confirmer dans le code)
E29	1.7 bis	la fiche de session compte les fichiers **non suivis** dans « N fichiers modifiés non commités » (21 le 24/10, 35 le 25/10)	à reformuler (ex. « suivis / non suivis »)
E30	1.8	`Monitoring-Runs.md` est modifié par chaque run (superviseur) alors qu'il est suivi par git : `git status` le montre toujours modifié	décider : ignorer, déplacer ou régénérer à la demande (D23)
E31	1.9	`zeta_run_watch.py` : l'en-tête annonce `--poll 5`, le code a `2.0` par défaut	corriger la documentation
E32	1.9	le run détecté par le watcher n'a ni code retour (« terminé (code inconnu) »), ni T dans son nom, ni fiche de session ; un seul PID (parent) enregistré ; fin confirmée avec ≈ 6 s de retard	utiliser `zeta-run-hint NOM CSV` avant le run, ou préférer `zeta_run.sh` (D25)
E33	1.10	`zeta-mon-on` écrase `logger.log` et `watch.log` (>) : l'historique du watcher (faux runs, détections) est perdu à chaque redémarrage	copier avant ; passer à >> (D27)
E34	1.10	`zeta_run.sh` : `set -e` et pas de `trap` : après Ctrl+C, échec ou signal, services et turbo ne sont pas restaurés ; le message `terminé avec erreur` ne s'affiche jamais (code inatteignable)	réparation manuelle (cas 1.10 f) ; correctif D27

#	Cas	Écart	Suite
E35	1.10	`pkill -f compute_zeros_v16` tue aussi l'enveloppe exec (la ligne de commande contient le chemin) : superviseur orphelin, `.actif` bloqué, durée d'index fausse (1 min 11 s pour un calcul tué à 8 s)	`zeta-mon-stop --code 143 --attendre` ; D27
E36	1.10	argument de `zeta_run.sh` non validé (abc accepté : services coupés avant l'échec)	D27
E37	1.10	après Ctrl+C : dossier de calcul vide dans `calculs/` et fichier `logs/run_T<N>.pid` périmé	nettoyage manuel ; non bloquant
E38	1.10	nombre de services actifs après panne : 34 (Ctrl+C, `pkill`) puis 35 (abc), pour 4 services coupés puis 3	question : un service se relance-t-il à la demande ? Piste : services transitoires (E40)
E39	2.2	le script turbo arrête `cups`, ce qui arrête `cups-browsed` (qui en dépend) ; la restauration ne relance que `cups` : `cups-browsed` reste arrêté (constaté le 01/11, relancé à la main avec `sudo systemctl start cups-browsed`)	corriger le script turbo (D30) ; cause du retour spontané observé le 25/10 : non établie
E40	2.1	le nombre de services actifs de référence n'est pas fixe : 38 ou 39 (`systemd-timedated`, service transitoire sans lien avec le turbo : déduit par Claude Code) ; peut expliquer en partie E38 (hypothèse)	comparer les listes de noms (bonne pratique « Avant » 21)
E41	2.0	run distribué du 15/10 : PC2 en échec immédiat, découvert 22 min plus tard : le script attendait PC1 en entier avant de regarder PC2	corrigé dans `zeta_distribute.py` (test préalable, arrêt croisé) ; non testé en réel
E42	2.2	`zeta_sync_pc2.sh` ne transfère pas les headers 3.3.1 et lie contre Arb 2.23 : incompatible avec la V16	n'est pas utilisé pour la V16 ; à corriger ou remplacer (D31)
E43	2.1	`load average` de repos de PC1 : 2,2 à 2,6 (deux tâches d'E/S bloquées) ; un seuil de charge ≤ 1,0 est inatteignable	critère « CPU occupé ≤ 10 % » (bonne pratique « Avant » 23)
E44	2.1	`TURBO_STATUS=ACTIF` quand des sessions Claude Code sont ouvertes : le hook de session active le turbo, donc l'état « initial » enregistré par `zeta_run.sh` n'est pas `powersave`	**déduit** (3 sessions `claude` ouvertes le 01/11) ; à clarifier avec Claude Code
E45	2.1 bis	le journal affiche une tolérance de 1e-12 absolue, inatteignable au-delà de t ≈ 16 384 (pas flottant supérieur)	tolérance relative (D35)
E46	2.1 bis	`test_illinois_arb.py` : 3 tests échouent ; j'avais supposé des critères à 1e-12, **faux** : critère 1e-8, appels hors domaine (t = 14,1 / 21,0 / 25,0) : défaut du test	corriger le test (D36)
E47	2.1 bis	deux sessions Claude Code en parallèle, même label F1 sur PC2 : écrasement mutuel ; comptages 136 244 / 136 479 en désaccord	une seule session par chantier ; labels uniques (D37)
E48	2.1 bis	`zeta_sync_pc2.sh` ne copie pas les en-têtes et lie contre Arb 2.23 : inadapté à la V16	D31
E49	2.1 bis	erreurs de Claude Code corrigées en cours de route : premier calcul de référence avec 46 NaN (recentrage, 978/978 certifiés), chronomètre à virgule décimale, `sed` cassé	contrôle croisé
E50	05/11	jeu $5 \cdot 10^6$: « 96 manquants » (état du 04/07) contre « +97 » (05/11)	97 expliqué (§ 1.3) ; 96 : **déduit** : `d750e3f` (attendu corrigé 10 016 474) ; fiche 2.1 bis (d) corrigée le 05/11
E51	03/11	`zeta_distribute` : l'état turbo initial enregistré serait celui d'après l'effacement	**vérifié dans le code** : rien ne répare l'état effacé ; noter l'état avant et après chaque test
E52	01/11	run distribué T=20000 (V15) : Turing COMPLET mais « Manquants : -1 » ; aucune session lue ne le relie à `d750e3f`	D46 (cause **déduite** : run à 20:39, `turing_validation.py` corrigé à 21:54, `turing_validation.py:297`)
E53	01/11	`tmux new -s dist20000` a échoué : « terminal does not support clear »	**hypothèse** : lancé depuis un shell sans vrai terminal ; tester `zeta-progress-tmux` dans un terminal ordinaire (fiche 2.4)
E54	05/11	`zeta_distribute` : Ctrl+C → calcul de PC1 non arrêté, services non redémarrés	D48
E55	05/11	alias `zeta-pid`, `zeta-log`, `zeta-progress-tmux` sans effet sur les runs actuels (pas de PID écrit, journaux `distribute_pc*`)	D49

9. Annexe D — Pistes à transmettre (code et science)

Ce guide décrit ; il ne modifie pas le code de calcul. Les points ci-dessous sont à confier à Claude Code (lecture du code) ou à traiter en session de recherche. Étiquette : **mesuré** = fait observé ; **question** = à trancher en lisant le code ou par une mesure.

#	Piste	Fondement	Nature
D1	Critère du rescan : un déficit de ±1 sur un segment déclenche ~104 s (28 % du run) pour +0 zéro net. Comparer le critère à la fluctuation naturelle S(b) − S(a)	fiche 1.5	question (heuristique)
D2	Taille du pool 2 : un processus par segment en déficit (1 à T=100000, 2 à T=200000) ; lire le dimensionnement dans le code	fiche 1.5	question
D3	Équilibrage du pool 1 : écart de 23 % entre le premier et le dernier worker, dû surtout aux 576 appels `mpmath_fallback` du worker 0	fiche 1.5	mesuré
D4	Production = V16, tests = V17 : confirmer lequel est utilisé en production et si les tests devraient viser V16	E19	question

#	Piste	Fondement	Nature
D5	Watcher : motif `compute_zeros` trop large, faux runs dans `monitoring/runs/auto/` ; motif strict `compute_zeros_v1[0-9]*\.py` lancé par `python`. Détection d'un vrai run **vérifiée** au cas 1.9	E12	motif : à faire ; détection : **fait**
D6	Load average de 13,75 à 14,26 avec 8 workers prêts. Hypothèse « plusieurs threads par worker » **réfutée** (1 thread chacun, mesuré). Mesurer avec `vmstat 2` pendant un run (colonnes `r` et `b`). Run `1_6b` : load max 14,83 **sans** fenêtre `zeta-proc-plot`, donc ce n'est pas elle	fiches 1.5, 1.5 bis, 1.6	question
D7	Précision des zéros : à vérifier sur un **échantillon aléatoire** du CSV final (un seul exemple n'est pas représentatif : constaté pendant le cas 1.3)	échanges du 25/10	à faire
D8	Lancement automatique de `zeta-proc` et du graphique avec les fenêtres de monitoring	fiche 1.6 bis	**fait** (v1, option `ZETA_MON_EXTRA`) ; reste à intégrer dans le dépôt (commit) et à documenter
D9	`zeta-proc` : seuil réglable pour la phase finale, palette de 20 couleurs, résumé des paliers (E17, E18)	fiche 1.5	à faire
D10	Journal du run : « Surplus total : 5 » (somme des surplus), estimations d'en-tête périmées (T=10 000 → ~4516 zéros ; T=100 000 → ~49k), « Seuil petit-t : 65.0 » contre `T_SEUIL_PETIT_T=20`	fiche 1.5 bis	question
D11	Principal : 6 threads alors qu'il est quasi inactif pendant le pool 1 ; rôle des 4 s de transition avant le rescan	fiche 1.5 bis	question
D12	Détection : 12,3 s à T=20000, 982 s à T=200000 (×80 pour T×10) ; temps par appel `arb_C` 4,42 ms contre 2,16 ms. Lire la complexité de la détection dans `compute_zeros_v16.py`	fiche 1.6	question
D13	Critère de déclenchement du secours mpmath (1 040 appels à T=200000, 55 % dans W0, nombres identiques d'un run à l'autre) et sa décroissance avec t	fiche 1.6	question
D14	298 192 appels `arb_C` pour 298 200 zéros : 8 appels non expliqués ; durée de la phase petit t variable (16,2 s contre 7,1 s)	fiche 1.6	question
D15	`zeta_plot_proc.py` et `zeta_log_live.py` : ajouter `--geometrie` pour les ranger avec les 3 autres fenêtres (`layout()` de `zeta_mon_common.py`)	fiche 1.6 bis	à faire
D16	Pic de l'histogramme GUE (≈ 1,05) au-dessus de la surmise de Wigner (0,94) à T=20000. Variance de s : 0,1569 (T=20 000), 0,1614 (T=100 000), 0,1693 ($t \approx 5 \cdot 10^6$) pour 0,18 attendu	fiches 1.6 et 1.6 ter	tendance **mesurée** ; cause (effet de hauteur finie) : hypothèse
D18	Écart de +13 % à T=20000 entre la fiche 1.3 (46,0 s) et 1.6 ter (52,0 s)	fiches 1.6 ter et 1.7 bis	**résolu** : outils d'observation ouverts à côté ; la température est écartée ; run de référence `1_6e` : 36 s suivies
D19	`zeta_run_report.py` / `zeta_mon_run.py` / `zeta_mon_common.py` : traduire les restes de français du rapport anglais (`actif`, `T moy`, `Go`, « MAJ … lignes », « moderate heights ») ; ligne « Rédigée par Claude Code » fixe : prévoir l'auteur réel (variable ou option) ; accepter `interpretation_en.md` pour la version anglaise ; « taille finie » → « hauteur finie »	fiche 1.7	à faire
D20	Mesurer le coût de chaque outil séparément (`htop`, `zeta-progress` avec ses SSH vers PC2, `zeta-proc`, `script`) : un run T=20000 avec un seul outil à la fois, 3 essais chacun	fiche 1.7 bis	à faire
D18 bis	`zeta_run_progress.py` : afficher le vrai avancement (pas de 1 000 arrondi par le haut, ou lecture du CSV), distinguer la barre « T_MAX confié » de l'avancement, suivre le rescan	E21	à faire
D17	**À faire à la fin de tous les tests de monitoring** : prompt pour Claude Code afin de documenter dans `Guide_Scripts_Cluster_Zeta.md` tous les scripts ajoutés (`zeta_proc_sample`, `zeta_plot_proc` v2, `zeta_log_graphes`, `zeta_log_live`, correctif `ZETA_MON_EXTRA`, alias `zeta-proc*`, `zeta-log-*`, fonctions `zeta-mon-*`)	demande du 25/10	à faire

#	Piste	Fondement	Nature
D21	9 processus du calcul ont un contexte CUDA ouvert (`cuda-uvmfd`) ; VIRT ≈ 29,6 Go par worker : trouver quelle bibliothèque l'importe, savoir si le calcul de production en a besoin, mesurer le coût (mémoire, démarrage)	fiche 1.8	question
D22	Lectures au démarrage d'un run : attacher `sudo strace -f -p <PID principal>` (sans envelopper `zeta_run.sh`, à cause de `sudo`)	fiche 1.8	à faire
D23	`Monitoring-Runs.md` modifié par chaque run mais suivi par git : le sortir du suivi, ou le régénérer à la demande	fiche 1.8, E30	décision
D24	18 s (38 % du run T=20000) sans écriture de fichier après la fin des workers : confirmer dans le code ce que fait cette phase (rescan, validation, consolidation) et ce qui peut se chevaucher	fiche 1.8	question
D25	Watcher : enregistrer tous les PID du calcul (workers inclus), déduire T de la ligne de commande ou de l'entrée pour le nom du run, faire une fiche de session minimale ; réduire le délai de confirmation de fin	fiche 1.9, E32	à faire
D26	Superviseur : valeurs de température lues dans la fin du fichier de mesures quand le logger est arrêté : restent-elles figées ? (non testé : run sans CSV de zéros)	fiche 1.10	question
D27	`zeta_run.sh` : (1) `trap` sur EXIT/INT/TERM qui restaure services et turbo ; (2) `wait "$PID" \|\| RET=$?` pour que le code retour et le message final soient atteints ; (3) valider T (entier positif) ; (4) nettoyer le dossier vide et le fichier PID. `zeta_mon_run.py exec` : traiter SIGTERM et clore le suivi. `zeta-mon-on` : `>>` pour les journaux. Motif de `pkill` à documenter	fiche 1.10, E33 à E37	à faire
D28	**Précision de la V16** : sur des paires de zéros proches, l'erreur de position atteint 5e-7 à 1,7e-6 (contre 1e-12 annoncé) ; 99 % des zéros tirés au hasard sont sous 1,4e-9, maximum 3,1e-9. Étudier le critère d'arrêt de l'affinage (résidu ?) et mesurer l'erreur sur un échantillon représentatif de paires proches, avant de brancher la V16 dans la distribution et avant toute présentation de résultats « à 1e-12 »	fiche 2.1 (f)	question (rapporté par Claude Code)
D29	Pivot **pondéré par le coût** croissant avec T plutôt que deux vitesses constantes ; tolérance de déduplication (0,01) et débits par défaut rendus paramétrables et documentés ; le modèle de coût doit d'abord être validé aux basses hauteurs	fiches 2.1 (e) et 2.2	à faire
D30	Script turbo : `cups-browsed` ajouté à la liste des services (36 tests OK)	E39	**résolu** le 01/11 (non commité)
D31	Brancher la V16 dans `zeta_distribute.py` (option `--version v15|v16`, environ 9 endroits) ; corriger ou remplacer `zeta_sync_pc2.sh` pour la V16 (headers 3.3.1, lien contre la `libflint` de `python-flint`)	fiche 2.2	à faire
D32	Goulot de PC2 : le balayage C (compilé pour `core2`) coûte 4,5 fois plus que sur PC1, l'affinage `arb_C` 1,8 fois : vérifier les options de compilation (`-march=native` donne `core2`) et le profil de `scan_arb.c` sur PC2	fiche 2.1 (d)	question (hypothèse non vérifiée)
D33	Durée totale de D1 (188 s contre 107 s) : 80 s perdues hors des phases du journal, après le redémarrage de PC1 ; vérifier l'hypothèse des caches froids	fiche 2.1 (c)	question
D34	Run distribué réel à petit T (20 000) puis 100 000 avec le défaut, puis avec `--part-pc2` ; tests d'échec réel (arrêt du calcul de PC2)	fiche 2.2 (c)	à faire
D35	Tolérance **relative** (≤ ~1 pas flottant) dans les journaux et les critères de validation ; décider de l'objectif de précision	fiche 2.1 bis	décision
D36	Corriger `test_illinois_arb.py` (appels hors domaine, critère 1e-8)	E46	à faire
D37	Une seule session Claude Code par chantier ; labels de run uniques sur PC2 ; trancher le désaccord de comptage 136 244 / 136 479	E47	à faire
D38	Seuil du second Newton : étudier les pentes 3 à 4,4 ($t = 2\cdot10^4$ à $6{,}5\cdot10^4$)	fiche 2.1 bis (d)	en cours côté Claude Code
D39	Recalculer ou valider la position des zéros du jeu $5\cdot10^6$ avec la V16 corrigée	fiche 2.1 bis (d)	à faire

#	Piste	Fondement	Nature
D40	Commiter (ou non) les fichiers modifiés : `illinois_arb.c`/`.so`, `zeta_turbo_on.sh`, tests turbo, `zeta_distribute.py`	2.1 bis	décision
D41	E19, D4	version de `zeta_run.sh`	**résolu** : V17 depuis 43134dd (rapporté)
D42	E50	expliquer 96 contre 97	**déduit** : `d750e3f` (voir fiche 2.2 bis, 1.3)
D43	05/11	section « complément $5 \cdot 10^6$ » du guide	à écrire (la session qui devait la faire s'est arrêtée)
D44	05/11	limite du rejeu (petites fenêtres, pas de run complet) à garder visible	décision (hprzeta)
D45	05/11	`.so` compilé contre `.c` commité ; `turing_validation.py` et `consolider_v17.py` sur PC2	**partiellement vérifié le 05/11** : 3 `.py` identiques ; `illinois_arb.c` différent sur PC2 → recompiler (en attente d'un « go »)
D46	E52	comprendre « Manquants : -1 »	**cause déduite** (ancien `N_exact` : run avant la correction de 21:54) ; non rejoué
D47	05/11	publication du jeu $5 \cdot 10^6$, comparaison V13 contre V17, documentation V17 dans le dépôt de code	non faits, en attente de votre OK
D48	`try/finally` dans `zeta_distribute.py` : sur SIGINT/SIGTERM, arrêter le calcul de PC1 et redémarrer les services	E54	à faire (décision)
D49	écrire `/tmp/zeta_distribute.pid`, nommer les journaux comme les alias, ou corriger les alias	E55	à faire (décision)
D50	recalibrer les vitesses par défaut PC1/PC2 (PC2 finit 3 fois plus tôt à $T=10^5$)	fiche 2.3	à faire

9 bis. Annexe E — Script de comparaison de deux jeux de zéros (non exécuté)

Lecture seule. Les CSV du projet ont une colonne `partie_imaginaire`. Le pas flottant au point `a` est `np.spacing(a)`.

```
#!/usr/bin/env python3
"""Compare deux fichiers de zéros (colonne partie_imaginaire). Lecture seule."""
import sys
import numpy as np
import pandas as pd


def lire(chemin):
    """Retourne les positions des zéros, triées."""
    t = pd.read_csv(chemin, usecols=["partie_imaginaire"])["partie_imaginaire"]
    return np.sort(t.to_numpy(dtype=float))


a, b = lire(sys.argv[1]), lire(sys.argv[2])
print(f"zéros : {len(a)} contre {len(b)}")
if len(a) == len(b):
    ecart = np.abs(a - b)
    ulp = np.spacing(a)  # pas flottant au point a
    print(f"écart max : {ecart.max():.3e} (soit {(ecart / ulp).max():.1f} pas flottants)")
    print(f"écarts > 1e-12 : {(ecart > 1e-12).sum()} ; > 2 pas flottants : {(ecart > 2 * ulp).sum()}")
else:
    print("comptes différents : comparer par plages de t, pas élément par élément")
```

Avant usage : le tester sur deux fichiers connus, par exemple les CSV `v15_T14_100000_cmpv15v16` et `v16_T14_100000_cmpv15v16` (138 069 lignes chacun).

10. Historique du guide

Version	Date	Contenu
v0.16	05/11/2026 (nuit : runs distribués T=20000 et 100000, pannes P1/P2, sync PC2, `docs/V17.md`)	guide mis en regard du code du 05/11 : production et distribution en V17 (43134dd, 931a5bf), mesures du guide étiquetées V16, modifications du 01/11 commitées, jeu $5 \cdot 10^6$ = V13 + 97 zéros (10 016 474), rejeu des 13 isolés sans gain ; fiches 2.3 à 2.6 : acquis (T=20000 en V15 le 01/11, T=1000 en V17 le 03/11) et plan des tests restants ; E50 à E53 ; D31, D40, D41 résolus, D42 à D47 ; bonnes pratiques « Avant » 26-27, « Pendant » 14, « Après » 21-22 (propositions) ; script de comparaison ; vérifications du 05/11 : 931a5bf (les deux), SHA-256 des 3 `.py` identiques PC1/PC2, `illinois_arb.c` de PC2 en retard, cause probable du « -1 » ; guide unique `md/Guide_Run_Zeta.md`, copies archivées
v0.15	01/11/2026	fiche 2.1 bis : cause de l'imprécision de la V16 (arrêt à 1e-6 puis un seul pas de Newton), second pas de Newton si $\lvert Z' \rvert < 3$ (erreurs ÷ 6 500, PC1/PC2 ≤ 1 pas flottant), pas flottant selon t, tolérance relative ; script turbo corrigé (E39, D30 résolus) ; mes erreurs corrigées (tests, tolérances absolues) ; E45 à E49 ; D35 à D40 ; bonnes pratiques « Avant » 24-25, « Pendant » 13, « Après » 20

Version	Date	Contenu
v0.14	01/11/2026	Cas 2 : fiche 2.0 (état de PC2, cause de la panne du 15/10 : `apt autoremove` du 19/09, réparation), fiche 2.1 (mesures PC1/PC2 en V16 : 1,85 à 3,25 à T égal, 2,61 sur [14, 100 000], 5,4 à 6,2 pour planifier ; précision et paires proches), fiche 2.2 (modifications de `zeta_distribute.py`, limites) ; correction de mon avis sur le « 5,9 » ; E39 à E44 ; D28 à D34 ; bonnes pratiques « Avant » 21-23, « Après » 18-19
v0.13	25/10/2026	fiche 1.10 : pannes volontaires (Ctrl+C, `pkill`, sans argument, argument invalide, `zeta-mon-off`), tableau des conséquences, procédure de réparation testée, `set -e` sans `trap` ; E33 à E38 ; D26, D27 ; bonnes pratiques « Pendant » 11-12, « Après » 16-17
v0.12	25/10/2026	fiche 1.9 : variantes de lancement (`zeta-mon-exec` avec code retour 3 classé « échec », `zeta-mon-start` / `zeta-mon-stop`, détection d'un vrai run par le watcher : 9 s, 502 z/s, code inconnu) ; `zeta_run.sh` utilise déjà exec ; E31, E32 ; D5 mis à jour, D25 ; bonnes pratiques « Avant » 20 et « Après » 15
v0.11	25/10/2026	fiche 1.8 : cartographie des écritures d'un run (`inotifywait` installé, `find -newer`, `lsof`), chronologie d'un run T=20000, carte des écritures, bibliothèques chargées, contexte CUDA dans 9 processus ; annexe B vérifiée ; E30 ; D21 à D24 ; bonnes pratiques « Pendant » 10 et « Après » 14
v0.10	25/10/2026	fiche 1.7 (rapports FR et EN, `zeta-index`, `interpretation.md`, défauts du rapport anglais E26 à E28) ; fiche 1.7 bis (les outils d'observation ralentissent le run : 36 s sans outil contre 43 s avec ; hypothèses code et température écartées ; surcoût constant de 9 s de l'enveloppe) ; D18 résolu, D19 et D20 ; bonnes pratiques 18, 19, « Pendant » 9, « Après » 13 ; correction de la fiche 1.6 ter (+13 % expliqué)
v0.9	25/10/2026	fiche 1.6 ter : run T=20000 en production sans fenêtre (52,0 s ; coût des fenêtres 3 à 4 s), lecture du `rapport.md`, correction de l'explication de `zeta-progress` (plafond = pas de 1 000), variance des espacements selon la hauteur (0,157 → 0,161 → 0,169), pistes D18 et D18 bis, bonnes pratiques
v0.8	25/10/2026	fiche 1.6 (run test T=20000 avec 3 fenêtres ; run prod T=200000 pour valider `zeta-log-live` en direct ; phases d'un run ; lecture de `zeta-progress` ; reproductibilité 1_5 / 1_6b) ; fiche 1.6 bis (lancement automatique `ZETA_MON_EXTRA`, validé en réel) ; kit v1.3 / v1.2 ; écarts E21 à E25 ; pistes D12 à D17 ; correction de la légende « détection »
v0.7	25/10/2026	fiche 1.5 bis : six graphiques du journal (`zeta-log-graph`), fenêtre en direct (`zeta-log-live`), repères du journal sur `zeta-proc-plot` v2, première mesure d'alignement (0,8 s, ±1,3 s), lecture du graphique du run 1_5, hypothèse « threads » réfutée (1 thread par worker), écart E20, pistes D10 et D11, bonnes pratiques
v0.6	25/10/2026	fiche 1.4 (graphique de suivi, nommage `--test`, comptage des processus, écart E16) ; fiche 1.5 (run T=200000 : 6 min 13 s, structure pool 1 / rescan / fin séquentielle, journal du run, critère du déficit, interprétation S(T)) ; écarts E15 à E19 ; annexe D ; plan des fiches renuméroté (1.6 à 1.10)
v0.1	24/10/2026	modèle de fiche, mise en place, fiche 1.0 (interprétation du terminal et de htop), plan des fiches 1.1 à 2.5, bonnes pratiques provisoires, lexique, carte des fichiers, écarts
v0.2	24/10/2026	reprise 1_0b : état du turbo (INACTIF, powersave), correction de l'indice fondé sur la fréquence, verdict de la fiche 1.0 mis à jour, lexique et écarts complétés
v0.5	25/10/2026	fiches 1.2 (premier run) et 1.3 (suivi en direct) : turbo actif par défaut, séries de mesures, lecture de htop, outil `zeta-proc`, phase finale à 45 %, chronologie, pas de doublon de monitoring, écarts E12 à E14
v0.4	24/10/2026	fiche 1.1 complète : test_compute_zeros_v17 (9 OK), test_zeta_turbo (34 OK), familles de tests, remarques GPU, fork, disque plein simulé, verdict 59 tests
v0.3	24/10/2026	fiche 1.1 : test_consolider_v17 (16 PASS), options `-B -I`, lecture du tableau de tests, leçon du collage après `script`, lexique et écarts complétés

Mis à jour : 2026-11-05 — 2050 lignes